



---

# APUNTE EXTRA OFICIAL DE ARQUITECTURA DE COMPUTADORAS

---

Hecho por: Nicolás Aguirre



2020

## Sistemas de numeración

**Definición de sistemas de numeración:** es el conjunto de reglas que permiten nombrar y escribir cualquier número a partir de un número finito de símbolos.

Definiciones varias pero importantes:

- I. **Cantidad o Número:** conjunto o grupo con una cardinalidad determinada y exacta de objetos o elementos.
- II. **Símbolo o signo:** representación (escrita, visual, auditiva, etc...) que puede usarse para hacer referencia a una cantidad o número.
- III. **Numeral:** conjunto o grupo escrito de símbolos, ordenado de una manera determinada y única, que representa una cantidad.
- IV. **Dígitos:** cantidad o número de símbolos o signos escritos de un numeral.

Definición: un sistema numérico está formado por:

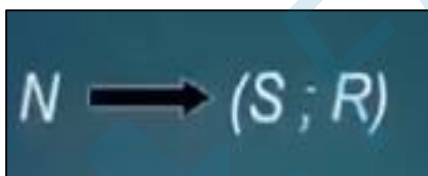
Un conjunto finito y no vacío de símbolos (ALFABETO)

Un conjunto de reglas para formar con los símbolos los numerales.

Un conjunto de reglas para operar entre los numerales

Representación gráfica de un sistema de numeración

Un sistema de numeración se puede representar como:



Donde:

**N** es un sistema de numeración como por el ejemplo el decimal o binario, está compuesto por un conjunto de símbolos permitidos en el sistema **S** y **R** son las reglas válidas para indicar la validez o no de los números que conforman el sistema de numeración.

Base de un sistema de numeración

El número de símbolos permitidos de un sistema de numeración posicional se conoce como la **base** del sistema de numeración.

Si un sistema de numeración posicional tiene una base  $b$  significa que disponemos de  $b$  símbolos para describir a un número.

Ejemplos:

Sist. decimal son  $\{0, 1 \dots 9\}$ ; BASE : 10  
 binario son  $\{0, 1\}$ ; BASE: 2  
 octal son  $\{0, 1 \dots 7\}$ ; BASE: 8  
 hexadecimal son  $\{0, 1 \dots 9, A, B, C, D, E, F\}$ ; BASE : 16

En todos los sistemas de numeración combinar el 1 con el 0 conforman la base del sistema.

Clasificación de los sistemas de numeraciónPosicionales

Los posicionales o ponderados son lo que más nos importan a nosotros porque son los que más usamos.

**El valor de un dígito depende del símbolo utilizado y de la posición que este símbolo ocupa en el número.**

Ponele en el sistema decimal y elijamos un símbolo, el 5, tiene valor propio (5 unidades) pero si escribís el número 53 **el 5 está en el lugar de las decenas**, entonces **está en una posición donde su peso es de 5 decenas**, o sea que es 50 lo que representa el 5 en ese lugar.

El número como el 145, el 5 representa ahí 5 unidades, el 4 tiene su valor propio (4 unidades) pero **en esa posición** representa 40 unidades ya que **ocupa el lugar de las centenas**.

No posicionales

**Los dígitos tienen el valor del símbolo utilizado y no dependen de la posición que ocupa el número.**

Ejemplo: números romanos, donde la X vale 10 y da igual donde la pongas, siempre vale 10, V vale 5 y así sucesivamente.

Escribir en esos sistemas es muy complicado por eso se descartó su uso.

Características de los sistemas numéricos posicionales

El sistema debe tener por lo menos dos símbolos, uno de ellos que represente la ausencia de elementos o **cantidad cero (0)** y otro que represente la unidad o **cantidad uno (1)**.

Los símbolos que posea el sistema deberán poder ordenarse de forma monótona y creciente y representar consecutivamente a las cantidades **cero (0)**, **uno (1)**, **dos (2)**, **tres (3)**,... etc...

La cantidad ***b*** de símbolos del sistema recibe el nombre de **base del sistema** y el sistema mismo recibe por este motivo el nombre de **sistema de numeración en base *b***

Teorema fundamental de la Numeración

Está áspero pero se va a ir entendiendo bien a medida que se avanza la lectura, no se detengan.

**Dado un sistema de numeración de base B con B > 1 cualquier número natural N puede descomponerse de la forma siguiente:**

$$N = \underbrace{d_{n-1} \dots d_1 d_0}_{\text{Parte entera}} , \underbrace{d_{-1} \dots d_{-k}}_{\text{Parte después de la coma}}$$

$$\underbrace{d_{n-1} \cdot b^{n-1} + d_{n-2} \cdot b^{n-2} + \dots + d_1 \cdot b^1 + d_0 \cdot b^0}_{\text{Parte entera}} + \underbrace{d_{-1} \cdot b^{-1} + \dots + d_{-k} \cdot b^{-k}}_{\text{Parte después de la coma}}$$

Parte entera

Parte después de la coma

Es preferible que termine la parte entera en  $n-1$ , porque siempre la parte entera arranca con un dígito que tiene el subíndice 0, siendo el mismo el menos significativo (por ejemplo el 5 en el 145, es el que menos incidencia tiene en el valor del número), a ese dígito siempre se la llama 0, no 1.

**Nosotros tenemos un total de  $n$  dígitos al descomponer donde  $n$  es el número de dígitos de la parte entera. En nuestro ejemplo del 145 el 5 es el dígito 0, el 4 es el dígito 1 y el 1 es el dígito 2. En total hay 3 dígitos, por tanto  $3 - 1 = 2$  que es el número del dígito 1 del 145.**

**Cuando dice “ $d$ ” hace referencia a cualquiera de los símbolos permitidos del sistema de numeración que queremos representar.**

Los números también se pueden representar de la forma polinómica, siendo este un polinomio de potencias de la base, **se escribe poniendo el dígito n-1 multiplicado por la base elevado al mismo número n-1 + dígito n-2 elevado al mismo número n-2**. Y así sucesivamente hasta llegar a la base a la 0 que nos da como resultado 1 y luego de eso finaliza la parte entera. En la parte fraccionaria de la representación en la forma polinómica se escribe realizando el mismo cálculo pero en vez de ir desde el dígito más significativo arrancamos desde el menos hacia el más, sería como hacerlo al revés.

**N** = número válido en el sistema de numeración

**b** = base del sistema de numeración representado (número de símbolos permitidos en el sistema)

**d** = símbolo cualquiera de los permitidos en el sistema

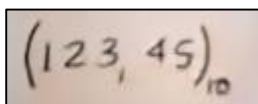
**n** = número de dígitos de la parte entera

**,** = coma fraccionaria. Símbolo utilizado para separar la parte entera de un número de su parte fraccionaria

**k** = número de dígitos de la parte decimal

### Ejemplo de representación en la forma polinómica

Supongamos que queremos representar este número:



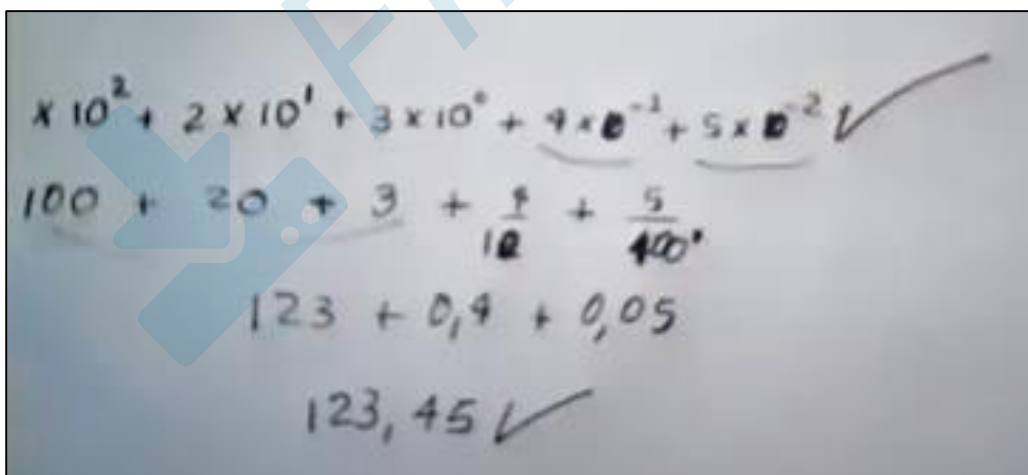
$$(123,45)_{10}$$

El número 10 representa la base del sistema...

En este caso  $n = 3$ , ya que la cantidad de dígitos en la parte entera es 3 (123)

Entonces escribamos (leer al mismo tiempo que se analiza la imagen)

**1** (que sería el dígito 2) por la base que es 10 elevado a la 2 (el dígito que representa) + el **segundo dígito** que es 2 multiplicado por 10 a la 1 (note que el polinomio va decreciendo) + 3 multiplicado por 10 elevado a la 0 (que es 1) y después sigo sumando, sin necesidad de poner la coma + 4 multiplicado por 10 elevado a la -1 más 5 por 10 elevado a la -2.



$$\begin{aligned}
 & 1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0 + 4 \times 10^{-1} + 5 \times 10^{-2} \checkmark \\
 & 100 + 20 + 3 + \frac{4}{10} + \frac{5}{100} \\
 & 123 + 0,4 + 0,05 \\
 & 123,45 \checkmark
 \end{aligned}$$

Perdonen la mala calidad de la imagen. Con un poco de ganas se llega a entender.

Pasaje de un número que está en base 10 a otra base cualquiera (Demostración)

$$\begin{array}{r}
 N \quad | \quad b \\
 \hline
 x_0 \quad c_0 \quad | \quad b \\
 \hline
 x_1 \quad c_1 \quad | \quad b \\
 \hline
 x_2 \quad c_2 \quad \vdots \\
 \vdots \\
 c_{n-2} \quad | \quad b \\
 \hline
 x_{n-1} \quad c_{n-1} \quad | \quad b \\
 \hline
 x_n \quad c_n
 \end{array}$$

Yo tengo un número  $N$  que está en decimal y lo divido por una base cualquiera a la que lo quiera pasar. Cuando hago la división me obtengo un primero cociente  $c_0$  y un resto  $x_0$ , dicho resto será el primer dígito (el más significativo) de la otra base que salió, posteriormente divido el cociente  $c_0$  que tenía antes nuevamente por la base obteniendo otro cociente  $c_1$  y un nuevo resto  $x_1$  que será el siguiente dígito y así sucesivamente se harán divisiones siempre repitiendo este mismo patrón. El último cociente y todos los restos formaran el número nuevo.

Como ustedes sabrán hay una forma de demostrar (esto viene del primario) que la división fue realizada correctamente. Esto es expresando que el dividendo es igual a el cociente por el divisor más el resto. Precisamente esto se puede aplicar en cada instancia de las sucesivas divisiones que ocurrieron para hacer esta conversión.

Si nosotros sustituimos el valor de E2 en la parte del cociente en E1 y hacemos la distributiva obtenemos la siguiente expresión:

$$\begin{aligned}
 N &= c_0 b + x_0 & [E.1] \\
 c_0 &= c_1 b + x_1 & [E.2] \\
 c_1 &= c_2 b + x_2 & [E.3] \\
 &\dots \dots \dots \\
 c_{n-2} &= c_{n-1} b + x_{n-1} & [E.4] \\
 c_{n-1} &= c_n b + x_n & [E.5]
 \end{aligned}$$

Si sustituimos el valor de la expresión [E.2] en [E.1] tenemos que:

$$N = (c_1 b + x_1) b + x_0 = c_1 b^2 + x_1 b + x_0$$

Podemos repetir el proceso con cada ecuación que se haya dado para llevar a cabo dicha sucesión de divisiones llegando a la fórmula original polinómica:

sustituyendo ahora la expresión [E.3] resulta:

$$N = (c_2 b + x_2) b^2 + x_1 b + x_0 = c_2 b^3 + x_2 b^2 + x_1 b + x_0$$

si repetimos el proceso con las demás ecuaciones, llegaremos a:

$$N = x_{n-1} * b^{n-1} + \dots + x_2 * b^2 + x_1 * b + x_0$$

La parte fraccionaria como tal no tiene una demostración, solo se usa el concepto práctico que escapa a este apartado que solo desarrolla el aspecto teórico de la materia.

## Tamaños usados para representar el peso de archivos en la PC

### MÚLTIPLOS DEL BIT

- Nibble: conjunto de 4 bits (1010)
- Byte: conjunto de 8 bits (10101110)
- Kilobyte: conjunto de 1024 bytes ( $1024 * 8$  bits)
- Megabyte: conjunto de 1024 Kb ( $1024^2 * 8$  bits)
- Gigabyte: conjunto de 1024 Mb ( $1024^3 * 8$  bits)
- Terabyte: conjunto de 1024 Gb ( $1024^4 * 8$  bits)

1024= es el múltiplo de 2 más próximo a 1000.     $2^{10}=1024$

## Distintos tipos de sistemas numéricos

### **Sistema decimal**

Proviene del sistema numérico indo arábigo, es un sistema posicional de conjunto de símbolos cuyo significado valor depende de su posición relativa al punto decimal. Tiene base 10 y sus dígitos van del 0 al 9.

### **Sistema binario**

Es el sistema utilizado internamente en los circuitos digitales que configuran el hardware, su base es 2 y los posibles dígitos que puede usar son el 0 y el 1. En la forma polinómica al multiplicas por la base que es 2

### **Sistema octal**

Es un sistema posicional donde la base es 8, la aritmética es similar a los sistemas binarios y decimal, y abarca del 0 al 7. En la forma polinómica al multiplicas por la base que es 8

### **Sistema hexadecimal**

Es un sistema posicional, cuya base es 16, con aritmética similar a los anteriores y tiene símbolos del 0 al 9 y luego de ese sigue A B C D E F, que representarían al 10, 11, 12, 13, 14, 15,16. En la forma polinómica al multiplicas por la base que es 16

• 0 1 2 3 4 5 6 7 8 9 A B C D E F

## Representación números enteros

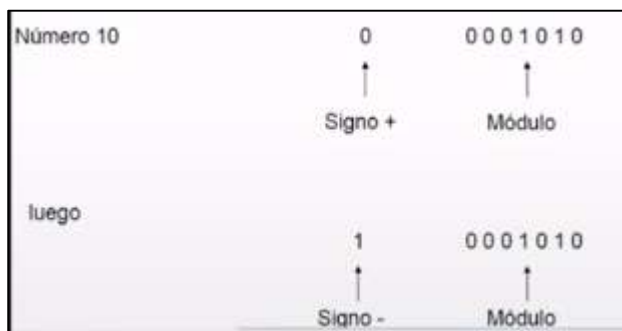
### **Modulo y signo**

En este sistema el bit que está situado más a la izquierda representa al signo, y su valor será un 0 para el signo + y un 1 para el signo. El resto de bits representan el modulo del número.

Ejemplo: supongamos que queremos representar los números 10 y -10. Disponemos de 8 bits, es decir  $n=8$ .

Es común llamar palabra a el espacio donde vas a guardar un número, generalmente las palabras en las maquinas son múltiplos de 8 bits, ya que las primeras máquinas tenían registros (lugares donde guardaban los datos) de 8 bits y después fueron creciendo estos registros, de 8 a 16, y de 16 a 32, para finalmente pasar a 64 bits en la actualidad.

En este caso disponemos de 8 bits, estamos en un contexto de palabra de 8 bits. Esto quiere decir que si querés guardar un número en un espacio pero te sobra espacio los vas a rellenar con ceros a la izquierda. En el lugar más significativo colocamos el signo.



Lo único que cambia es el signo, pero el modulo del número es el mismo.

- La ventaja de este sistema es poseer un rango simétrico (igual número de positivos y de negativos). Es decir si vos tenés una longitud de palabra dada vas a poder escribir la misma cantidad de números positivos y negativos.
- La desventaja es que posee dos representaciones para el número 0. Gráficamente:

Para  $n = 8$  bits

|           |      |
|-----------|------|
| -00000000 | (+0) |
| -10000000 | (-0) |

Alguien si escribe esto estaría diciendo que es +0 o -0 lo cual no tiene sentido, pero ocurriría lamentablemente en este sistema.

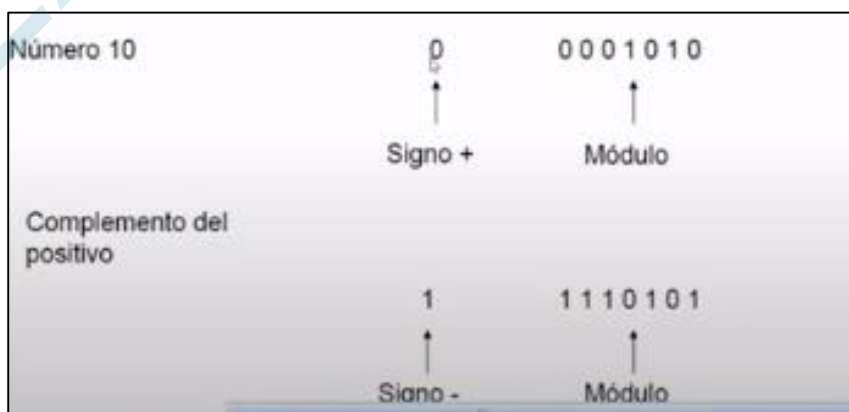
### Complemento a 1 (C-1)

En este sistema también el bit que está situado más a la izquierda representa el signo y su valor será 0 para el signo m+ y 1 para el signo -. El resto de bits representan el módulo del número.

El negativo de un número positivo se obtiene complementando todos sus dígitos y eso significa de forma práctica cambiar 0 por 1 y 1 por 0 incluido el bit de signo.

Ejemplos: supongamos que queremos representar los números 10 y -10. Nuevamente disponemos de 8 bits, es decir  $n = 8$ .

Vemos como el complemento del número para representar el negativo en este formato es cambiar 0 por 1 y 1 por 0.



### Complemento a 2 (C-2)

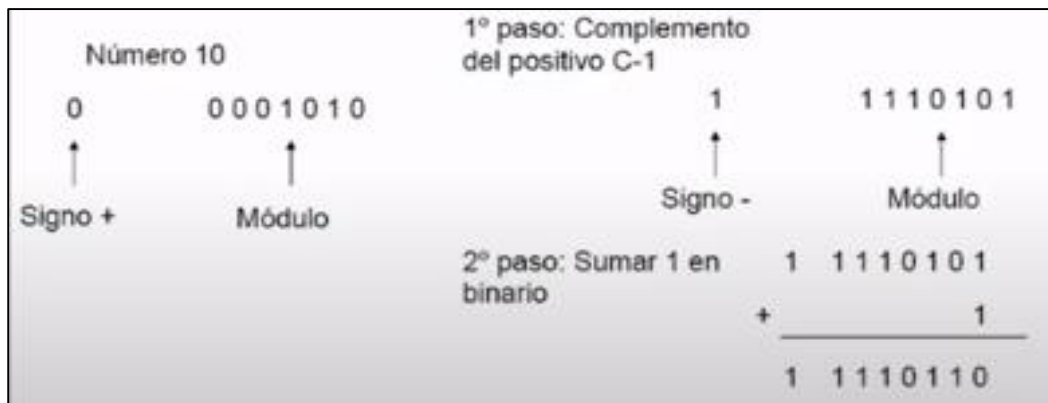
Como 2 es la base del sistema binario a este sistema le decimos complemento a la base, o complemento a 2.

En este sistema, también el bit que está situado más a la izquierda representa el signo y su valor será 0 para el signo + y 1 para el signo -. El resto de bits representa el modulo del número,

El negativo de un número se obtiene en dos pasos, el positivo es igual que siempre, para hacer entonces la representación negativa tenés que:

- 1) Hacer complemento a 1 (cambiar 0 por 1 y 1 por 0)
- 2) Al resultado del paso 1 le sumas un 1 en binario, despreciando el ultimo acarreo si es que existe.

Ejemplo: queremos representar los números 10 y -10. Disponemos de 8 bits, es decir  $n=8$



- 1) Cambiar todos los 0 por 1 y los 1 por 0.
- 2) Sumar 1 en binario.

- La ventaja de este sistema es poseer una única representación para el número cero.
- El ultimo acarreo se desprecia por lo tanto, el 0 y el -0 tienen la misma representación en un complemento a 2.

### Exceso a $2^n-1$

En este método la representación no usa bits para signo, por tanto todos los bits representan el modulo o valor. Este valor se corresponde con el numero representado más el exceso, que para  $n$  bits viene dado por  $2^n-1$ .

El exceso por ejemplo si  $n = 4$  entonces  $2^4$  es 16, vos vas a tener 16 valores para representar los números. Pero  $2^n-1$  que sería  $2^3 = 8$ , eso quiere decir que de los 16 valores para representar los primeros 8 serán números positivos y los otros 8 serán negativos, eliminando así la necesidad del bit de signo.

No hay un ejemplo a la mano :(

### Complemento a la base y a la base -1

El complemento a1 y a2 hacían alusión siempre a números binarios, pero nosotros podemos aplicar estos conocimientos a todos los sistemas conocidos, por eso veremos estas dos formas de representación junto a sus demostraciones. Entenderemos de donde viene lo de cambiar los 1 por 0 y los 0 por 1.

### Complemento a la base -1

En todo sistema de numeración de notación posicional, para un número "y" de  $n$  dígitos, o sea en un contexto de longitud de palabra  $n$ , existe un número "y'" (y prima) que es si complemento a la base -1 tal que la suma de ese número en cuestión más el otro prima da como resultado siempre  $10^n - 1$  siendo 10 la base en todo sistema o en cualquier sistema de numeración, cuando vos venís contando con los símbolos, cuando se te acaban la primera combinación que haces es el 1 con el 0, eso vale para todas las bases.

Luego despejamos el famoso número "y" prima, pasando la y restando al otro lado y entonces así te queda la definición. Y prima entonces es el complemento a la base -1 de y.

$$y + y' = 10^n - 1$$

$$y' = (10^n - 1) - y$$

Para hacerlo general como les gusta a los del teórico lo vamos a escribir con esta notación que parece complicada pero no lo es:

$$C_{\beta-1}^n(N) = (\beta^n - 1) - N$$

Siendo la C dice que es el complemento a la base -1. Entonces si decimos que la base es 8, en el subíndice de la C le pondremos un 7 para que la gente sepa que hablas de complemento a 1 y no de complemento a 2. Arriba en la "n" decís la longitud de palabra y en el paréntesis pones el número es cuestión.

Este es el complemento a la base -1 y así se lo obtiene, ahora veremos un ejemplo, aplicando la misma fórmula.

Sea:  $\beta = 10$   $N = 2358$

$$C_{10}^4(2358) = (10^4 - 1) - 2358 = 9999 - 2358 = 7641$$

Sea:  $\beta = 2$   $N = 0110$

$$C_2^4(0110) = (2^4 - 1) - 0110 = 1111 - 0110 = 1001$$

La base  $10 - 1$  es 9, así que obtenemos el complemento a 9 de este número 2358.

Hay que agarrar la base que es 10, elevarla a la 4, restarle 2358 y después restarle 1.

$$10^4 = 10000 - 1 = 9999$$

Hace una resta sin pedir prestado porque el número de arriba siempre es el más grande de todos. Restamos siempre el símbolo más grande que en este caso es el 9 para nunca tener que pedir prestado.

Obtenemos finalmente el número 7641, que es lo que la falta al número (2358) para alcanzar al 9999 original.

Luego aplicamos exactamente lo mismo para el número en binario, donde comparando el resultado con el número original podemos ver que cambia los 1 por 0 y los 0 por 1, por eso en el práctico para no tener que hacer todo ese proceso de resta directamente haces ese cambio.

### Complemento a la base

Del mismo modo que antes en todo sistema de numeración de notación posicional, decimos que para un número "x" de n dígitos, existe un número x' tal que:

$$x + x' = 10^n$$

$$x' = 10^n - x$$

Entonces de esta forma despejamos x' que es el número que queremos definir, entonces paso x restando, por tanto decimos que x' es el complemento a la base de x que si fuera en binario seria el complemento a 2.

Para escribirlo de forma generalizada sin hacer referencia a ninguna base ponemos.

$$C_{\beta}^n(N) = \beta^n - N$$

La definición práctica es: se define el complemento a la base como el complemento a la base -1 más la unidad, esto debido a que si uno compara la fórmula de este complemento con la del anterior que vimos si a la anterior le sumamos un 1 se convierte en esta nueva.

**Ejemplos:**

$$C_{\beta}^n(N) = \beta^n - N$$

Sea:  $\beta = 10$   $N = 2358$

$$C_{10}^4(2358) = (10^4) - 2358 = 7642 = C_2^4(2358) + 1$$

Sea:  $\beta = 2$   $N = 0110$

$$C_2^4(0110) = C_2^4(0110) + 1 = 1001 + 1 = 1010$$

**Definición práctica:**

Se define el complemento a la base-1 como la cantidad que le falta a cada cifra del número para alcanzar la base menos una.

Resta usando complemento

Mirar la foto mientras explico

Supongamos que  $x$  e  $y$  sean dos números expresados en una base  $\beta$  (B) cualquiera en un contexto de longitud de palabra  $n$ . Si haces la resta de  $x - y$  vos podrías poner lo mismo del otro lado del paréntesis sumando y restando  $\beta^n$  (porque eso no modifica la ecuación), yo al hacer esto si observamos esta estructura que se armó poniendo  $\beta^n - y$  es igual al complemento a la base de  $y$  (que vimos anteriormente), entonces la resta podría decirse que es igual a sumarle el complemento al número.

Al final le agregamos a  $\beta^n$  restando porque eso es un acarreo que puede ocurrir, o sea un arrastre, pero el mismo se va a caer fuera del campo de la longitud de palabra  $n$ , va a estar fuera del número, en la posición  $n + 1$  donde ya no existe nada ahí así que es como que hay un arrastre pero que no va a tener lugar en la memoria donde guardarse.

RESTA USANDO COMPLEMENTO

Sean  $x$  e  $y$  números expresados en base  $\beta$  en un contexto de longitud de palabra  $n$

$$\begin{aligned} x - y &= x + \beta^n - y - \beta^n \\ &= x + C_{\beta}^n(y) - \beta^n \end{aligned}$$

Traducido a algoritmo:

1. Obtener el complemento a la base del sustraendo ( $y$ )
2. Sumar este complemento al minuendo ( $x$ )

Si tenés que restar dos números donde  $x$  es el minuendo e  $y$  es el sustraendo lo que vas a hacer es sumar y restas  $\beta^n$  y te das cuenta entonces que hacer la resta es lo mismo que sumarle al minuendo el complemento a la base del sustraendo (o sea el número que resta).

Ejemplo:

Acá hay una demostración de cómo hacer una resta es igual a sumarle al minuendo el complemento a la base del sustraendo:

Ejemplo:Resta usando complemento

Sea  $\beta = 10$   $n = 4$   $X = 6532$   $Y = 2358$  ;  $C_{10}^4(2358) = 7642$

$$\begin{array}{r} X = 6532 \\ - Y = 2358 \\ \hline 4174 \end{array}$$

$$\begin{array}{r} X = 6532 \\ + C_{\beta}^n(Y) = 7642 \\ \hline 14174 \\ - 10000 \\ \hline 4174 \end{array}$$

Descartando el acarreo  $\beta^n = 10^4$  se obtiene

La verdadera importancia de este concepto es para el Sistema binario ya que al permitir realizar la resta sumando, se evita el "pedir prestado" simplificando de este modo la construcción del circuito restador de la ALU. Se puede usar el mismo circuito sumador para la resta.

## Sistemas de codificación

Un código establece una correspondencia entre un conjunto de información y otro conjunto de símbolos o señales que la representan, pudiendo existir reglas para pasar de un conjunto al otro. Tenemos por un lado información y por el otro un conjunto de símbolos o señales que la representa a esa información y se pasa de un conjunto a otro con reglas que la iremos estudiando.

Estos códigos digitales pueden ser considerados como los lenguajes digitales que permiten almacenar, manipular y comunicar la información.

Tal como existen numerosos lenguajes hablando, también existe una gran variedad de códigos digitales.

Así que cuando dos computadoras o dispositivos se tienen que comunicar se deben poner de acuerdo de que código van a usar como dijera dos personas en que lenguaje vamos a hablar y en esos se basa el código de codificación.

### Categorías de los códigos

**Categoría 1:** es el código binario base que usan los circuitos electrónicos, llámese computadoras, iPod, cualquier circuito electrónico, etc.

**Categoría 2:** son los códigos utilizados para convertir números decimales del 0 al 9 en forma digital. Estos serían los códigos utilizados para transferir números entre un dispositivo y otro. Ejemplo: BCD (Decimal codificado en binario "Binary-Coded Decimal")

**Categoría 3:** son utilizados para convertir números decimales, las 26 letras del alfabeto, símbolos y operaciones. Ej: ASCII, EDCDIC, etc.

**Categoría 4:** es más especializada en los fabricantes de computadoras, porque las instrucciones de máquina, nativas del procesador, están escritas en binario. Dicen que 1 y 0 representan a cada instrucción.

### Categoría 1

El sistema binario usa solo dos símbolos, el 0 y el 1, cada uno de los cuales se le llama dígito binario o bit. Es usual denominar ponderado a los sistemas binarios en los que se puede asignar a cada posición de los dígitos una potencia de 2 ordenada desde el dígito menos significativo hasta el más significativo denominando a la potencia peso.

### Categoría 2: códigos BCD

Convierten números decimales en binario, codifican los símbolos que conocemos del 0 al 9 en paquetes de 1 y 0. Consiste básicamente en representar la estructura decimal del número representado en binario sus cifras

Cada dígito se representa mediante 4 dígitos binarios. Ya que queremos representar solo 10 cifras distintas del sistema decimal (0 al 9) necesitaríamos  $2^n$  dígitos en el sistema binario. Ahora para  $n = 3$  tenemos 9 símbolos distintos por lo tanto no son suficientes, mientras que para  $n = 4$  tenemos 16 símbolos distintos sobrando 6 solamente, usaríamos 10 y nos sobran 6.

Entonces se lo conoce como BCD natural o puro ya que sus pesos coinciden con los 4 primeros pesos en sistema binario o natural.

**Código BCD Natural (8-4-2-1)**

| Dec. | binario |   |   |   |
|------|---------|---|---|---|
|      | 8       | 4 | 2 | 1 |
| 0    | 0       | 0 | 0 | 0 |
| 1    | 0       | 0 | 0 | 1 |
| 2    | 0       | 0 | 1 | 0 |
| 3    | 0       | 0 | 1 | 1 |
| 4    | 0       | 1 | 0 | 0 |
| 5    | 0       | 1 | 0 | 1 |
| 6    | 0       | 1 | 1 | 0 |
| 7    | 0       | 1 | 1 | 1 |
| 8    | 1       | 0 | 0 | 0 |
| 9    | 1       | 0 | 0 | 1 |

Lo que hay que hacer es convertir dígito por dígito en binario y así quedara codificado el número de manera fácil y rápida.

Acá en la primera columna tenemos los símbolos del 0 al 9 que queremos representar en binario, así que tomaremos 4 bits para representar los símbolos ya que 3 no nos alcanzaba. Con 4 bits comenzamos entonces a escribir el equivalente.

0000 representa al número decimal 0.

Fíjense por otro lado que para poder armar el número ahí tengo los pesos puestos que son los mismos que los del polinomio. Sumábamos el peso que tenía un 1 en la palabra o en este paquete de 1 y 0.

Así que por ejemplo para formar el 1 esta prendido nomás que el ultimo bit y entonces sumo y me da uno.

Si agarro uno cualquiera por ejemplo tendría que sumar el peso 4, 2 y 1 para formarlo. Es decir, debo multiplicar el 1 por el peso y sumarlo con el valor de los otros pesos. Quedando así para representar al 7, la suma de  $4 \times 1 + 2 \times 1 + 1 \times 1 = 7$ .

Eso debe hacerse con todos los números.

Esto es mucho más rápido que hacer una conversión a binario convencional, me ahorro el paso de hacer divisiones sucesivas por 2 hasta encontrar el número. Con esta codificación podemos mandar 1 por 1 cada dígito del número decimal que queremos convertir. Ejemplo:

|                                                  |                                                   |      |      |      |
|--------------------------------------------------|---------------------------------------------------|------|------|------|
| Ej: Supongamos (1048) <sub>10</sub> en BCD será: | 0001                                              | 0000 | 0100 | 1000 |
|                                                  | 1                                                 | 0    | 4    | 8    |
| En binario común :                               | (1048) <sub>10</sub> = (10000011000) <sub>2</sub> |      |      |      |

### Código BCD Aiken (2-4-2-1)

Fue pensado por un señor que se llamaba aiken y el propone que se cambie el peso 8 por el peso 2.

Este es un sistema ponderado de peso 2-4-2-1.

**Código BCD Aiken (2-4-2-1)**

| Dec. | binario |   |   |   |
|------|---------|---|---|---|
|      | 2       | 4 | 2 | 1 |
| 0    | 0       | 0 | 0 | 0 |
| 1    | 0       | 0 | 0 | 1 |
| 2    | 0       | 0 | 1 | 0 |
| 3    | 0       | 0 | 1 | 1 |
| 4    | 0       | 1 | 0 | 0 |
| 5    | 1       | 0 | 1 | 1 |
| 6    | 1       | 1 | 0 | 0 |
| 7    | 1       | 1 | 0 | 1 |
| 8    | 1       | 1 | 1 | 0 |
| 9    | 1       | 1 | 1 | 1 |

Tiene la ventaja de presentar simetría de complementación respecto de la línea punteada, esto consiste que se invierten los 1 por 0 y los 0 por 1.

¿Cómo se arma?

El primero son todos 0, sumas los pesos y te da cero, esta fácil

En el otro (el siguiente al 0) haces  $1 \times 1 = 1$  y todo por 0 da 0 por tanto el 1 se escribe como 0 0 0 1.

Así hasta el número 4.

Para armar el 5 vos tendrías a lo mejor más de una forma, podrías poner un 1 en la columna del 2, 2 1, también en la de 4 y 1 dándote exactamente lo mismo. Por eso existe una regla extra. Vos tiras una línea y empezás a hacer reflexiones.

Si ustedes se fijan en 4 y 5 tienen valores opuestos, es decir donde uno tiene 0 el otro tiene 1 y viceversa, eso quiere decir la reflexión. Esa es la única forma válida de representar el 5 en este código.

Para armar el 6, tengo que reflejar el 3, para armar el 7 tengo que reflejar 2 y así sucesivamente.

Se reflejan complementados.

### Código BCD exceso de tres o de Stibitz

El exceso de tres es un código muy usado y dice que los códigos no ponderados como este, no tiene peso, la correspondencia entre decimal y binario es arbitraria ya que al no haber peso no se verifica el número, aunque por lo general las combinaciones se forman según ciertas reglas, en este caso la regla, es que a cada número decimal N le corresponde el número N+3 del código 8-4-2-1.

**Código BCD Exceso De tres o Stibitz (XS-3)**

| Dec. | binario |   |   |   |
|------|---------|---|---|---|
|      |         |   |   |   |
| 0    | 0       | 0 | 1 | 1 |
| 1    | 0       | 1 | 0 | 0 |
| 2    | 0       | 1 | 0 | 1 |
| 3    | 0       | 1 | 1 | 0 |
| 4    | 0       | 1 | 1 | 1 |
| 5    | 1       | 0 | 0 | 0 |
| 6    | 1       | 0 | 0 | 1 |
| 7    | 1       | 0 | 1 | 0 |
| 8    | 1       | 0 | 1 | 1 |
| 9    | 1       | 1 | 0 | 0 |

Si vos agarras el código 8-4-2-1 y le sumas tres, acordate que 3 es el 0011 entonces ahí arranca el 0 en el código exceso de tres. Por eso se le llama exceso de 3.

El 1 es el 4 del código 8-4-2-1, el 2 es el 5 y así sucesivamente. (N+3)

Este también posee una reflexión después de la línea de puntos de simetría de complementación de la misma forma que ocurría en el anterior, el 4 se refleja en el 5, el 3 en el 6 y así sucesivamente.

Código BCD progresivo cíclico

El código progresivo cíclico está muy astuto, porque de las 16 combinaciones que teníamos para elegir, de 1 y 0, este tipo elige estas curiosas 10.

Para que un código sea progresivo todos los números deben ser adyacentes entre el anterior y el siguiente. Esto quiere decir que solo cambian en un bit entre ellos.

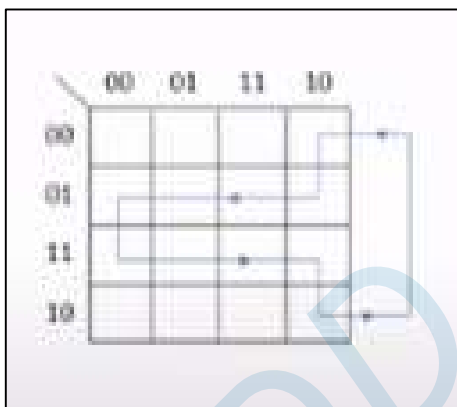
Ejemplo de adyacencia: 000 y 001 son adyacentes porque solo cambian en un bit (el ultimo)

Ejemplo de no adyacencia: 010 y 001. Cambian los dos últimos bits.

**Código BCD Progresivo Cíclico.**

| Dec | binario |   |   |   |
|-----|---------|---|---|---|
| 0   | 0       | 0 | 1 | 0 |
| 1   | 0       | 1 | 1 | 0 |
| 2   | 0       | 1 | 1 | 1 |
| 3   | 0       | 1 | 0 | 1 |
| 4   | 0       | 1 | 0 | 0 |
| 5   | 1       | 1 | 0 | 0 |
| 6   | 1       | 1 | 0 | 1 |
| 7   | 1       | 1 | 1 | 1 |
| 8   | 1       | 1 | 1 | 0 |
| 9   | 1       | 0 | 1 | 0 |

Seleccionen arbitrariamente un número y verán que el de arriba y el de abajo solo cambian en un bit entre sí, cumpliendo la propiedad antes mencionada.



Se arma usando este dibujo llamado mapa de Karnaugh.

Primero se ingresa por el extremo derecho, y se escribe la coordenada de ese casillero, en este caso 0010 y ese será el primer número. Luego seguimos el camino, el segundo número (el 1) será el 0110 y así seguiremos sucesivamente por el camino indicado rellenando con los números adecuados.

También se denomina código de grey porque puede generarse como parte de otro código mayor conocido como "Reflejado"

Código reflejado de Gray

Grey se tomó el trabajo de extender todos los números del código progresivo cíclico a todas las combinaciones posibles.

Comienza poniendo todo 0000 y después llega hasta 0001 y se detiene, tira una línea de simetría y complementa el más importante nada más y los otros refleja tal cual.

Después tira una segunda línea, cambia todos los cuatro 0 (miren de forma vertical) y los cambia por cuatro 1 después (sigan mirando vertical) y refleja tal cual lo que está en el rectángulo de al lado.

Por ultimo tira una última línea donde complementa solo los más importante (miren vertical) todos los ceros (los que están al lado del número decimal 4 5 6 7, se cambian por 1 abajo y después refleja como veníamos haciendo solo que no lo complementa, lo pone tal cual está. Refleja el 7 con el 8, el 9 con el 6 y así sucesivamente. El código anterior (el progresivo cíclico) está contenido desde el número 4 hasta el número 12.

| Dec | binario |   |   |   |
|-----|---------|---|---|---|
| 0   | 0       | 0 | 0 | 0 |
| 1   | 0       | 0 | 0 | 1 |
| 2   | 0       | 0 | 1 | 1 |
| 3   | 0       | 0 | 1 | 0 |
| 4   | 0       | 1 | 1 | 0 |
| 5   | 0       | 1 | 1 | 1 |
| 6   | 0       | 1 | 0 | 1 |
| 7   | 0       | 1 | 0 | 0 |
| 8   | 1       | 1 | 0 | 0 |
| 9   | 1       | 1 | 0 | 1 |
| 10  | 1       | 1 | 1 | 1 |
| 11  | 1       | 1 | 1 | 0 |
| 12  | 1       | 0 | 1 | 0 |
| 13  | 1       | 0 | 1 | 1 |
| 14  | 1       | 0 | 0 | 1 |
| 15  | 1       | 0 | 0 | 0 |

1<sup>ra</sup> línea de simetría  
2<sup>da</sup> línea de simetría  
3<sup>ra</sup> línea de simetría

También se lo construye con la ayuda del mapa de Karnaugh.

• Se construye reflejando a partir de una primera línea de simetría 2 bits y agregando otros no reflejados. A su vez e conjunto así se vuelve a reflejar utilizando otra línea de simetría y así sucesivamente.

## Código de redundancia

Que pasa cuando vos transmitís una señal de un lado a otro y la misma se corrompe por un ruido electrónico, o cualquier razón y se te cambia un 1 por un 0, es un problema porque si el 1 por el 0 que se cambia se te cambia otro paquete de 1 y 0, sonamos, el receptor lo va a tomar sin saber que hay un error y eso es grave si se trata que estamos mandando información importante como por ejemplo una información bancaria de mucho dinero y se cambia el bit más significativo. Para evitar esto se hace lo que se explicará a continuación.

Si se usa un código cuyos elementos tengan más información que la necesaria, esto lo hace resisten a errores y perdida de información, esta información por encima de la necesaria se la conoce como redundancia.

Esto se arregla con los códigos detectores de errores con paridad, una de las formas pero no la única aunque la más famosa es que ahora temes los 4 pesos como el binario natural, le agregamos un bit más que se llama paridad. Ahora en vez de transmitir 4 bits transmitido 5, lo cual baja la velocidad de transmisión, pero gana en seguridad.

| Dec. | binario |   |   |   | p |
|------|---------|---|---|---|---|
|      | 8       | 4 | 2 | 1 |   |
| 0    | 0       | 0 | 0 | 0 | 1 |
| 1    | 0       | 0 | 0 | 1 | 0 |
| 2    | 0       | 0 | 1 | 0 | 0 |
| 3    | 0       | 0 | 1 | 1 | 1 |
| 4    | 0       | 1 | 0 | 0 | 0 |
| 5    | 0       | 1 | 0 | 1 | 1 |
| 6    | 0       | 1 | 1 | 0 | 1 |
| 7    | 0       | 1 | 1 | 1 | 0 |
| 8    | 1       | 0 | 0 | 0 | 0 |
| 9    | 1       | 0 | 0 | 1 | 1 |

Vos podés elegir que la paridad sea par o impar. El receptor cuando recibe cuenta la cantidad de 1 sin importar su posición, entonces si yo antes había decretado que la paridad iba a ser impar por ejemplo y recibo una cantidad impares de 1 entonces se puede inferir que el número fue transmitido sin errores.

Entonces el bit p se lo va a completar estratégicamente de modo tal que la palabra total que va a transmitir, este paquete de símbolos 0 y 1 tenga una cantidad impar de 1.

Si el transmisor tuviera que mandar 0 0 0 0 el completaría con un 1 lo que trasmite así el receptor cuando reciba y cuente todos los 1 se dé cuenta de que 1 es impar, un solo 1 es impar entonces el dato puede estar bien, aunque obviamente solo se van a tomar los números en negro como los verdaderos en el mensaje que se envió.

Y así sucesivamente con el resto de números. La columna roja no sirve para nada más que para comprobar si está bien el mensaje.

## Código con cantidad constante de unos

### **Código “dos entre cinco” (7-4-2-1)**

Se utiliza también para corregir errores, en este caso este bit de paridad tiene otra aplicación distinta a la de recién. También son códigos con redundancia para detectar errores, hay redundancia porque hay más bits de lo necesarios para transmitir el mensaje.

Se le llama con cantidad constante de unos porque todas las palabras que vas a recibir tienen nada más que dos unos, ninguna palabra tiene más de dos unos. El receptor cuando reciba la información tiene que contar que haya nomás dos uno, si está bien verifica el número controlando cuánto vale según los pesos.

| Dec. | binario |   |   |   |   |
|------|---------|---|---|---|---|
|      | 7       | 4 | 2 | 1 | p |
| 0    | 1       | 1 | 0 | 0 | 0 |
| 1    | 0       | 0 | 0 | 1 | 1 |
| 2    | 0       | 0 | 1 | 0 | 1 |
| 3    | 0       | 0 | 1 | 1 | 0 |
| 4    | 0       | 1 | 0 | 0 | 1 |
| 5    | 0       | 1 | 0 | 1 | 0 |
| 6    | 0       | 1 | 1 | 0 | 0 |
| 7    | 1       | 0 | 0 | 0 | 1 |
| 8    | 1       | 0 | 0 | 1 | 0 |
| 9    | 1       | 0 | 1 | 0 | 0 |

La única crítica que tiene este código que se llama este código dos entre cinco porque son dos 1 entre 5, es que el cero no se forma con los pesos binarios, todos los otros sí.

Por ejemplo para armar el uno se pone un uno en el peso 1, y el otro uno se lo pone en el peso 0 que no cuenta entonces así armas el uno.

Así lo vas armando menos el cero que es el único que o te cumple porque el cero si sumas 7 y 4 te da 11, entonces el receptor sabe que si le llega un 11 se trata del cero. Bueno esto no quedó tan elegante pero es la forma que lo hace.

Entonces el receptor tiene que contar los unos que recibió como tarea.

### **Código “Biquinario” (5-0 4-3-2-1-0)**

Este código también sirve para transmitir número del 0 al 9, en vez de usar 4 pesos usa 7, hay mucha redundancia, por tanto es muy seguro.

| Dec. | binario |   |   |   |   |   |   |
|------|---------|---|---|---|---|---|---|
|      | 5       | 0 | 4 | 3 | 2 | 1 | 0 |
| 0    | 0       | 1 | 0 | 0 | 0 | 0 | 1 |
| 1    | 0       | 1 | 0 | 0 | 0 | 1 | 0 |
| 2    | 0       | 1 | 0 | 0 | 1 | 0 | 0 |
| 3    | 0       | 1 | 0 | 1 | 0 | 0 | 0 |
| 4    | 0       | 1 | 1 | 0 | 0 | 0 | 0 |
| 5    | 1       | 0 | 0 | 0 | 0 | 0 | 1 |
| 6    | 1       | 0 | 0 | 0 | 0 | 1 | 0 |
| 7    | 1       | 0 | 0 | 0 | 1 | 0 | 0 |
| 8    | 1       | 0 | 0 | 1 | 0 | 0 | 0 |
| 9    | 1       | 0 | 1 | 0 | 0 | 0 | 0 |

Tenemos dos unos en el paquete que queremos transmitir, se forma con dos sistemas, el bi con peso de 5-0 y qui 4-3-2-1-0. Estos son los pesos con los cuales vos vas a recuperar el número.

Parate en el numero 6 fijate que le puse un uno acá en el 5 y uno acá en el peso 1 del otro, entonces tenes  $5+1 = 6$ . Pero también podrías haber puesto el 1 en el peso 4 y 2 haciendo  $4+2 = 6$  pero estaría mal porque el receptor divide en dos tablas.

Tiene que haber obligatoriamente un 1 que esté en la parte del Bi y otro que se encuentre en qui, entonces de esa forma hay una sola forma de armar el numero por tanto sería muy difícil que se cambien los números, por ende es muy robusto este sistema.

Del 0 al 4 le pones un 1 en el peso 0.

Del 5 al 9 le pones un 1 al 5.

### Códigos de categoría 3

Son los usados para convertir números decimales, las 26 letras del alfabeto, símbolos y operaciones.

Si vos agarras y te fijas en el teclado de tu maquina vas a ver la cantidad de letras, símbolos, signos, etc. que lo conforman, tiene que haber una muy buena codificación para que no se confundan ninguna de estas teclas cuando yo mando información. Entonces eso se lo conoce como los códigos que mandan todo este tipo de información numéricas y de símbolos y letras se llaman los alfanuméricos y están en la categoría 3. El más famoso es el ASCII

### **Código ASCII**

Quiere decir Código Americano Estandarizado Para el Intercambio de Información (American Standard Code for Information Interchange) fue establecido por la ANSI en 1915.

Cuando nació este código era del teclado QWERTY original y estaban definidas las letras (solo mayúsculas) después los números (sin "ñ" ni nada porque era americano) y los caracteres de puntuación y de control (retorno de carro, etc.),

Existen versiones de 6 bits que es la original y la de siete y ocho bits que es la corriente actualmente que incluyen un montón de símbolos más.

|    | 0 | 1 | 2 | 3 | 4  | 5 | 6 | 7 | 8        | 9 | A | B | C | D | E | F |
|----|---|---|---|---|----|---|---|---|----------|---|---|---|---|---|---|---|
| 00 |   |   |   |   |    |   |   |   |          |   |   |   |   |   |   |   |
| 10 |   |   |   |   |    |   |   |   |          |   |   |   |   |   |   |   |
| 20 |   | ! | " | # | \$ | % | & | ' | (        | ) | * | + | , | - | . | / |
| 30 | 0 | 1 | 2 | 3 | 4  | 5 | 6 | 7 | 8        | 9 | : | ; | < | = | > | ? |
| 40 | @ | A | B | C | D  | E | F | G | H        | I | J | K | L | M | N | O |
| 50 | P | Q | R | S | T  | U | V | W | X        | Y | Z | [ | \ | ] | ^ | _ |
| 60 |   | a | b | c | d  | e | f | g | <b>h</b> | i | j | k | l | m | n | o |
| 70 | p | q | r | s | t  | u | v | w | x        | y | z | { |   | } | ~ |   |
| 80 | À | Á | Â | Ã | Ä  | Å | Ö | Ç | È        | É | Ê | Ë | Ì | Í | Î | Ï |
| 90 | Ð | Ñ | Ò | Ó | Ô  | Õ | Ö | × | Ø        | Ù | Ú | Û | Ü | Ý | Þ | ß |
| A0 |  |  |  |  |   |  |  |  |         |  |  |  |  |  |  |  |
| B0 |  |  |  |  |   |  |  |  |         |  |  |  |  |  |  |  |
| C0 |  |  |  |  |   |  |  |  |         |  |  |  |  |  |  |  |
| D0 |  |  |  |  |   |  |  |  |         |  |  |  |  |  |  |  |
| E0 |  |  |  |  |   |  |  |  |         |  |  |  |  |  |  |  |
| F0 |  |  |  |  |   |  |  |  |         |  |  |  |  |  |  |  |

Char: h    Decimal: 104  
Hex: 0x68    Octal: 150

Otros códigos alfanuméricos

Otros códigos son:

**EBCDIC:** Extended BCD Interchange Code fue desarrollado por IBM, que es la única que lo usa en grandes sistemas.

**UNICODE:** es el código más usado para el manejo de fuentes, procesamiento de palabras, tiene muchos caracteres extraños con sus equivalentes en unos y cero, permitiendo la representación de múltiples alfabetos.

Categoría 4

Son los códigos de instrucciones usados por los procesadores, haciendo que realicen una determinada secuencia de operaciones. Se las ve recién en el final de materia, así que nomás sepan eso que dije recién.

Códigos correctores y detectores de errores

Es necesario detectar y corregir el error cuando el receptor no tiene capacidad de transmisión. Por ejemplo yo transmito información a muchos receptores de radio que están a 20kms de acá pero el receptor recibe el texto y si hay cambios de bits, el receptor no tiene una antena gigante para decirme: ¡che llego mal! Si fuera el caso habríamos usado los casos anteriores, aquel tiene que tomar la información y tiene que ser buena sino debe corregirse. Por tanto se usan estos métodos que detectan el error y también lo corrigen, son más caros porque viajan varios bits de control en la información.

**Código de Hamming**

Detecta y corrige un error. Usa esta fórmula:

$p$  = cantidad de bits de control que tenes que poner en la palabra para transmitirla

$i$  = cantidad de información que vos querés enviar.

$$2^p \geq i + p + 1$$

Palabra a transmitir =  $m = i + p = 2^p - 1$  bits

**Ejemplo del código en acción**

Sea  $p = 4$   $i = 11$  y  $m = 15$

Los bits de paridad ocupan lugares que son potencias de dos, se sientan en los lugares recuadradas en rojo.

$2^0 = 1$  ;  $2^1 = 2$  ;  $2^2 = 4$  ;  $2^3 = 8$

Rellenamos el espacio con el número que vamos a mandar excepto los espacios donde va el bit de control. Voy a poner números random en mi ejemplo.

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 |
|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|
|    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 |
|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|
| 1  | 0  | 0  | 1  | 0  | 1  | 1 |   | 0 | 0 | 1 |   | 1 |   |   |

| 15 | 1 | 1 | 1 | 1 |
|----|---|---|---|---|
| 12 | 1 | 1 | 0 | 0 |
| 10 | 1 | 0 | 1 | 0 |
| 9  | 1 | 0 | 0 | 1 |
| 5  | 0 | 1 | 0 | 1 |
| 1  | 0 | 0 | 1 | 1 |
|    | 0 | 1 | 1 | 0 |

Luego con esto tenemos que hacer otra tabla poniendo en que posiciones hay colocados 1.

En este caso hay 1s en el 15, 12, 10, 9, 5 y en 3.

A continuación escribiremos al lado de los lugares donde encontramos 1 como se escriben en binario.

Luego debo sumar la cantidad de 1s en cada columna y obtener el resto al dividirlo por 2.

**Esos números son los que irán rellenando los espacios vacíos que dejamos antes para los bits de control.**

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 |
|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|
| 1  | 0  | 0  | 1  | 0  | 1  | 1 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 0 |

Ahora vamos a cambiar un número adrede para mostrar cómo es que este código puede detectar el error. Supongamos que el 14 en vez de mandar un 0 mandamos un 1.

Entonces volvamos a realizar la operación de poner todos los lugares donde hay un 1 y después hacer mod 2 es decir, obtener el resto de sumar la cantidad de unos por columna y dividir ese resultado por 2.

| 15 | 1 | 1 | 1 | 1 |
|----|---|---|---|---|
| 14 | 1 | 1 | 1 | 0 |
| 12 | 1 | 1 | 0 | 0 |
| 10 | 1 | 0 | 1 | 0 |
| 9  | 1 | 0 | 0 | 1 |
| 5  | 0 | 1 | 0 | 1 |
| 4  | 0 | 1 | 0 | 0 |
| 1  | 0 | 0 | 1 | 1 |
| 2  | 0 | 0 | 1 | 0 |
|    | 1 | 1 | 1 | 0 |

Si debajo de todo en vez de 1 1 1 0 me hubiese dado 0 0 0 0, entonces el receptor entiende que el mensaje está bien, pero en su lugar como hay un error nos informa cuál es el número que está mal, en este caso el 1 1 1 0, que corresponde al 14 en binario que habíamos decidido alterar.

### Distancia de Hamming

Sean los elementos de un código a y b, donde pertenecen al grupo del código, la probabilidad de que a y b se confundan disminuye mientras más distintos sean de ellos. Es decir, en una transmisión si son bien distintos es más difícil que uno se confunda por el otro.

La medida de la diferencia entre dos términos contiguos de un código se la llama distancia de hamming del mismo, mientras más distancia exista entre los caracteres de un código menor es la posibilidad de error cuando se los usa.

Supongamos tenemos dos letras a = 11011 y b = 11101, la distancia de Hamming entre ellas será de dos porque si vos comparas los bits, cambia dos veces, hay dos números que son distintos, por tanto se entiende a la distancia de Hamming como la cantidad de cambios que existen entre dos números. Mientras más robusto es un código, más distancia de Hamming habrá.

## Formato de representación de los números fraccionarios

### **Representación de los números fraccionarios en coma fija**

Concepto de palabra, longitud de palabra

- La representación de los números en los sistemas de computación son un compromiso entre las necesidad de elevada exactitud, esto es, elevado números de dígitos, con las restricciones tecnologías que establecen los costos.
- Esto se traduce en que los procesadores usan un conjunto de dígitos de longitud fija para representar las variables. Este conjunto de dígitos de longitud fija recibe el nombre de palabra de datos.

La palabra es la longitud del registro del procesador, los primeros procesador tenían registro de 8 bits, hoy en día son de 64, por eso decimos que la palabra, es decir, donde vamos a anotar los números son de 64 bits.

Al representar un número de coma fija tendríamos un dato de 8 bits, a modo de ejemplo, porque 64 es muy largo. De los cuales se han destinado dos bits para la parte fraccionaria y 6 bits para la parte entera. La coma se ha dejado en un lugar fijo, de esta forma podríamos nosotros imaginar que se podrían anotar los datos en los registros destinando otra vez un compromiso, una parte para la parte entera y otra para la parte fraccionaria. Esa coma la ponemos imaginariamente en algún lugar y se va a quedar fija.

Si se opera con una longitud de palabra de 8 bits, la coma se asigna de forma arbitraria en cualquiera de las posiciones pero una vez elegida no se modifica.

La principal ventaja es que los algoritmos de realización de las diferentes operaciones son los mismos que para los números enteros.

$$\begin{array}{r}
 011011,11 \\
 + 100001,01 \\
 \hline
 111101,00
 \end{array}$$

El algoritmo de la suma es el mismo a que si sumara el numero como si fuese un entero.

El principal inconveniente de este formato es que no se aprovecha la capacidad de los operadores aritméticos, el número máximo sería nada más que 6 unos en la parte entera y 2 unos en la parte fraccionaria (111111,11) y el número mínimo es el 0,01 aunque la capacidad de los operadores permitirían operar el número máximo como 8 unos (11111111) o corriendo la coma al otro extremo 0,00000001.

No puede representar números muy grandes ni muy pequeños, esa es la desventaja, por eso no se lo usa mucho.

La solución a este problema es:

### **Representación de los números fraccionarios en coma flotante**

Es así como se representan hoy en día los números fraccionarios.

La representación de los números en coma flotante en un sistema de numeración de base B se realiza mediante una mantisa m y un exponente e tal que:

$$n = m B^e$$

La mantisa multiplicada por la base elevada al exponente.

M = número en coma fija que se denomina mantisa.

E = número entero que se denomina exponente.

B = base del sistema de numeración.

## Ejemplo

Ej. en decimal,  $B=10$  sea  $x = 22,5 \cdot 10^3$  se lo puede representar de muy diversas maneras:

$$22,5 \cdot 10^3 = 0,0225 \cdot 10^6 = 225000 \cdot 10^{-1}$$

Que se diferencian por la posición de la coma en la mantisa y el valor del exponente.

En la práctica las formas más utilizadas son las llamadas *normalizadas*:

- Situación de la coma inmediatamente antes de la cifra más significativa distinta de cero:

$$0,225 \cdot 10^5$$

- Situación de la coma inmediatamente después de la cifra más significativa distinta de cero:

$$2,25 \cdot 10^4$$

Ej. en binario:

$$0,011 \cdot 2^{0011} = 1,1 \cdot 2^{0001} = 110 \cdot 2^{-0001}$$

Al igual que en el sistema decimal las expresiones *normalizadas* más usadas son:

$$0,11 \cdot 2^{0010}$$

$$1,1 \cdot 2^{0001}$$

Diversas formas (denominadas formatos) de representación de los números binarios en coma flotante han sido desarrolladas:

Dichos formatos definen:

- El número total de bits usados.
- La forma de representación de los números positivos y negativos del exponente.
- El número de bits de la mantisa y el exponente.
- El orden correlativo entre la mantisa y el exponente.

El mas utilizado se indica en la figura:

|         |           |         |
|---------|-----------|---------|
| SIGNO   | EXPONENTE | MANTISA |
| MANTISA |           |         |

IEEE Standard for Floating-Point Arithmetic¿Cómo se escribe un número en el estándar IEEE 754?

El estándar ha sido definido por el Instituto de Ingenieros Eléctricos y Electrónicos y establece los formatos básicos para representar a los números reales en la computadora digital: precisión simple, precisión doble y cuádruple.

**Precisión Simple en el Estándar IEE 754**

Para escribir un numero se usa 32 bits (4 bytes), hay un bit para el signo, 23 bit para la mantisa y 8 bits para el exponente y se distribuyen de la siguiente forma



Arranca desde el 0 hasta el 22 son 23 bits y esto es para guardar la mantisa, el exponente va desde el 23 hasta el 30 son 8 los bits y por ultimo hay un solo bits para el signo que tiene la mantisa. En total da un tamaño de 32 bits. En vez de definir una coma fija en algún lugar y poner parte entera y fraccionaria lo que se hace es anotar los números de la manera anterior descrita.

La mantisa se va a tener que truncar en algún momento porque solo tenes 23 bits, los bits que estén más allá se quedaran afuera y vos no tendrás una representación característica.

Con el tamaño del exponente vos podrás elevar con 8 bits exponentes positivos y negativos pero no muy muy grandes, en ese caso debería cambiar el tipo de precisión.

El exponente se lo representa en exceso a la  $2^{n-1}-1$ , este es un número, es el exceso y con el exceso lo que se hace es no tener el bit de signo sino que directamente es como que vos tenes una gran tabla de verdad que la partís en dos y desde la línea para arriba tenes los exponentes negativos y para abajo los exponentes positivos. En este caso n vale 8 bits, haces  $8-1 = 7$ , por ende  $2^7 - 1 = 127$  que es la mitad de la tabla cuando vos haces  $2^8$  que da 256, que dividido 2 da 128 - 1 127, prácticamente estas en la mitad de la tabla, de esa mitad para arriba tenes exponentes negativos y para abajo los exponentes positivos. Vas a poder elevar a la -127 y a la +127 en binario.

El exponente es un número escrito en formato exceso 127, lo que significa que  $2^0$  (o sea el 1) es el número 127 que es el 7Fh en hexadecimal. De esa forma nos ahorramos a usar el bit de signo, hacer complementos y demás

El rango dinámico entonces el conjunto de valores que pueden ser representados, es de:

$2^{+128/-128}$  lo que equivale a  $10^{+38/-38}$  que es importante.

La precisión numérica usando una mantisa de 24 bits es de  $6 \frac{1}{2}$  dígitos, vos tenes 24 bits pero son nada más que 6 dígitos y un  $\frac{1}{2}$  en decimal,  $\frac{1}{2}$  quiere decir que algunos dígitos de 7 bits los vas a poder representar bien (después de la coma) y otros números de 7 dígitos después de la coma no lo vas a poder representar bien, entonces vas a tener 6 dígitos representativos después de la coma y algunos de 7. En el práctico se analizará por qué se habla de 24 bits (tema de bit implícito) y no de 23 para la mantisa.

## Precisión doble

Se ponen dos registros de 32 bits pegaditos entonces el número tendrá un registro de 64 bits. Es 1 bit para el signo (s) del número, 62 bits para la mantisa (m) y 11 bits para el exponente (exp). Se pueden representar números mucho más grande con esta precisión.

El bit 53 de la mantisa es un 1, que es implícito, lo hacen para ganar un espacio.

El exponente es un numero de 11 bits codificado en exceso 1023 ( $2^{10} - 1$ ), la misma fórmula de antes ( $2^n - 1$ ), entonces se dice que se parte en dos la tabla de exponentes y se divide en dos la cantidad de exponentes que puedes elevar.

El rango dinámico es que vos podrías elevar de la siguiente forma:

$2^{-1023/+1023}$  lo que equivale a  $10^{-308/+308}$  que es suficiente.

La precisión numérica obtenible usando mantisa de 54 bits es de 14 ½ dígitos.

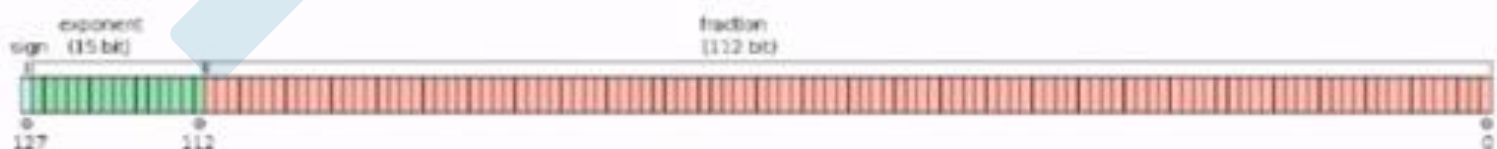
## Casos especiales en el Estándar IEEE 754 (me lo tomaron en un parcial)

Tanto en precisión doble como simple existen algunos casos especiales que dependen de los valores del signo del exponente y de la mantisa:

| Signo (s)    | Exponente (exp)     | Mantisa (m)            | Significado                            |
|--------------|---------------------|------------------------|----------------------------------------|
| Positivo (0) | Todo unos (11...1)  | Todo ceros (00...0)    | Más infinito (+∞)                      |
| Negativo (1) | Todo unos (11...1)  | Todo ceros (00...0)    | Menos infinito (-∞)                    |
| 0 ó 1        | Todo unos (11...1)  | Distinta de todo ceros | No es un número (Not a Number, NaN)    |
| 0 ó 1        | Todo ceros (00...0) | Todo ceros (00...0)    | Representa al cero (0)                 |
| 0 ó 1        | Todo ceros (00...0) | Distinta de todo ceros | Número muy pequeño cercano al cero (0) |

## Precisión cuádruple

Cuando se requiere un cálculo muy exacto se usa este método, que sigue la misma idea que siempre:



Ahora son 128 bits, 1 solo bit para el signo de la mantisa, exponente de 15 y para la mantisa dejan 112 bits, quedando un exceso de 16383 que sale siempre de la misma formula ( $2^n - 1$ )

## Función lógica

Una función lógica del álgebra de Boole es una variable binaria, cuyo valor es igual al de una expresión algebraica, en la que se relacionan entre sí variables binarias por medio de las operaciones suma lógica (+), producto lógico (.) e inversión, se puede representar como:

$$f = f(a, b, c, d, \dots)$$

Es una expresión en la cual las variables se relacionan entre sí mediante las operaciones que habíamos dicho antes. Son las únicas operaciones permitidas en el Álgebra de Boole: sumar y multiplicar lógicamente, que difiere de la común, siendo que acá  $1 + 1 = 1$ .

## Funciones y formas canónicas

### **Término canónico**

Definimos término canónico de una función lógica a toda suma binaria (+) o producto binario (.) en la cual aparecen todas las variables ya sea en su forma directa o inversa.

Supongamos una función de tres variables  $f(a, b, c)$  los siguientes términos se conocen como:

**$a \cdot b \cdot c$  Producto Canónico (miniterm)**  
 **$a + b + c$  Suma Canónica (maxiterm)**

Cualquier variable podría estar negada o directa, se dice que el término es canónico porque si yo sé que de antemano la función tiene 3 variables, aquí ya tengo todas. Si le llegase a faltar alguno no sería canónico.

### **Formas Canónicas**

Tienen que estar compuestas por términos productos o por términos sumas no mezclados, o son todos productos o son todas sumas, en ese caso se llaman funciones canónicas.

Cuando una función se expresa como **suma de productos canónicos** o como **producto de sumas canónicas** se dice que la misma se encuentra en su forma canónica.

### **Ejemplo de sumas de productos canónicos.**

$$f(a, b, c) = \bar{a} \cdot b \cdot \bar{c} + \bar{a} \cdot b \cdot c + a \cdot \bar{b} \cdot c$$

$$(0 \ 1 \ 0)_2 \quad (0 \ 1 \ 1)_2 \quad (1 \ 0 \ 1)_2$$

$$(2)_{10} \quad (3)_{10} \quad (5)_{10}$$

Por convención diremos que una variable negada (raya arriba) se le asigna valor cero y a una sin negar (sin raya arriba) se le asigna valor 1. Se llama negar una variable cuando le pones la raya y directa en caso contrario.

### **Representación numérica**

$$= \sum_3 (2, 3, 5)$$

**Sumatoria de productos**

### **Ejemplo de producto de sumas**

$$f(a, b, c) = (\bar{a} + \bar{b} + c) \cdot (\bar{a} + b + \bar{c}) \cdot (a + b + c) = \prod_3 (1, 2, 7)$$

$$(0 \ 0 \ 1)_2 \quad (0 \ 1 \ 0)_2 \quad (1 \ 1 \ 1)_2$$

$$(1)_{10} \quad (2)_{10} \quad (7)_{10}$$

**Productoria de sumas**

Método Algebraico**Obtencion de sumas de productos canonicos**

Supongamos que yo tenga una expresion pequeña que no sea canonica, no son todos terminos canonicos, la misma puede ser llevada a una función canónica.

- 1) Se le aplica a la funcion logica la propiedad distributiva del producto respecto de la suma llevando a la expresion a una suma de productos no canonicos
- 2) A cada termino no canonico se lo multiplica por la suma de las variables que faltan y sus inversos
- 3) Finalmente se aplica nuevamente la propiedad distributiva del producto respecto de la suma llevando ahora a cada termino a su forma canonica

**Ejemplo:**

## Sumas de Productos Canónicos

### Ejemplo

$$\begin{aligned}
 f(a,b,c) &= a(\bar{b} + \bar{c}) + c && \text{Distributiva del producto respecto de la suma} \\
 &= a \cdot \bar{b} + a \cdot \bar{c} + c \\
 &= a \cdot \bar{b} \cdot (c + \bar{c}) + a \cdot (b + \bar{b}) \cdot \bar{c} + (a + \bar{a}) \cdot (b + \bar{b}) \cdot c \\
 &= a \cdot \bar{b} \cdot c + a \cdot \bar{b} \cdot \bar{c} + a \cdot b \cdot \bar{c} + a \cdot \bar{b} \cdot c + (a + \bar{a}) \cdot b \cdot c + (a + \bar{a}) \cdot \bar{b} \cdot c \\
 &= \cancel{a \cdot \bar{b} \cdot c} + \cancel{a \cdot \bar{b} \cdot \bar{c}} + a \cdot b \cdot \bar{c} + \cancel{a \cdot \bar{b} \cdot c} + a \cdot b \cdot c + \bar{a} \cdot b \cdot c + \cancel{a \cdot \bar{b} \cdot c} + \cancel{a \cdot \bar{b} \cdot c} \\
 &= a \cdot b \cdot c + a \cdot \bar{b} \cdot \bar{c} + a \cdot b \cdot \bar{c} + a \cdot b \cdot c + \bar{a} \cdot b \cdot c + \bar{a} \cdot \bar{b} \cdot c \\
 &= 101_2 \quad 100_2 \quad 110_2 \quad 111_2 \quad 011_2 \quad 001_2 \\
 f(a,b,c) &= \sum_3(5,4,6,7,3,1) = \sum_3(1,3,4,5,6,7)
 \end{aligned}$$
**Obtención de producto de sumas canónicas**

- 1) Se aplica a la función lógica la propiedad distributiva de la suma respecto el producto llevando la expresión a productos de suma no canónicos.
- 2) A cada termino no canónico se le suma el producto de las variables que faltan y sus inversos
- 3) Finalmente se aplica nuevamente la propiedad distributiva de la suma respecto del producto llevando ahora a cada término a su forma canónica.

**Ejemplo**

## Productos de Sumas Canónicos

### Ejemplo

$$\begin{aligned}
 f(a,b,c) &= a(\bar{b} + \bar{c}) + c && \text{Distributiva de la suma respecto del producto} \\
 &= (a+c) \cdot (\bar{b} + \bar{c}) \\
 &= (a+c) \cdot (\bar{b} + 1) \\
 &= (a+c) \cdot 1 \\
 &= (a+c) \\
 &= (a+c) + (b \cdot \bar{b}) \\
 &= (a+b+c) \cdot (a + \bar{b} + c) \\
 &= 111_2 \quad 101_2 \\
 f(a,b,c) &= \prod_3(5,7)
 \end{aligned}$$

Método práctico a partir de la tabla de verdad**Ejemplo:**

| a | b | c | f |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

Supongamos que tenemos esta tabla de verdad.

Se conecta con dos funciones canónicas, la suma de productos y el producto de sumas.

Se obtiene a partir de la tabla de verdad de la siguiente forma. Los valores para los cuales la función es "1" corresponden a Términos Canónicos Productos, es decir que al evaluar la función en cada variable arroja el valor "1".

Con los "0" hacemos los productos de sumas

**Para obtener los Términos Canónicos Productos se deberá leer en la parte izquierda de la tabla los "1" como las variables directas y los "0" como las variables negadas.**

Supongamos que queremos escribir la función de sumas de productos, tenemos que anotar cuando la función toma valor 1 y escribir la combinación de variables que permitió dicha salida,

| a | b | c | f |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

$$f(a,b,c) = (\bar{a}.\bar{b}.c) + (\bar{a}.b.c) + (a.\bar{b}.\bar{c}) + (a.b.\bar{c}) + (a.b.c)$$

$$f(a,b,c) = \sum_3 (1,3,4,6,7)$$

Sabemos que anotamos bien las variables porque los 1 aparecen en el mismo lugar que indica la sumatoria. Lo mismo se hace con el producto de sumas canónico, nomás que se hace con los ceros

| a | b | c | f | $\Sigma$ |
|---|---|---|---|----------|
| 0 | 0 | 0 | 0 | 0        |
| 0 | 0 | 1 | 1 | 1        |
| 0 | 1 | 0 | 0 | 2        |
| 0 | 1 | 1 | 1 | 3        |
| 1 | 0 | 0 | 1 | 4        |
| 1 | 0 | 1 | 0 | 5        |
| 1 | 1 | 0 | 1 | 6        |
| 1 | 1 | 1 | 1 | 7        |

Se parte de tener la tabla de verdad de una función lógica sea por ejemplo:

| a | b | c | f | $\Pi$ |
|---|---|---|---|-------|
| 0 | 0 | 0 | 0 | 7     |
| 0 | 0 | 1 | 1 | 6     |
| 0 | 1 | 0 | 0 | 5     |
| 0 | 1 | 1 | 1 | 4     |
| 1 | 0 | 0 | 1 | 3     |
| 1 | 0 | 1 | 0 | 2     |
| 1 | 1 | 0 | 1 | 1     |
| 1 | 1 | 1 | 1 | 0     |

**Producto de Sumas canónicos**

Los valores para los cuales la función es "0" corresponden a Términos Canónicos Suma, es decir que al evaluar la función en cada variable arroja el valor "0".

**Para obtener los términos Canónicos Suma se deberá leer en la parte izquierda de la tabla los "0" como variables directas y los "1" como variables negadas.**

$$f(a,b,c) = (a+b+c) . (a+\bar{b}+c) . (\bar{a}+b+\bar{c})$$

$$f(a,b,c) = \sum_3 (7,5,2)$$

## Simplificación

### **Criterio de Simplificación o minimización**

Obtener una expresión en forma de suma de productos o producto de sumas que contenga la “menor cantidad de términos posibles con la menor cantidad de variables en cada uno de ellos”, tratar de agarrar algunas de esas canónicas que vimos que tienen montonazos de letras y tratar de llevarla a una suma ya no canónica, ahora a una suma de productos o productos de sumas pero con pocos términos y pocas letras. Cada término con la mínima cantidad de letras

$$\begin{aligned} a.b.c. .... + \bar{a}.b.c. .... &= b.c. .... \\ (a+b+c+....) . (\bar{a}+b+c+....) &= (b+c+....) \\ (\cancel{a}+\bar{a}).(b.c. ....) &= 1.(b.c. ....) = b.c. .... \end{aligned}$$

La propiedad se llama adyacencia lógica

Se dice que dos términos canónicos son lógicamente adyacentes cuando difieren en el estado de una de las variables.

Por ejemplo la variable “a” difieren, una está negada y otra no.

Si tienes dos términos lógicamente adyacentes los puedes escribir como un solo término en el cual se ha eliminado dicha variable. Dos términos son lógicamente adyacentes cuando solo difieren en una de las variables, en el estado de solo una y ¿Qué se hace con dos términos lógicamente adyacentes? Se escribe un único término en el cual se ha eliminado dicha variable.

$$f(a,b,c) = \underbrace{a.b.c + \bar{a}.b.c}_{bc} + \underbrace{a.\bar{b}.c + \bar{a}.\bar{b}.c}_{\bar{b}.c} = c \checkmark$$

Teniendo lo anterior en cuenta ya ahora si podemos hablar formalmente de

### **Simplificación**

Aplicando la propiedad vista se puede reducir al mínimo cualquier expresión en forma de suma de productos o productos de sumas canónicas (eliminando de una variable a la vez).

Cuando no se puede suprimir ningún término ni eliminar más variables se llega a la expresión mínima e irreducible

### Método Tabular de Karnaugh de Simplificación

Es un método tabular, usa una gráfica, una tabla, haciendo mucho más fácil agrupar términos lógicamente adyacentes, porque te los va a poner a todos los términos en una tabla de modo tal que todos aquellos que estén continuos o sea que son físicamente adyacentes corresponden a los términos que son lógicamente adyacentes.

Es una forma gráfica de representar la tabla de verdad de una función lógica. Los TC (Términos Canónicos) lógicamente adyacentes se encuentran físicamente contiguos de manera que resulte sencillo el agrupamiento de los términos, permitiendo reducir al mínimo la función.

## Mapa de Karnaugh para dos variables

Karnaugh hace un mapa que tiene cuatro casilleros, un casillero para cada una de las líneas de la tabla, en los mismos se van a acomodar los términos estos

|   | a | b |                  |
|---|---|---|------------------|
| 0 | 0 | 0 | $\bar{a}\bar{b}$ |
| 1 | 0 | 1 | $\bar{a}b$       |
| 2 | 1 | 0 | $a\bar{b}$       |
| 3 | 1 | 1 | $ab$             |

Arriba voy a poner la variable a que puede valer 0 o 1, estas son las coordenadas del mapa.

Luego ponemos la letra b que puede valer 0 o 1 que también son coordenadas del mapa desde el lado horizontal.

| a \ b | 0 | 1 |
|-------|---|---|
| 0     |   |   |
| 1     |   |   |

Esto se juega como la batalla naval, cuando a vale 0 y b vale 0, o sea la primera línea de la tabla (a vale 0 por arriba y b vale 0 por la horizontal) el casillero que se forma es el casillero 0, que corresponde a la primera línea de la tabla.

Si a vale 0 y b vale 1, ese será el casillero 1

Si a vale 1 y b vale 0, ese será el casillero 2

Si a vale 1 y b vale 1 ese será el casillero 3

| a \ b | 0 | 1 |
|-------|---|---|
| 0     | 0 | 2 |
| 1     | 1 | 3 |

Ahora agarras los términos y metelo en su lugar, quedando de la siguiente manera:

| a \ b | 0                  | 1            |
|-------|--------------------|--------------|
| 0     | $\bar{a}\bar{b}$ 0 | $\bar{a}b$ 2 |
| 1     | $a\bar{b}$ 1       | $ab$ 3       |

Casilleros que tengan un lado en común, se tocan, son físicamente adyacentes tienen términos que son lógicamente adyacentes.

Busca cualquiera de los cuatro lados comunes y siempre son adyacentes lógicamente. Pero si miras cruzado deja de haber adyacencia. Coincide con la adyacencia física, en casilleros contiguos, son lógicamente adyacentes.

## Mapa de Karnaugh para 3 variables

|   | a | b | c |                         |
|---|---|---|---|-------------------------|
| 0 | 0 | 0 | 0 | $\bar{a}\bar{b}\bar{c}$ |
| 1 | 0 | 0 | 1 | $\bar{a}\bar{b}c$       |
| 2 | 0 | 1 | 0 | $\bar{a}b\bar{c}$       |
| 3 | 0 | 1 | 1 | $\bar{a}bc$             |
| 4 | 1 | 0 | 0 | $a\bar{b}\bar{c}$       |
| 5 | 1 | 0 | 1 | $a\bar{b}c$             |
| 6 | 1 | 1 | 0 | $ab\bar{c}$             |
| 7 | 1 | 1 | 1 | $abc$                   |

Karnaugh propone un mapa que obviamente tiene ahora 8 casilleros porque 8 son las líneas de la tabla, del 0 al 7.

Las coordenadas ahora son arriba las dos primeras (ab) y c en la parte de abajo

Por el lado de arriba como dos son las coordenadas pongo dos valores de índice (porque son dos variables "a" y "b"). En la parte de 11 y 10 no hay un error, es un truco de Don Karnaugh para que después adentro los términos se acomoden bien.

Por el lado de las horizontales c puede valer 0 y 1.

Si a vale 0, b vale 0 y c vale 0 entonces ese es el casillero 0 y así se hace sucesivamente rellenando las coordenadas.

Acuérdense que el 4 y el 5 van en el extremo porque se cambia de lugar el 11 el 10.

| ab \ c | 00 | 01 | 11 | 10 |
|--------|----|----|----|----|
| 0      |    |    |    |    |
| 1      |    |    |    |    |

| ab \ c | 00 | 01 | 11 | 10 |
|--------|----|----|----|----|
| 0      | 0  | 2  | 6  | 4  |
| 1      | 1  | 3  | 7  | 5  |

Finalmente reemplazando el mapa con las variables:

| $ab$ | 00                      | 01                | 11          | 10                |
|------|-------------------------|-------------------|-------------|-------------------|
| $c$  |                         |                   |             |                   |
| 0    | $\bar{a}\bar{b}\bar{c}$ | $\bar{a}b\bar{c}$ | $ab\bar{c}$ | $a\bar{b}\bar{c}$ |
| 1    | $\bar{a}bc$             | $\bar{a}b\bar{c}$ | $abc$       | $a\bar{b}c$       |

### Mapa de Karnaugh con 4 variables

| $a$ | $b$ | $c$ | $d$ |                                |
|-----|-----|-----|-----|--------------------------------|
| 0   | 0   | 0   | 0   | $\bar{a}\bar{b}\bar{c}\bar{d}$ |
| 1   | 0   | 0   | 0   | $\bar{a}\bar{b}\bar{c}d$       |
| 2   | 0   | 0   | 1   | $\bar{a}\bar{b}c\bar{d}$       |
| 3   | 0   | 0   | 1   | $\bar{a}\bar{b}cd$             |
| 4   | 0   | 1   | 0   | $\bar{a}b\bar{c}\bar{d}$       |
| 5   | 0   | 1   | 0   | $\bar{a}b\bar{c}d$             |
| 6   | 0   | 1   | 1   | $\bar{a}bcd$                   |
| 7   | 0   | 1   | 1   | $\bar{a}bcd$                   |
| 8   | 1   | 0   | 0   | $a\bar{b}\bar{c}\bar{d}$       |
| 9   | 1   | 0   | 0   | $a\bar{b}\bar{c}d$             |
| 10  | 1   | 0   | 1   | $a\bar{b}c\bar{d}$             |
| 11  | 1   | 0   | 1   | $a\bar{b}cd$                   |
| 12  | 1   | 1   | 0   | $ab\bar{c}\bar{d}$             |
| 13  | 1   | 1   | 0   | $ab\bar{c}d$                   |
| 14  | 1   | 1   | 1   | $abc\bar{d}$                   |
| 15  | 1   | 1   | 1   | $abcd$                         |

Es siempre hacer lo mismo, no hay cambios.

| $ab$ | 00                             | 01                       | 11                 | 10                       |
|------|--------------------------------|--------------------------|--------------------|--------------------------|
| $cd$ |                                |                          |                    |                          |
| 00   | $\bar{a}\bar{b}\bar{c}\bar{d}$ | $\bar{a}b\bar{c}\bar{d}$ | $ab\bar{c}\bar{d}$ | $a\bar{b}\bar{c}\bar{d}$ |
| 01   | $\bar{a}\bar{b}c\bar{d}$       | $\bar{a}b\bar{c}d$       | $ab\bar{c}d$       | $a\bar{b}c\bar{d}$       |
| 11   | $\bar{a}\bar{b}cd$             | $\bar{a}bcd$             | $abcd$             | $a\bar{b}cd$             |
| 10   | $\bar{a}b\bar{c}\bar{d}$       | $\bar{a}b\bar{c}d$       | $ab\bar{c}\bar{d}$ | $ab\bar{c}d$             |

Recuerden que se cambian de lugar el 11 y el 10 por eso los números de casilleros tienen un orden "raro".

### ¿Cómo se hace para simplificar?

Cada cuadro corresponde a un TC (Sumas o Productos) cuyo equivalente decimal se ubicara en el vértice inferior derecho.

Los cuadros físicamente contiguos (lado en común) corresponden a TC lógicamente adyacentes.

Los cuadros correspondientes a TC que forman parte de la función se indican con un "1" y los que no se dejan en blanco.

### Procedimiento de simplificación

Para obtenerla expresión algebraica más sencilla (mínima) se deberá realizar en la tabla el mínimo numero de agrupamientos de la máxima complejidad de manera de cubrir todos los "unos" de la tabla (eliminación de variables).

- 1) Se toman todos los "unos" que no se pueden agrupar con ningún otro "uno"
- 2) Se forman los grupos de dos "unos" que no pueden formar un grupo de cuatro "unos"
- 3) Se forman los grupos de cuatro "unos" que no pueden formar un grupo de ocho "unos"
- 4) Cuando se han cubierto todos los "unos" se detiene el proceso.

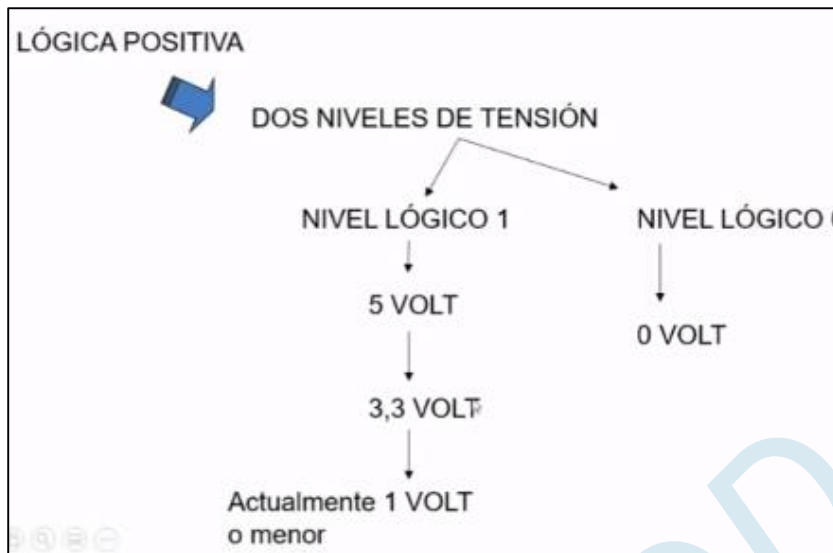
Los grupos deben ser potencia de dos.

Esto se verá en la parte práctica del apunte.

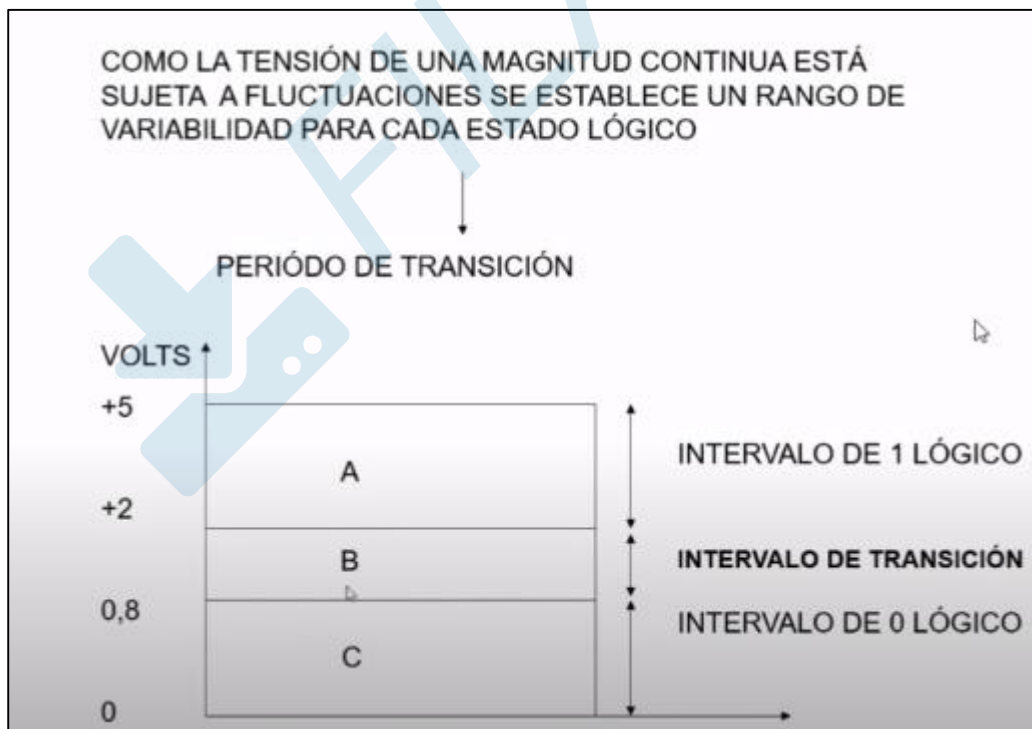
## Compuertas Lógicas

Las compuertas lógicas son símbolos entonces que sirven para entender las operaciones lógicas como la suma (compuerta or), multiplicación (Compuerta and) e inversión (compuerta not).

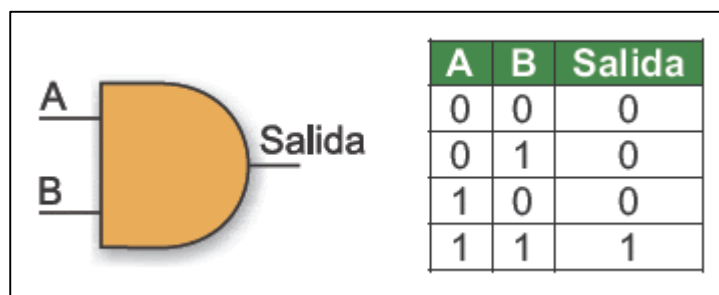
Son dos niveles de tensión en la lógica positiva, el 1 era de 5 voltes en la antigüedad, luego se bajó a 3,3 voltes, y actualmente la tensiones son tan bajas como 1 voltio o menos, entonces el nivel lógico 1 es la tensión de alimentación y 0 voltes representa al nivel lógico 0.



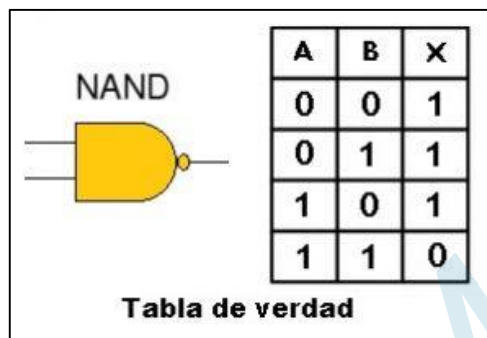
Tomemos la tensión de 5 voltes y fijemonosno que hay un margen de funcionamiento. Por ejemplo, tensiones comprendidas entre dos y cinco van a ser entendidas como el 1 lógico. Si vamos a tomar esa tensión hay un margen para decir que cualquier valor que más o menos se halla entre esto es un 1 y luego cualquier valor cercano al cero hasta casi un voltio (si trabajamos con el 5) es un intervalo para el 0. Lo que queda es un intervalo de transición que no es ni 1 ni 0 lógico.



## Compuertas AND



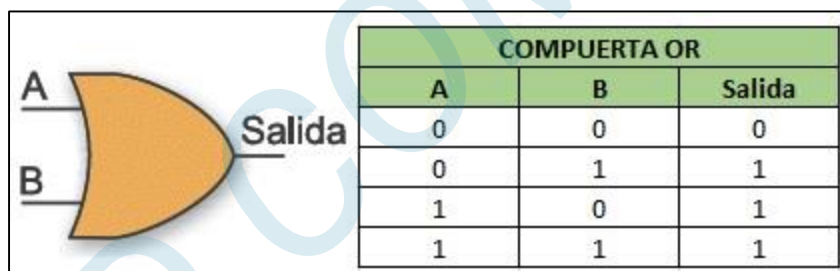
## Compuerta NAND



## Compuertas NOR

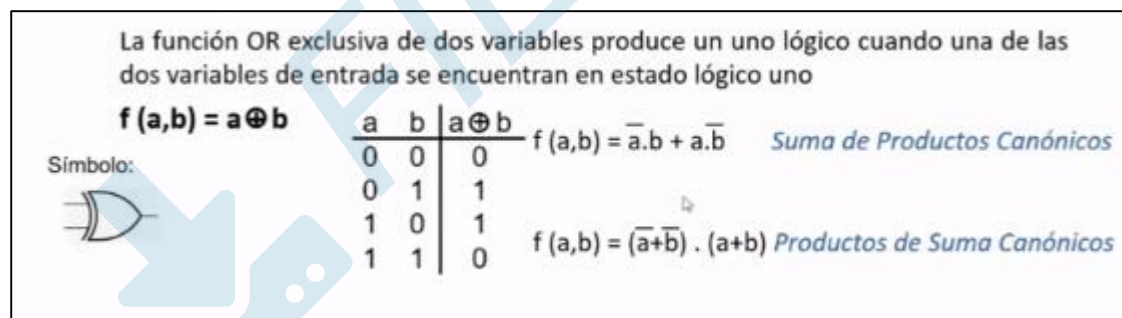


## Compuerta OR



## Función OR exclusiva

La función or exclusiva te vale uno cuando son distintas las variables de entrada, la puedes escribir como suma de producto o productos de sumas. No son adyacentes, no se puede eliminar nada, hay que gastar muchas puertas para armar todo esto, por tanto armaron esta compuerta para no gastar tanto.



Si ahora tenes que hacer una or exclusiva de muchas entradas sonaste, no se puede, para hacerlo deberías ir conectando una a continuación de la otra en cascada. Se debe aplicar primero a dos variables y luego el resultado de este a la tercera variable y así sucesivamente.

| a | b | c | f |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 1 |

La función se prende cuando hay números impares de 1 (tabla de la derecha) y cuando hay par te da 0 (tabla de la izquierda). Sirve para hacer el cálculo de bit paridad, o para el código de hamming.

| a | b | c | f |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 1 |

Función NOR exclusiva (X-NOR)

La definición es todo lo contrario de lo anterior, se le llama función de equivalencia o comparador, porque cuando los bits son iguales te prende un 1 y cuando son distinto te escribe un 0. Es lo opuesto de lo anterior.

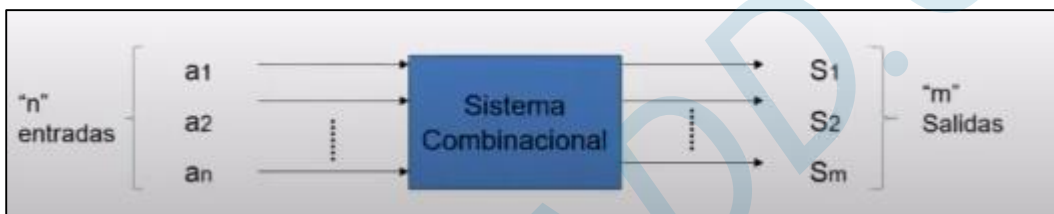
La función **NOR** exclusiva de dos variables produce un uno lógico cuando las dos variables de entrada se encuentran en el mismo estado lógico. Equivalencia o igualdad.

**$f(a,b) = a \oplus b$**

| a | b | $a \oplus b$ |
|---|---|--------------|
| 0 | 0 | 1            |
| 0 | 1 | 0            |
| 1 | 0 | 0            |
| 1 | 1 | 1            |

$f(a,b) = \bar{a} \cdot \bar{b} + a \cdot b$  *Suma de Productos Canónicos*

$f(a,b) = (a + \bar{b}) \cdot (\bar{a} + b)$  *Productos de Suma Canónicos*

Sistemas Combinacionales

Ustedes ven ese diagrama de bloques, del lado izquierdo hay entrada, y del derecho salida. Y adentro del sistema está lleno de compuertas electrónicas conectadas de alguna forma astuta para resolver un problema particular.

Son aquellos sistemas en los cuales en cada instante el estado lógico de la salida depende solamente del estado lógico de las variables de entrada. Cambia la entrada y por ende cambia la salida. Podemos tener muchas salidas que dependen de las mismas variables de entrada.

Pasos para diseñar un sistema combinacional

- 1) A partir de los requerimientos de un usuario (enunciado) identificar y codificar las variables de entrada y salida. Esto es, detectar cuáles son las variables de entrada y salida y asignarles nombres y un nivel lógico correspondiente. Declarar que es el 0 y que es el 1.
- 2) Obtención de la tabla de verdad. A partir de las variables vos sabes que a la izquierda pones todas las variables de entrada y a la derecha las salidas y empezás a confeccionar la tabla de verdad
- 3) Con la tabla de verdad sacás las canónicas (Suma de Producto y Producto de sumas)
- 4) Simplificás con métodos analíticos o por métodos gráficos como karnaugh
- 5) Implementar el circuito lógico con compuertas conocidas (And, Or, etc.).

Todo eso se ve en el práctico.

## Circuitos combinacionales de base

### Circuitos de mediana escala de integración (Medium Scale of Integration MSI)

Surgen allá por la década del 70 y algo, más adelante, cuando la industria empieza a incorporar muchos transistores en una misma pastilla de silicio.

Hasta ahora veíamos viendo baja escala de integración (LSI Low Scale of Integration) hay pocas compuertas que están asociadas a pocos transistores en el anterior.

Pero los circuitos de mediana escala de integración poseen una alta cantidad de compuertas adentro, puede haber 200 compuertas en el anterior por ejemplo.

Esos circuitos fueron funciones que se repitieron muchas veces, los investigadores hicieron un bloque, una macro.

Hoy en día se llama **VLSI** (Very LARGE Scale Of Integration) la tecnología actual, donde se usan las mismas cosas de siempre, nada más que densamente integrado.

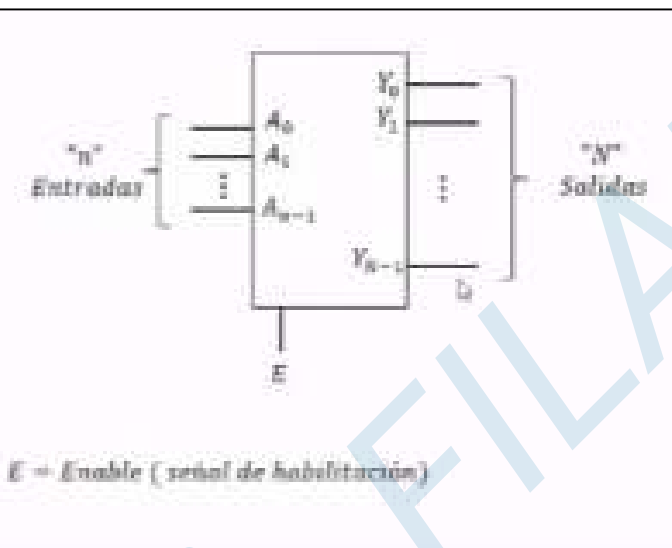
Hacer esto permite reducir el número de elementos necesarios en su realización y disminuir el tiempo de diseño.

### Decodificador 1 entre N

Se llama así porque las salidas son N mayúsculas y de todas ellas solo una va a estar prendida, mientras que el resto estará apagada, la que está prendida será la que corresponde a la combinación binaria que pusiste en la entrada. Si en todas las entradas pusiste 0 por ejemplo, se prenderá la primera salida.

Tenemos una relación que se cumple siempre, las entradas son poquitas "n" chiquitas y  $2^n = N$  las salidas son un montón. Así funciona el circuito, si vos le pones un 2 a n tenes  $2^2 = 4$  es un decodificar a 4, y cuando pones 0 0 se prende la de arriba, 0 1 se prende la que sigue, 1 0 la del medio, y 1 1 la última. Así funcionaría.

Después tenes en el bloque vas a tener una señal de habilitación (E = enable) si esto es 1 funciona lo anterior y si es 0 todas las salidas van a estar apagadas a pesar de que se modifiquen las entradas.



Ejemplo: Si  $n=2$  Decodificador de 2 a 4

| E | $A_1$ | $A_0$ | $Y_0$ | $Y_1$ | $Y_2$ | $Y_3$ |
|---|-------|-------|-------|-------|-------|-------|
| 1 | 0     | 0     | 1     | 0     | 0     | 0     |
| 1 | 0     | 1     | 0     | 1     | 0     | 0     |
| 1 | 1     | 0     | 0     | 0     | 1     | 0     |
| 1 | 1     | 1     | 0     | 0     | 0     | 1     |
| 0 | x     | x     | 0     | 0     | 0     | 0     |

Tabla de verdad

$$\begin{aligned}
 Y_0 &= \bar{A}_1 \cdot \bar{A}_0 \\
 Y_1 &= \bar{A}_1 \cdot A_0 \\
 Y_2 &= A_1 \cdot \bar{A}_0 \\
 Y_3 &= A_1 \cdot A_0
 \end{aligned}$$

minitérminos

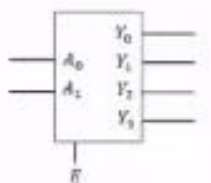


Diagrama de Bloques

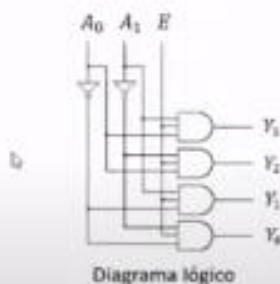


Diagrama lógico

$$2^n = N$$

En la tabla de verdad, mientras E vale 1, si pongo 0 0 en la entrada, la salida que se prende es la Y0, mientras las demás están apagadas, si pongo 0 1 la salida que se prende es la Y1 mientras todas las demás están apagadas, si pongo 1 0 se prende Y2 y si pongo 1 1 se prende Y3.

Si E está en 0 la x dice hace lo que quieras porque total todo va a estar apagado.

### Implementación de funciones lógicas usando un decodificador

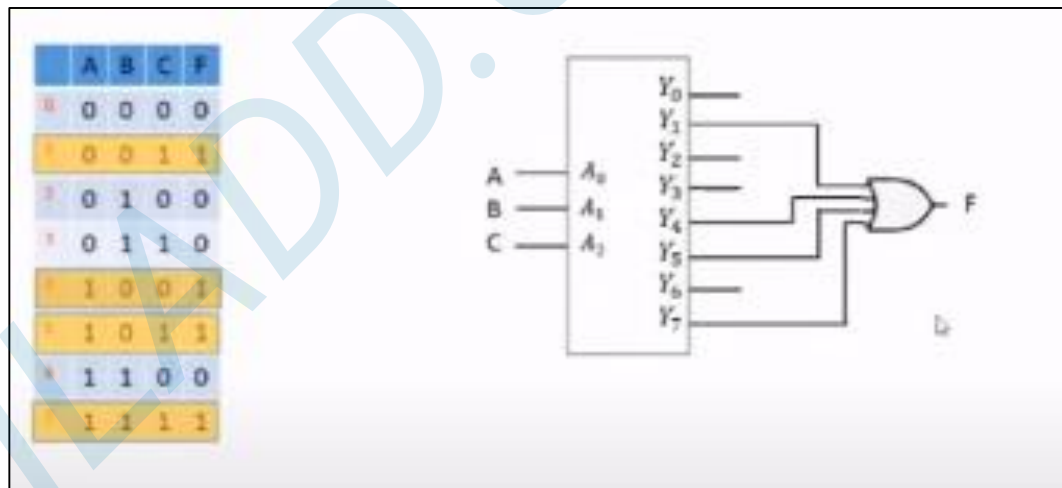
Un decodificador también puede utilizarse para implementar funciones lógicas en la forma de suma de productos, basta colocar una compuerta OR que tome todas las salidas correspondientes para las cuales la función tiene que vale 1.

Ejemplo: sea una función de tres variables cualquiera:

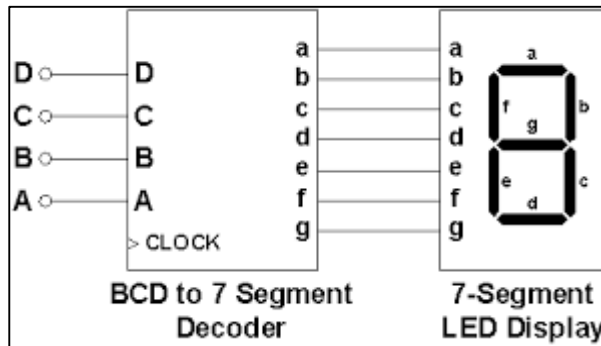
|   | A | B | C | F |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 | 1 |
| 2 | 0 | 1 | 0 | 0 |
| 3 | 0 | 1 | 1 | 0 |
| 4 | 1 | 0 | 0 | 1 |
| 5 | 1 | 0 | 1 | 1 |
| 6 | 1 | 1 | 0 | 0 |
| 7 | 1 | 1 | 1 | 1 |

Como ven yo estiré las líneas 1, 4, 5,7 que son las que tienen una salida 1 en la función.

Vos podés hacer karnaugh, como siempre, pero si vos quisieras velocidad de diseño, ponés un decodificador de 3 entradas y 8 salidas  $2^3 = 8$ , en estas entradas del decodificador le ponés las tres letras A, B, C las conectas ahí, luego agarrás y te fijás donde están los 1 porque es una de producto, entonces lo que tenés que hacer es prolongar esas líneas de salida porque van a ser las únicas que líneas que usarás. Cuando le des valores a la función vas cambiando el valor de la variable, una línea de la tabla va a ocurrir a un tiempo, mientras vos le vas dando las líneas que vos estiraste son las que se van a prender cuando están las combinaciones que la función quiere, por lo tanto agarro y les pongo una compuerta or que tenga tantas entradas como 1 había y eso quiere decir que si se prende una que no está conectada no pasa nada, se prende la salida pero como no está conectada no pasa a la Or, no hace que la salida F se haga 1.



## Decodificador de BCD a 7 segmentos



Esto le toco hacer muchas veces a los diseñadores, por tanto hicieron un chip que lo haga solo.

Acá tenes el bloque, diagrama de bloque, es el circuito combinacional de base porque siempre se repite, es para manejar el 8 de luces, se pusieron 8 luces llamadas: a, b, c, d, e, f, y la del centro g, todo el mundo entero lo llama así. Entonces esto es un circuito que hay que realizarlo para que cuando entre una información en BCD, entonces el circuito haga la conversión, lo decodifique y prenda las luces que correspondan para representar números.

| E | A | B | C | D | a | b | c | d | e | f | g |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| 1 | 0 | 1 | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 1 | 1 |
| 1 | 0 | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 1 | 1 |
| 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 | 0 | X | X | X | X | X | X | X |
| 1 | 1 | 0 | 1 | 1 | X | X | X | X | X | X | X |
| 1 | 1 | 1 | 0 | 0 | X | X | X | X | X | X | X |
| 1 | 1 | 1 | 0 | 1 | X | X | X | X | X | X | X |
| 1 | 1 | 1 | 1 | 0 | X | X | X | X | X | X | X |
| 1 | 1 | 1 | 1 | 1 | X | X | X | X | X | X | X |
| 0 | X | X | X | X | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Tabla de Verdad

Haces una tabla de verdad, donde la variables de entrada que van a ser el BCD, A B C y D. Y a la derecha van las 7 luces que son los segmentos a, b, c, d, e, f, g.

Vas a escribir todas las combinaciones de las variables con la tabla y a la derecha vas completar con cuales son las luces que tienen que estar prendidas para cada combinación. Mientras yo tenga E = 1, si a las variables de entrada le pones 0 0 0 0 hay que prender el número 0, prendemos todas las luces menos la del centro que es la g, con eso podremos ver el 0.

Para prender el 1, hay que a prender a b y a c, para prender el numero 2 hay que prender a todos menos al segmento f y c.

Y así sucesivamente para representar a cada número hasta al 9, por eso necesitaste 4 bits, porque con 3 bits llegaste hasta el 7. Por eso el resto tiene una x porque no me importa lo que viene después.

## Circuito codificador decimal a BCD

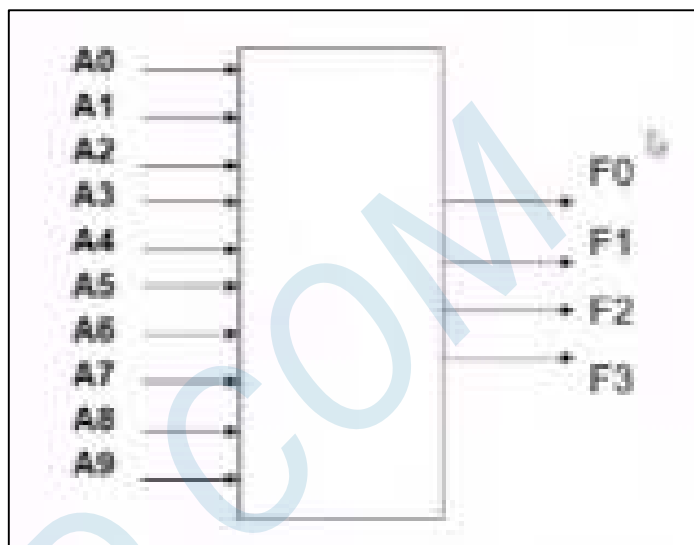
Trabaja justamente al revés del decodificador. Son sistemas que se encargan de transformar una serie de señales sin codificar en un conjunto que corresponde a un código.

Tiene N entradas y n salidas debiéndose cumplir la siguiente condición.

$$N \leq 2^n$$

Pasa de un código 1 entre n a un código binario, cuando por ejemplo está habilitado y vos apretas la primera de

las teclas (0) la función que se te forma se te codifica al 0 en BCD. Si apretas la tecla número 2 te queda el número 2 que es el 0 0 1 0, si apretas la última tecla que es la 9 te va a quedar el 1 0 0 1 que es el 9 en BCD, apretas una tecla y te crea la codificación del otro lado, por tirar un ejemplo, para cada tecla la función te codificara en binario al otro lado. No se pueden apretar dos teclas juntas.



| A0 | A1 | A2 | A3 | A4 | A5 | A6 | A7 | A8 | A9 | F0 | F1 | F2 | F3 |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 1  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
| 0  | 0  | 1  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 0  |
| 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 0  | 1  | 1  | 0  | 0  | 1  |

## Multiplexor

Es un circuito que posee  $n + N$  entradas y una salida.

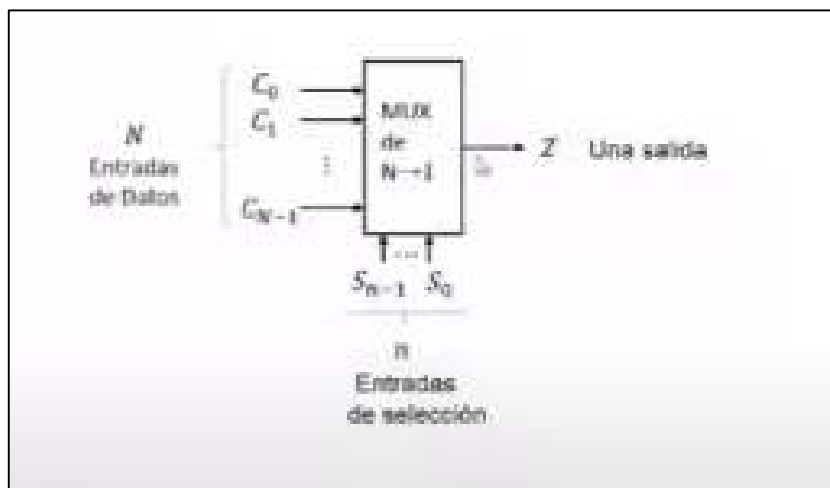
$n$  = entradas de selección

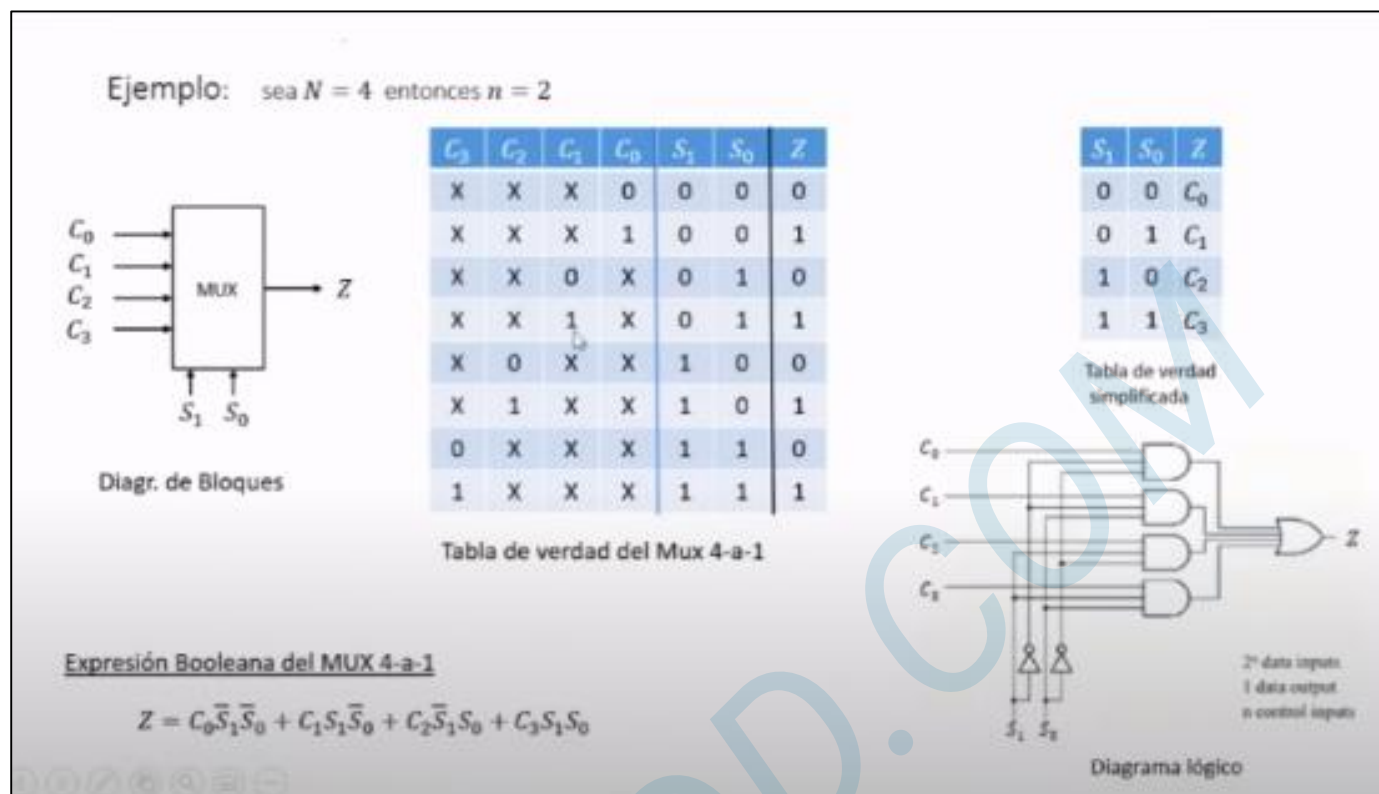
$N = 2^n$  entradas de datos

1 salida

## Ejemplo de su funcionamiento

Esto es muy usado en las comunicaciones con fibra óptica. Vos tenes una sola fibra óptica y muchos canales de entrada que representa a cada persona, y una sola salida porque fibra óptica hay 1 sola, entonces todas tienen que hablar al mismo tiempo por una única fibra óptica. Vas a usar un multiplexor para asignarle un tiempito a cada uno de ellos, cuando el señor del canal 0 quiera hablar lo vas a conectar por un instante a la salida, luego cambiamos la llave de posición al siguiente canal y así hasta llegar al último. Cambiamos de lugar la llave de posición con una combinación binaria que está en las entradas de selección. Cuando la entrada de selección sean todas 0 elegimos el primer canal que va a estar pasando a la fibra óptica, cuando todas las entradas de selección tengan 1 estarás eligiendo la última persona y así con cada combinación elige una persona diferente, así funciona el multiplexor.





Si vos seleccionas 0 0 en las entradas de selección, y el canal 0 tiene un 0 lógico, que pasa por tanto a la salida. Y los otros canales dan igual lo que tengan porque el elegido en este momento es el canal 0 así que no le voy a dar importancia a lo que tengan los demás, por eso le pone x ahí.

Si vos pones 0 0 y el canal 0 tenía un 1 te dice que ese 1 pasa a la salida. Cuando se elige entonces 0 0 las dos primeras líneas te dice que si el canal 0 tiene un 0 pasa un 0 y si tiene un 1 pasa un 1.

Si seguís con el resto de la tabla te vas a dar cuenta que es más de lo mismo, pero con los diferentes canales.

La ecuación dice que si vos tenes 0 0 en la selectora entonces se van a hacer 1 porque están negadas y por tanto responden a lo que tenga el canal cero debido a la multiplicación lógica. De esta forma podemos observar como el resto de términos dan todos 0 haciendo que si la entrada selectora es 0 0 nomás se tome en cuenta el valor del canal 0.

El diagrama lógico se entiende analizando las variables de entrada para las selectoras y analizando con quien está conectado y de qué forma (si usa el inversor o no), para cada combinación solo un canal se va a usar y para que eso suceda le tienen que llegar todos 1.

## Implementación de funciones lógicas con decodificador MUX

Si con el decodificador era una pavada porque vos solamente estiraba las líneas y ponías una or de tantas entrada como 1 tenías, bueno con el multiplexor es más ingenioso y es agradable porque encima sale también más reducido el circuito.

Elijo una función cualquiera de 3 variables que la voy a hacer con un multiplexor.

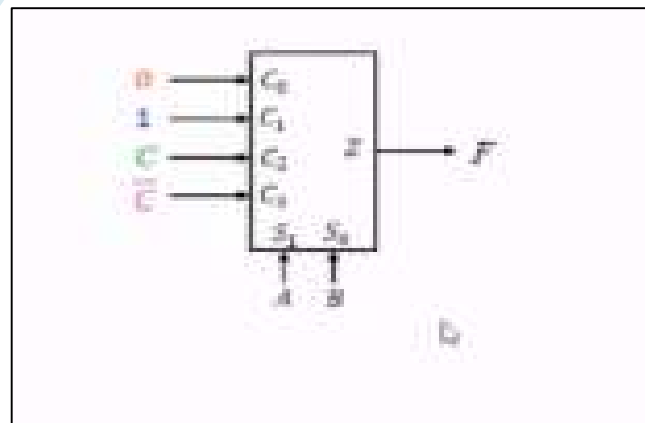
| A | B | C | F |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 1 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 0 |

Tomo entonces un multiplexor que tenga una entrada menos de selección que el número de variables que tiene la función, en la tabla de verdad veo que tiene 3 variables por tanto voy a elegir un multiplexor con una entrada de selección menor, 2 en este caso. A continuación se toma las dos más importantes de la función (A y B) y se las acomoda en la selectora A va al S1 porque es más importante y B al S0. Vos comenzás haciendo el análisis de pares en la tabla. Tomas las dos primeras líneas de la tabla y te preguntás si A y B valen 0 0 elegiste por tanto el canal 0, es el que va a pasar a la salida. Si te fijás la función en esos dos lugares tiene un 0, por tanto vas a poner 0 en la entrada del canal y como en este momento está elegido ese canal la función tiene lo que tiene, el resto da igual, porque no están elegidas y ya arreglaste para las dos primeras líneas de la tabla.

Para las siguientes dos pusiste A = 0 y B = 1 eligiendo así el canal 1, la función en esos dos canales tiene un 1, entonces el canal 1 le pongo un 1.

A continuación tomo los otros dos pares de líneas y veo que A = 1 y B = 0 por tanto estamos en el canal 2 pero la función tiene salida 0 1, entonces como los valores de la función son iguales que los que tiene C en esos dos pares de línea se coloca C en el canal número 3 del multiplexor.

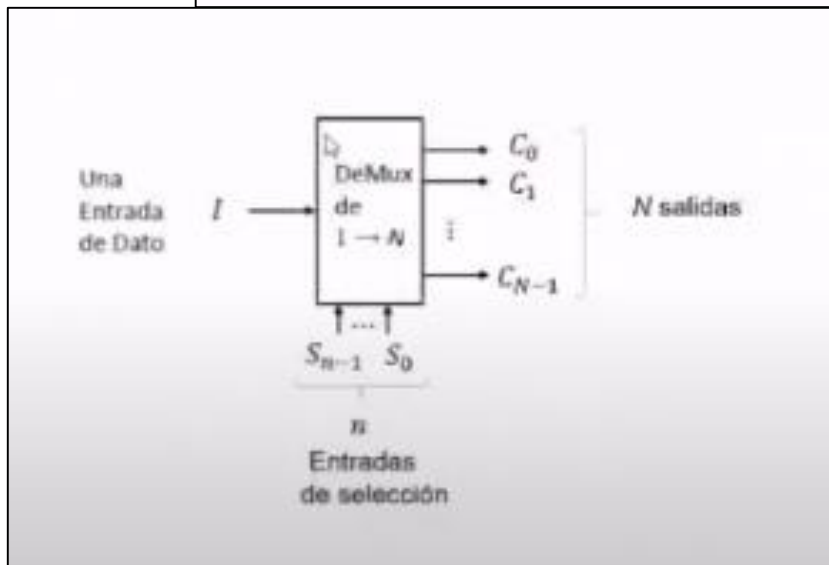
Luego tomo los últimos dos pares de línea, en la última combinación cuando las letras A y B valen 1 1, el canal elegido es el último, por tanto en la salida de la función tenemos 1 0, que es justamente lo opuesto que los valores mostrados por C, por tanto se pone C negado.



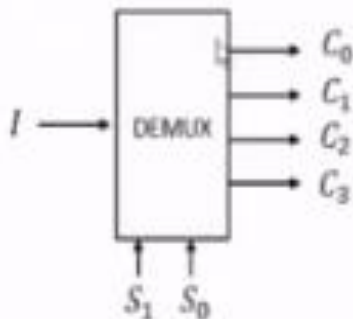
## Demultiplexor

Hace la tarea inversa al multiplexor, posee una entrada de datos, n entradas de selección y N canales de datos. Con  $N = 2^n$

Si acá llega la fibra óptica con un chorro de 1 y 0 de altísima velocidad, el selector lo que hace es ir pasándole al canal 0, al 1, al 2 y a las n salidas le va dando, un tiempito a cada uno un paquete de 1 y 0 y así repite y le va dando. Podemos relacionarlo con la telefónica, cuando una persona A quiere hablar con otra se disponen los selectores de cierta forma para que A pueda hablar con quien desee por un lapso de tiempo. Por cada ráfaga de comunicación que dura la comunica solo 8 1 y 0 viajan, para mandar la voz humana con 8 bits está muy bien, la conversión de señal analógica a digital lo hace el micrófono de nuestros celulares.



Ejemplo: sea  $N = 4$  entonces  $n = 2$



Diagr. de Bloques

| $S_1$ | $S_0$ | $C_3$ | $C_2$ | $C_1$ | $C_0$ |
|-------|-------|-------|-------|-------|-------|
| 0     | 0     | 0     | 0     | 0     | 1     |
| 0     | 1     | 0     | 0     | 1     | 0     |
| 1     | 0     | 0     | 1     | 0     | 0     |
| 1     | 1     | 1     | 0     | 0     | 0     |

Tabla de verdad del DeMux 1-a-4

#### Expresión Booleana del DEMUX 1-a-4

$$\begin{aligned} C_0 &= I \cdot \bar{S}_1 \cdot \bar{S}_0 \\ C_1 &= I \cdot \bar{S}_1 \cdot S_0 \\ C_2 &= I \cdot S_1 \cdot \bar{S}_0 \\ C_3 &= I \cdot S_1 \cdot S_0 \end{aligned}$$

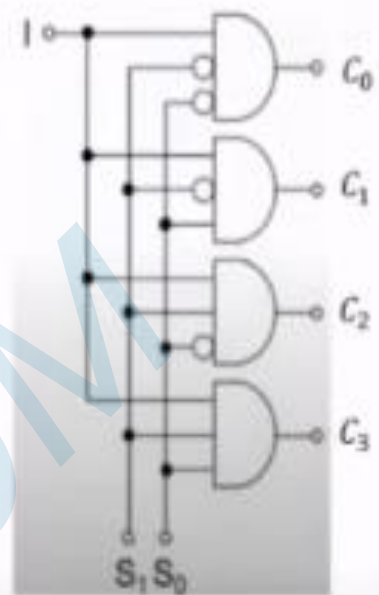


Diagrama lógico

Si el selector pone 0 0 el canal que pasa a la salida entonces es el 0, que va a recibir la información que entra por I, si pones 0 1 entonces el canal elegido es el 1, si pones 1 0 entonces el canal elegido es el número 2, por donde va a salir la información que entra y por ultimo si pones 1 1 el canal elegido es el último.

En el diagrama lógico toda la información le llega a la compuerta, pero solo aquello que tengan la selección correcta, como poner 0 0, como ahí está negado en la formula, y el multiplexor lo niega se hace 1 1 y por tanto se activa la compuerta del canal 0.

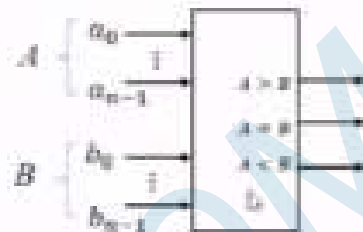
## Comparador binario

Otro de los circuitos lógicos muy utilizados, compara dos números binarios de muchos dígitos y te da como salida,  $A > B$ ,  $A = B$  o  $A < B$ . que son las tres salidas que tiene este circuito.

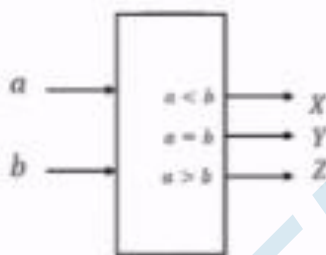
### Comparador de n bits

$$A = a_{n-1} a_{n-2} \dots a_1 a_0$$

$$B = b_{n-1} b_{n-2} \dots b_1 b_0$$



### Comparador de un bit



Diagr. de Bloques

| a | b | X       |   |   | Y       |   |   | Z       |   |   |
|---|---|---------|---|---|---------|---|---|---------|---|---|
|   |   | $a < b$ |   |   | $a = b$ |   |   | $a > b$ |   |   |
| 0 | 0 | 0       | 0 | 0 | 1       | 0 | 0 | 0       | 0 | 0 |
| 0 | 1 | 1       | 0 | 0 | 0       | 0 | 0 | 0       | 0 | 0 |
| 1 | 0 | 0       | 0 | 0 | 0       | 0 | 1 | 1       | 0 | 0 |
| 1 | 1 | 0       | 0 | 0 | 1       | 0 | 0 | 0       | 0 | 0 |

Tabla de verdad

Expresiones Booleanas:

$$X = \bar{a}b$$

$$Y = \bar{a}\bar{b} + ab$$

$$Z = a\bar{b}$$

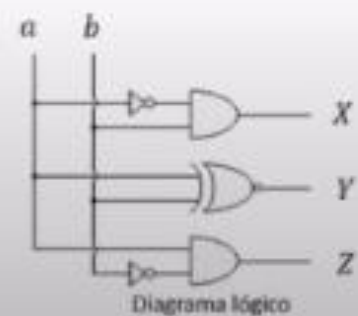


Diagrama lógico

En la primera línea veo que son iguales a y b, por tanto es decir, prendo la salida e indicándole que son iguales los datos que han entrado.

En la segunda línea a vale 0 y b vale 1 por tanto b es más grande por tanto tiene que prenderse X.

En la tercera línea a vale 1 y b vale 0 indicando que a es más grande que b por tanto se prende Z.

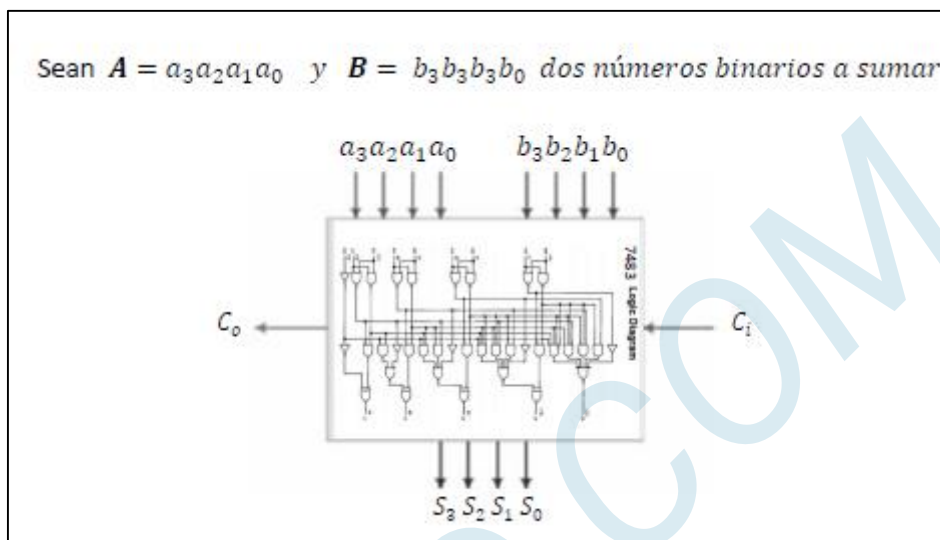
En la última a y b valen lo mismo por tanto se prende nuevamente Y.

## Sumadores

Los sumadores son muy importantes en los sistemas donde se procesan datos numéricos, realizan la suma aritmética de números binarios.

Las reglas básicas para la suma aritmética binaria son:

|            |
|------------|
| 0 + 0 = 0  |
| 0 + 1 = 1  |
| 1 + 0 = 1  |
| 1 + 1 = 10 |



Se le ha puesto acá un dato A que vale  $a_3a_2a_1a_0$  (el número más grande que puedes escribir es el 15 en decimal)  $15 + 15$  podrías hacer, pero a izquierda y a derecha hay unas entradas conocidas como carry o arrastre, quiere decir que si copias y pegas muchos módulos y lo vas enganchando en cascada, uno a continuación del otro como las fichas del domino que cae, vas a ir colocando estos circuitos y esta salida de carry se la conectas a otra entrada y así vas a poder empezar a expandir y hacer ya si pones uno más suma de números de 8 bits y si seguís poniendo vas a poder hacer sumas de números muchos más grandes llegando a la cantidad de 64 bits que es en la actualidad.

Tenés dos números A y B que se representan en binario, en la salida te sale la suma. Adentro está lleno de compuertas, entonces el diseñador solo toma el bloque y lo va pegando para hacer las sumas que requieran.

Es un circuito de mediana escala de integración.

### Semi-sumador (Half Adder)

El semisumador tiene carry de salida pero no tiene carry de entrada, un semisumador admite dos dígitos binarios en sus entradas y genera dos bits binarios en sus salidas: un bit de suma y uno de acarreo.

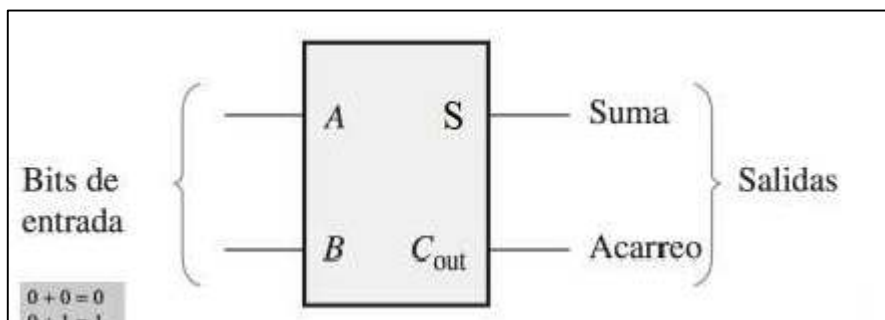
En la entrada tenés los dos bits, (1 bit por cada entrada) y entonces serían estas todas las posibilidades de sumar:

|            |
|------------|
| 0 + 0 = 0  |
| 0 + 1 = 1  |
| 1 + 0 = 1  |
| 1 + 1 = 10 |

Si haces  $0 + 0$  la suma te da 0 no te llevas nada en el acarreo

Si haces  $0 + 1$  o  $1 + 0$  entonces la suma te da 1 pero no te llevas nada

Si haces  $1 + 1$  pones en la salida un 0 y en el carry de salida un 1.



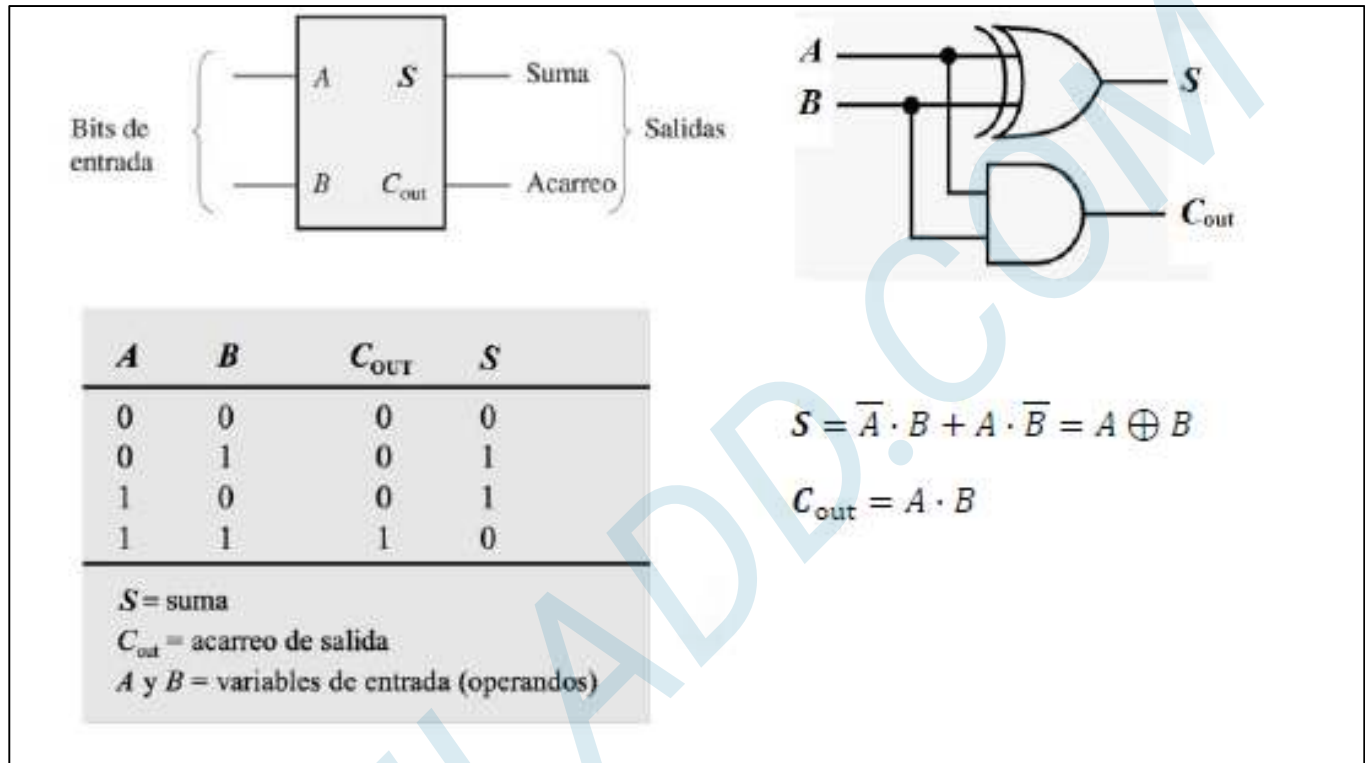
Es muy poco útil porque nomás suma de a 1 bit debido que no tenés carry de entrada y por ende no puedes poner en cascada para sumar más números.

Lógica del semisumador

Se hace a la suma con una OR exclusiva, la primera da 0 + 0 no te llevas nada, 1 + 0 o 0 + 1 da 1 la suma no te llevas nada y 1 + 1 te da 0 en la suma y te llevas uno en el acarreo.

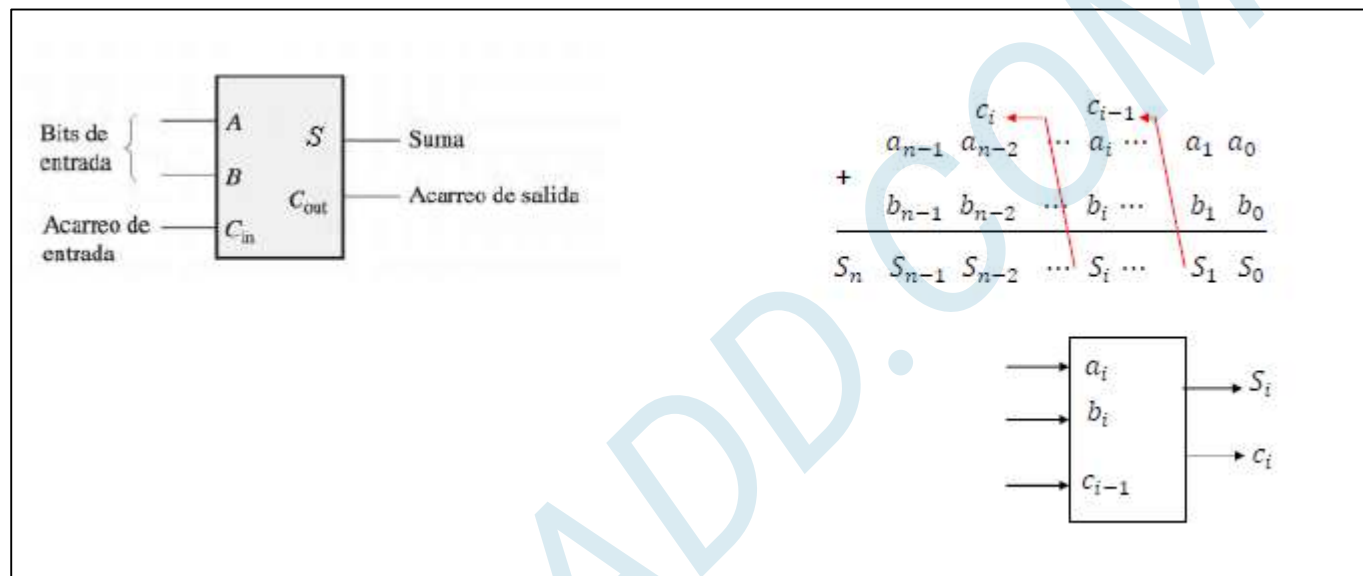
Si escribís las ecuaciones canónicas de la tabla de la verdad te da una OR exclusiva, que era la negada de la primera por la segunda sin negar más la primera por la negada de la segunda, es una operación que no se puede simplificar.

Para el carry hay 1 acá en el último que sería  $A \cdot B$ .

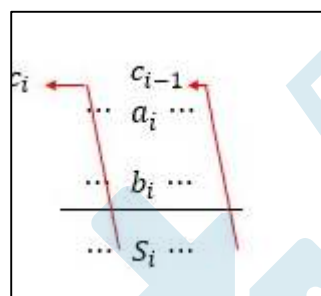


## Sumador total (Full Adder)

Tiene un acarreo de entrada a diferencia del semisumador. En vez de hacer un sumador de 4 bits, hago uno de un solo bit con entrada y salida de acarreo. Entonces si te toca hacer una suma de muchos bits vos vas a empezar desde izquierda a derecha y si te paras en una columna cualquiera para poder hacer la suma tenes que estar pendiente que se lleva la etapa anterior y si se llevó algo recibís eso por el carry de entrada. Luego vas a sumar las 3 cosas, si el carry está o no está vos siempre haces la suma de las 3, si no está no pasa nada. Cuando sumas las 3 cosas que serían los bits propiamente dicho más el arrastre anterior y ahí vas a poner el resultado que te toca y verás si te llevas algo, que lo vas a mandar por el carry out, es decir por el carry de salida. Es decir, corté a la suma en rebanadas. Si vos lo cortás en rodajas y vos sabes que todas hacen lo mismo, entonces estudio muy bien una, fabrico el circuito electrónico para esta tajada y luego lo voy a copiar y pegar tantas veces como sea necesario.



Vamos a ver cómo se construye esto para lo cual tomaré una tajada de la suma anterior.



Viendo la tabla de verdad me doy cuenta que son 3 variables de entrada que son los datos propiamente dichos de cada uno de los números de la columna que quieres sumar, que son el  $a_i$  y el  $b_i$  y el carry que viene de la etapa anterior.

En la primera fila de la tabla asumimos que  $a_i$  y  $b_i$  valen ambos 0 y por último que la etapa que venía anterior mando un 0, diciendo: "no se llevó nada". Entonces hago la suma:  $0+0+0$  el resultado es 0 para la suma y no me llevé nada, por eso coloco 0 en la salida y en el carry. Y así se hace con el resto de la tabla

|   | $a_i$ | $b_i$ | $c_{i-1}$ | $S_i$ | $C_i$ |
|---|-------|-------|-----------|-------|-------|
| 0 | 0     | 0     | 0         | 0     | 0     |
| 1 | 0     | 0     | 1         | 1     | 0     |
| 2 | 0     | 1     | 0         | 1     | 0     |
| 3 | 0     | 1     | 1         | 0     | 1     |
| 4 | 1     | 0     | 0         | 1     | 0     |
| 5 | 1     | 0     | 1         | 0     | 1     |
| 6 | 1     | 1     | 0         | 0     | 1     |
| 7 | 1     | 1     | 1         | 1     | 1     |

En la segunda línea  $a$  y  $b$  valen 0 pero parece que la columna de al lado se había llevado, entonces si hago la suma es:  $0+0+1 = 1$  entonces pongo el 1 en la suma y como no me llevo nada, pongo cero en el carry de salida.

Después hacemos el karnaugh, haciendo una función para la  $S_i$  y otra para el  $C_i$ , teniendo dos karnaugh distintos y paso los 1 y resolviendo como se vió en el práctico.

Como puede verse, en el karnaugh de Si son todos diagonales los 1 por tanto no se puede simplificar, me veo obligado a hacer áreas todas de 1, haciendo que te de lo mismo que haber hecho o no el karnaugh, obteniéndose así la función canónica:

|           |   |           |    |    |    |
|-----------|---|-----------|----|----|----|
|           |   | $a_i b_i$ |    |    |    |
|           |   | 00        | 01 | 11 | 10 |
| $c_{i-1}$ | 0 |           | 1  |    | 1  |
|           |   | 0         | 2  | 6  | 4  |
|           | 1 | 1         |    | 1  |    |
|           |   | 1         | 3  | 7  | 5  |
|           |   | $S_i$     |    |    |    |

$$S_i = \bar{a}_i \bar{b}_i c_{i-1} + \bar{a}_i b_i \bar{c}_{i-1} + a_i \bar{b}_i \bar{c}_{i-1} + a_i b_i c_{i-1}$$

Cuando no funciona Karnaugh seguro hay OR exclusivas ocultas, por tanto intento sacar factor común  $a_i$  negada entre los primeros dos términos y entre los últimos dos haces factor de  $a_i$  directo, quedando de la siguiente forma:

$$S_i = \bar{a}_i (\bar{b}_i c_{i-1} + b_i \bar{c}_{i-1}) + a_i (\bar{b}_i \bar{c}_{i-1} + b_i c_{i-1})$$

Si hacemos memoria y nos fijamos en la función de la OR exclusiva la podremos detectar que en el primer paréntesis del factor común encontramos una OR exclusiva y en el segundo paréntesis encontramos lo que corresponde a una NOR exclusiva. Por tanto se puede reescribir así:

$$S_i = \bar{a}_i (b_i \oplus c_{i-1}) + a_i (\bar{b}_i \oplus \bar{c}_{i-1})$$

La negada de la primera ( $a_i$ ) por el segundo término que está entre paréntesis sin negar + el primero sin negar ( $a_i$ ) x el negado de lo que está dentro del paréntesis hace que esto en realidad sea todo una gran OR exclusiva entre la letra  $a$  y lo que está dentro del paréntesis, entonces reescribo:

$$S_i = (a_i \oplus b_i \oplus c_{i-1})$$

Quedando así finalmente.

Ahora hacemos el Karnaugh de la segunda (la del acarreo  $c_i$ ) pasando las posiciones donde hay unos.

|           |   |           |    |    |    |
|-----------|---|-----------|----|----|----|
|           |   | $a_i b_i$ |    |    |    |
|           |   | 00        | 01 | 11 | 10 |
| $c_{i-1}$ | 0 |           |    | 1  |    |
|           |   | 0         | 2  | 6  | 4  |
|           | 1 | 1         | 1  | 1  |    |
|           |   | 1         | 3  | 7  | 5  |
|           |   | $C_i$     |    |    |    |

$$C_i = b_i c_{i-1} + a_i b_i + a_i c_{i-1}$$

De lo anterior y para ahorrar recursos podemos reutilizar algunos términos de los que teníamos antes en la salida para poder obtener OR exclusivas.

Pudiéndose escribir de la siguiente forma:

Primero se escribe toda la canónica de  $c_{i-1}$  para que después se pueda hacer factor común:

$$C_i = \bar{a}_i b_i c_{i-1} + a_i \bar{b}_i c_{i-1} + a_i b_i \bar{c}_{i-1} + a_i b_i c_{i-1}$$

En los dos primeros términos se saca factor común  $c_{i-1}$  y en los últimos dos términos saca factor común  $a_i b_i$ , quedando de la siguiente manera:

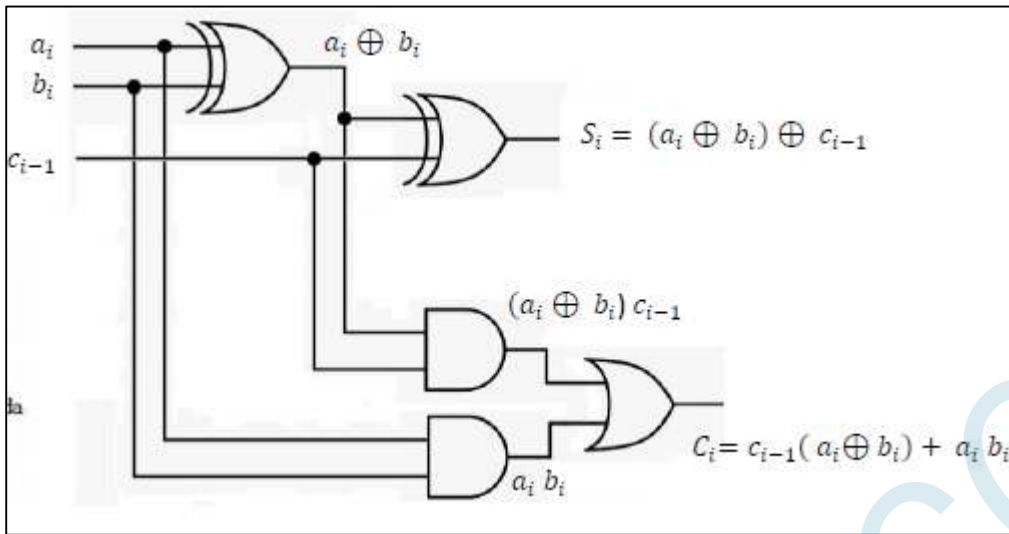
$$= c_{i-1} (\bar{a}_i b_i + a_i \bar{b}_i) + a_i b_i (c_{i-1} + c_{i-1})$$

Dentro del primer paréntesis te queda una OR exclusiva y en el segundo paréntesis se eliminan porque una es negada y la otra es directa.

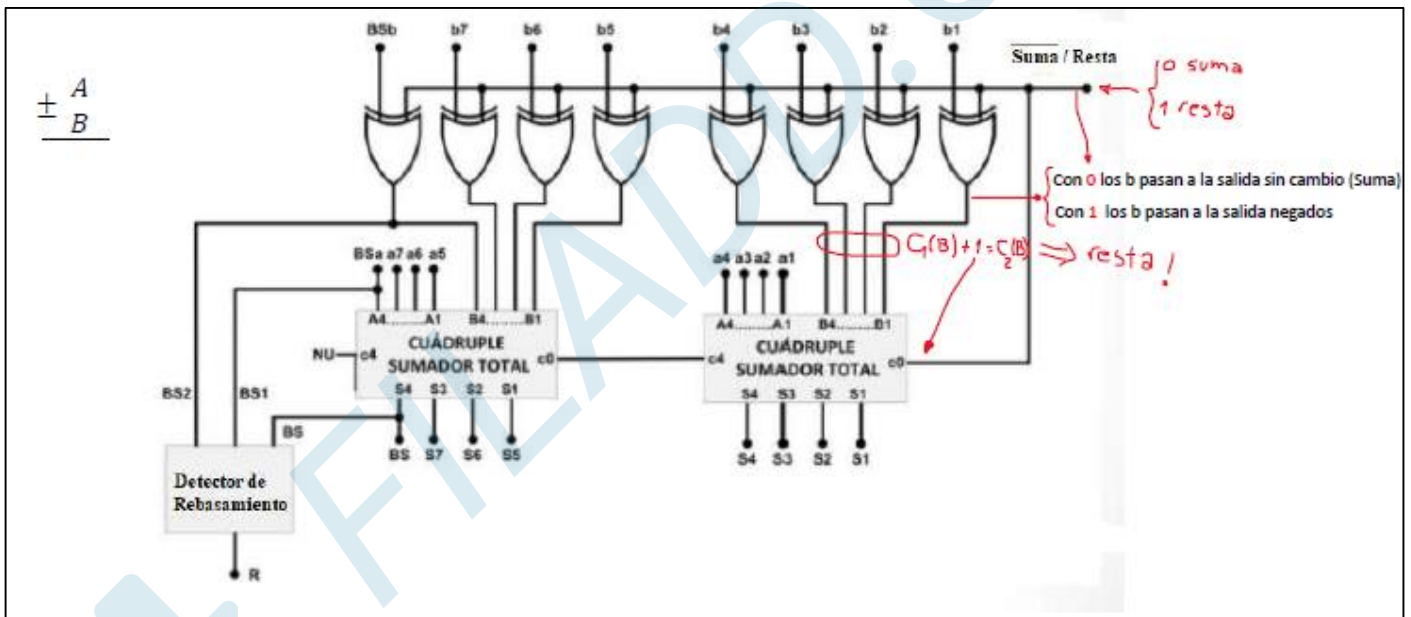
Reescribiéndose así:

$$= c_{i-1} (a_i \oplus b_i) + a_i b_i$$

Como nosotros ya teníamos de antes la OR exclusiva entre  $a_i$  y  $b_i$ , la vamos a reutilizar de forma parcial para así poder crear el circuito de  $C_i$ , ahorrando con eso una compuerta, aumentando la velocidad.



### Sumador/Restador de 8 bits en complemento a dos

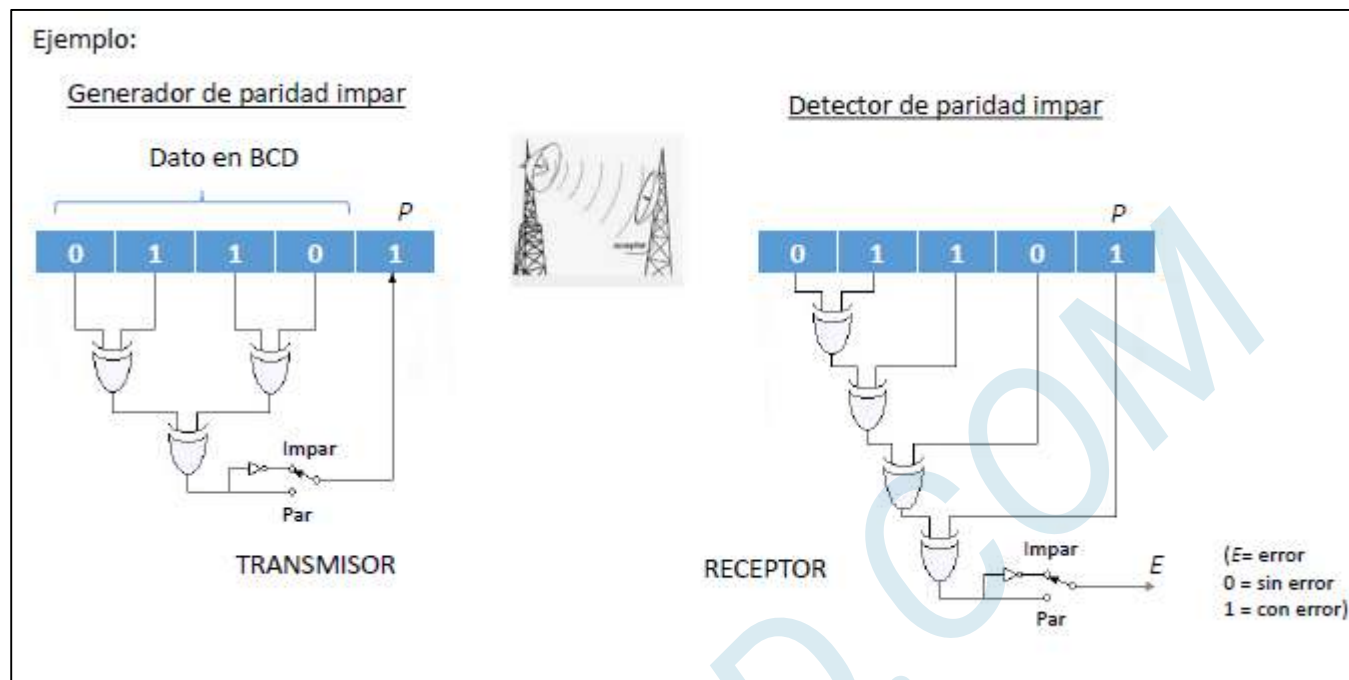


En este caso usamos dos cuádruples de comienzo, con un 0 en la entrada se transforma en un sumador y con 1 se transforma en un restador. Las  $b$  son datos que están entrando y el dato  $a$  entra mediante el cuádruple con el bit de signo, el bit de signo se procesa todo igual a cualquier bit de la resta.

Pone una OR exclusiva porque es un inversor inteligente, entonces el bit que se mete junto a  $b$  en la OR exclusiva es el bit de control, cuando ponías 0 hace que pase por la OR exclusiva los bits de las entradas  $b$ , pero cuando vos pones un 1 entonces los bits de control cambian los 0 por 1 de  $b$ , y entonces vas a sumar a el complemento a-1 de  $b$ , luego le sumas un 1 que viene del carry (de donde nosotros poníamos el 1 o 0 para seleccionar entre suma o resta) y lo convertís en complemento a-2 al número  $b$ , entonces al sumar esto con  $a$  obtenemos la resta entre ellos, porque sumar a más el complemento de  $b$  es lo mismo a efectuar la resta entre  $a$  y  $b$ .

## Generador y detector de paridad

Este circuito crea automáticamente la paridad par o impar según la cantidad de unos del dato a transmitir.



En el dibujo de la izquierda (Generador de paridad) tomamos los datos de entrada que van a ser variables, no importa cuales, que van ser los que vos en BCD quieres transmitir, en este caso 6, la pregunta es cuál es el bit que se tiene que crear. Si queremos paridad impar el circuito va a crear haciendo las cuentas siguientes: si los valores que están conectados a la OR exclusiva son distintos te va a largar un 1, un 1 con un 1 son iguales, si son iguales te mando un 0 a la salida, por lo tanto lo negué al resultado porque quiero paridad impar y entonces ese uno es el que se grabó ahí en la P. Si hubiera querido paridad par solo debía correr el switch para este otro lado y tenía paridad par, el switch es el grupo al final que invierte o no invierte, según tus necesidades, moviendo la llave entre la no inversión y la inversión.

Esta información se trasmite y viaja al dibujo de la derecha, pudiéndose meter ruido y cambiando algún bit.

El detector de paridad es otra cadena de OR exclusiva que hacen todas las cuentas incluyendo a P, para terminar dándote un bit de salida que te dice que E es el bit de error, si te da 0 quiere decir que es sin error y si te da uno quiere decir que tiene un error, en este caso el receptor debería recibir paridad impar. En este caso el número recibido parece que está bien porque tiene 3 unos en total, siendo esto impar.

En la primera tanda, si son distintos te da un uno, en la segunda como son iguales arroja un cero, volviendo a ser iguales en la siguiente OR exclusiva por lo tanto te da un 0 y como son distintos con el bit de paridad la última OR exclusiva arroja un uno y un uno pasado por el inversor da un 0, por tanto se lo toma como que llegó bien al número.

## Sistemas secuenciales

Las variables de entrada aparecen una a continuación de la otra, la salida no depende nomas de las entradas sin importar el tiempo, en este sistema depende del estado de las entradas pero también de los estados internos que se encuentran almacenados en la memoria de la máquina. La máquina se fija en lo que está entrando, pero también lo que había entrado en el tiempo pasado, que está guardando en la memoria.

Cuando se procesa información, hay gran cantidad de acciones que se realizan en forma secuencial, es decir una a continuación de la otra.

Estas acciones que se suceden en el tiempo dependen de un conjunto de condiciones previamente establecidas.

Por lo tanto, los sistemas combinacionales no nos permiten por si solos resolver el problema de almacenar el estado de las entradas en un instante y utilizarlo para tomar decisiones posteriormente cuando cambie el estado de aquellas.

Los sistemas que puedan actuar de la forma anteriormente descrita han de ser capaces de memoriza el estados de las entradas y convertirlo en un estado interno del propio sistema.

### **Definición de sistemas secuenciales**

Un sistema secuencial reacciona ante secuencias de estados de entrada de una forma determinada que depende de su construcción física y por tanto es un sistema automática, que recibe el nombre de autómeta finito debido a poseer un número finito de estados internos.

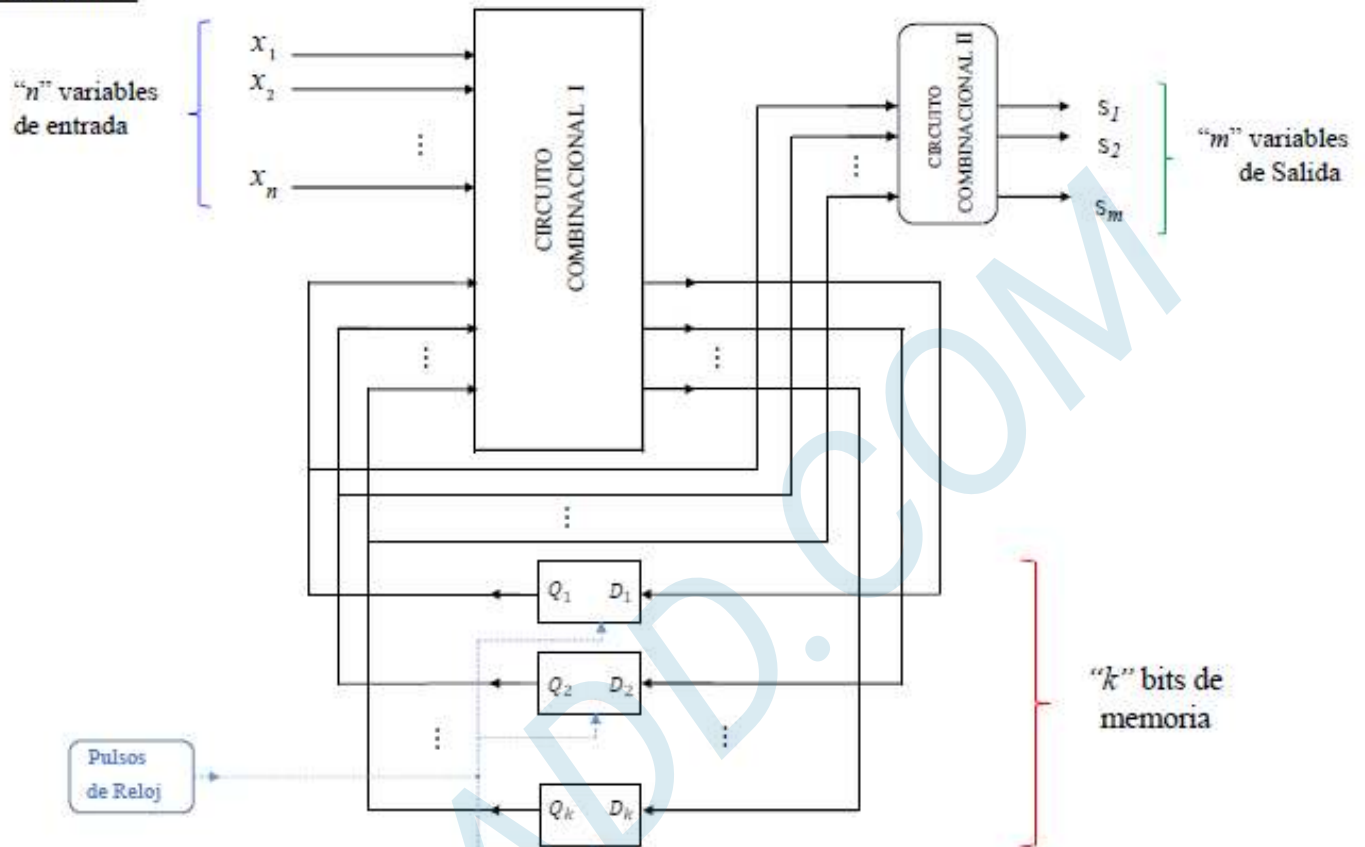
La denominación de secuencial es debida a que el valor de su salida en un instante determinado no depende solamente de los estado s lógicos de las entradas en dicho instantes sino de la secuencia de estados de las entradas.

Los estados lógicos de las entradas se almacenan en la memoria del sistema, por tanto es necesario en los sistemas secuenciales tener en cuenta la noción del tiempo.

Todo sistema secuencial posee:

- 1) Un conjunto finito de  $2^n$  estados de entrada siendo  $n$  el número de variables de entrada  $x_1, x_2, \dots, x_n$ , son todas las combinaciones que uno tiene, siendo  $n$  el número de variables de entradas
- 2) Tenes un conjunto finito de  $2^k$  de estados internos siendo  $k$  el número de bits de memoria  $Q_1, Q_2, Q_k$ ,  $2$  elevado a la  $k$  es la cantidad de cosas que te podes acordar, porque vas haciendo una combinación binaria de todos estos estados.
- 3) Tenes un conjunto finito de  $2^m$  estados de salida siendo  $m$  el número de variables de salida  $S_1, S_2, \dots, S_m$ .

### Autómata de Moore:



Tenés las entradas que son las  $x$ , las salidas que son las  $s$ , y acá adentro tenemos la memoria, que son esos casilleros que tiene una  $Q$  y una  $D$ , donde cada casillero permite recordar 1 bit, entonces tenemos  $k$  memorias, vos te podés acordar  $2^k$  con esas memorias.

Los pulsos de reloj llegan a la máquina para decirle cuando la memoria va a cambiarse, le dice cuando toma la decisión, mientras más rápido ande el reloj más rápido será el circuito.

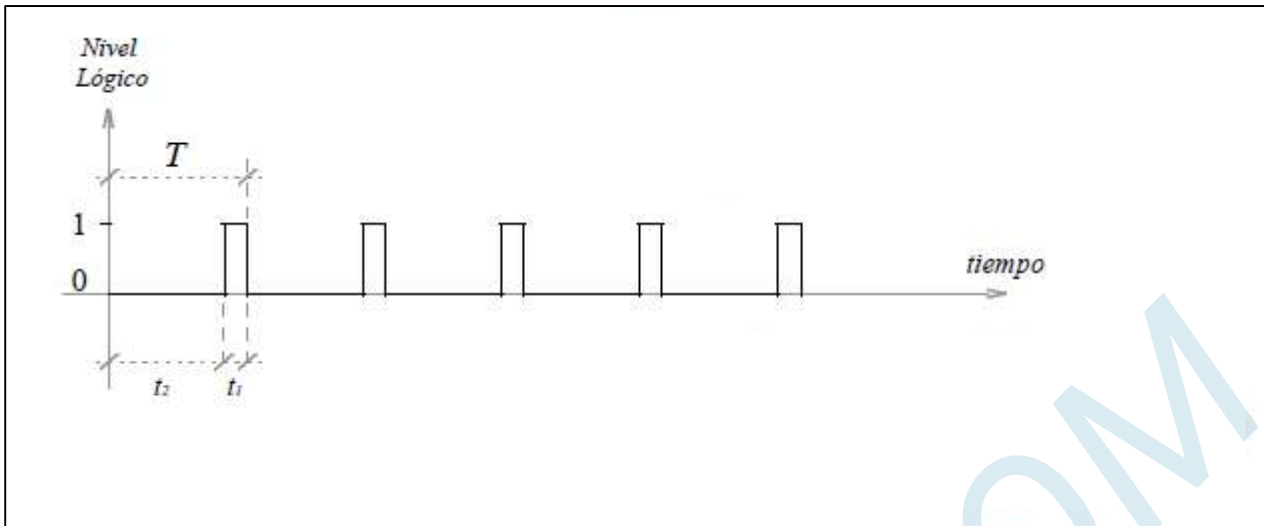
Arriba tenés circuitos combinacionales que ya habíamos vistos.

Las variables de entrada van a ir apareciendo de a una, y se van a hacer las cuentas en función de lo que antes estaba guardado en la memoria, que sale por  $Q$ , y esos son los valores que junto con las entradas se recalculan para poder volver a guardar en la memoria a través de estos cables de nuevo a la memoria con los nuevos valores. Hay datos en la memoria viejos que desaparecen para guardar datos nuevos, porque no son necesarios.

### El reloj

La sincronización de los circuitos secuenciales se logra mediante un dispositivo de tiempo llamado generador de pulso de reloj (clock) que produce un tren periódico de pulsos de reloj a la salida de ese circuito para dárselo a la máquina.

Los pulsos de reloj se distribuyen a través del sistema de manera tal que los elementos de almacenamiento (memorias) se afectan solo con la llegada del pulso de sincronización.



El reloj es parte vital de la maquina porque la va diciendo la cadencia de cómo va funcionando, como van pasando el tiempo y cuando tiene que hacer los cambios. La memoria hace los cambios justamente con el pulso de reloj.

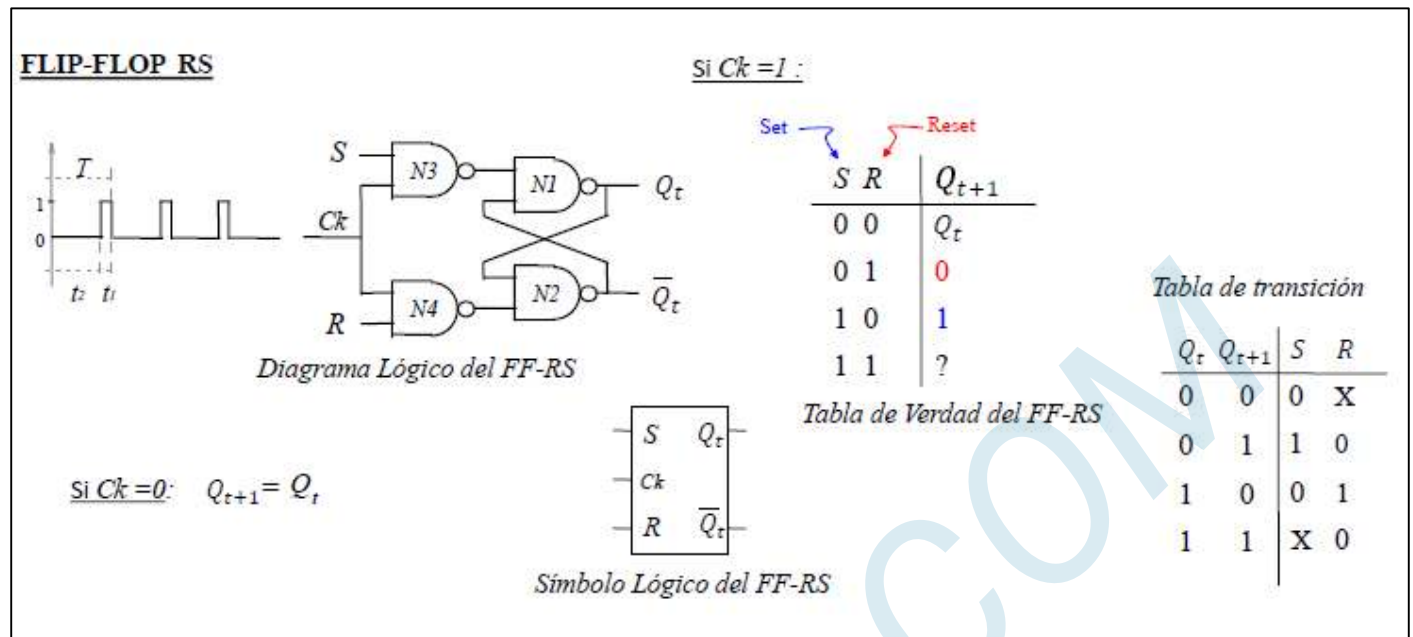
Analizando el dibujo el reloj tiene un nivel lógico que puede ser un 0 o un 1, entonces el reloj esta en cero y pasa en uno, si vos lo vieras como una lamparita y te la imaginas, ella se encendería y luego se apagaría, regularmente. Pasa de 0 a 1 y de 1 a 0, lo hace a espacios de tiempo regulares, la reciproca de este tiempo es la frecuencia, que es la velocidad con la que se repiten los pulsos, es una onda periódica. Si el cambio entre estados lógicos se hace en periodos de un segundo se diría que ese reloj trabaja a 1 Hertz, si este pulso se hace mil veces en un segundo estarías en un kilo Hertz, si vos haces en un segundo un millón de cambios, tenes un Mega Hertz, las maquinas actuales les agregamos tres ceros más, teniendo en un segundo mil millones de cambios, teniendo un Giga Hertz.

El tiempo que tarda ( $t$ ) si haces la reciproca porque entre periodo y frecuencia se relacionan la inversa uno del otro, entonces el tiempo sería de un nanosegundo, es decir, es  $1 \times 10^{-9}$  de segundo, eso sería el tiempo que tarda en hacer un ciclo correspondiente a una frecuencia de un Giga Hertz actual.

### Elementos de memoria

Los elementos de almacenamientos o memorias que se emplean en circuitos secuenciales controlados por reloj reciben el nombre de flip-flops o multivibradores biestables, los cuales son dispositivos de almacenamiento binario que pueden contener un bit de información. Son 4

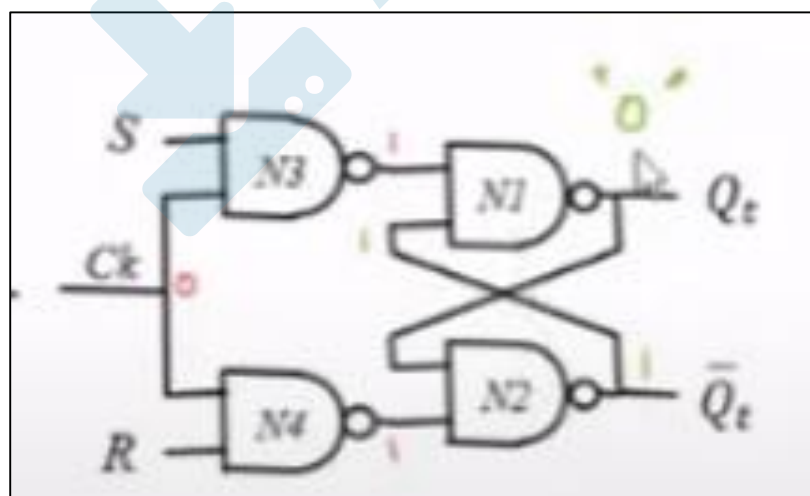
## FLIP-FLOP RS



El truco para que estas memorias se acuerden del valor, estuvo en cruzar los cables (esa x que se forma), esas líneas de retroalimentación que toma información de la salida para ingresarlo en la entrada, son el secreto de por qué el circuito puede recordar en su interior. Las compuertas del inicio (Que tienen como entradas R y S) están multiplicadas por el reloj, mientras el reloj diga 0 ni S ni R son los comandos de la memoria podrían entrar al sistema, solamente se permitirá que S y R ingresen o hagan cuentas cuando el reloj valga 1, pero el reloj vale 1 en pequeños instantes de tiempo, quiere decir que el reloj es el jefe, dice cuando los comandos de cierre puede ingresar al interior de las células de memoria. S y R son los comandos que se utilizan para grabar el valor deseado del bit dentro de la máquina, son los comandos para decirle a la memoria, escribí un 0, escribí un 1 o quedate como estás.

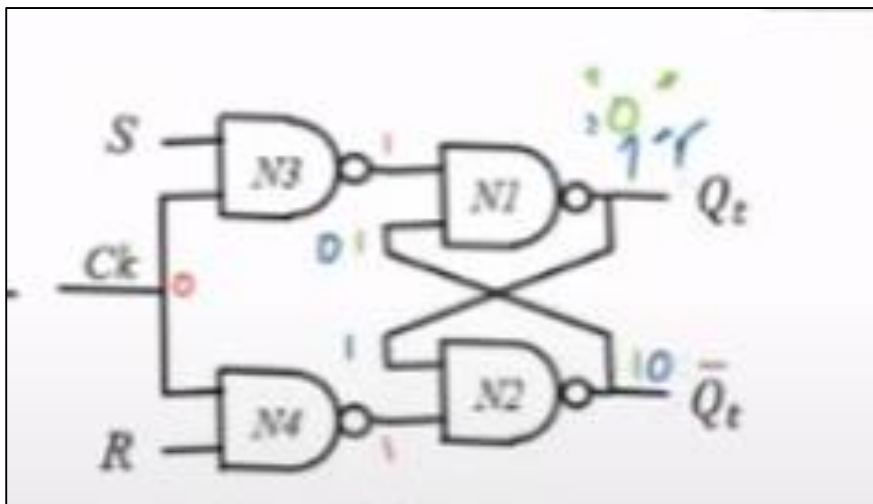
La salida tiene el valor Q y el Q negado abajo, los Q son las salidas por donde se va a leer el estado del bit que está guardado ahí dentro de la memoria.

El reloj se la pasa en 0, porque está más tiempo en 0 en 1, si está en 0 la memoria no hace cambios y se queda recordando el valor que tenía antes, mientras tanto S y R se podrán mover, total da igual porque van a multiplicarse por 0 que es el valor del reloj en este momento.



Si el reloj vale 0, como la compuertas que están primero multiplica y el reloj vale 0 va a dar 0 independientemente de S y R, pero la NAND niega el resultado por tanto el resultado final de esa primera compuerta vale 1. Pero ahora no puedo seguir haciendo las cuentas porque necesito saber lo que tienen las otras líneas que dependen de la salida de la otra, entonces voy a suponer un valor, primero supongamos que es 0 el valor de la salida (el que está recordando), el 0 bajaría y te dejaría hacer la multiplicación,  $0 \times 1$  el resultado es 0, por lo tanto la segunda

NAND de abajo negaría el resultado y tendríamos un 1 en la segunda compuerta, ahora recién el 1 lo puedo subir multiplicando 1 por el 1 que ya teníamos de antes al haber multiplicado S por el reloj, obteniendo en la NAND un resultado 0 porque niega la variable, por tanto cuando el reloj vale 0 y el valor que teníamos también vale 0, el resultado final de salida en esa situación es 0.



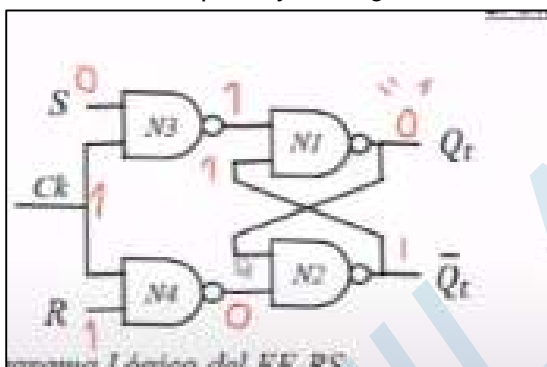
Ahora bien, si el valor que hubiese tenido antes el flip-flop hubiese sido un 1, si hacemos el mismo razonamiento de antes obtendremos en la salida un 1 en vez del cero que obtuvimos suponiendo que el valor anterior era 0

El nuevo camino por si lo quieren seguir es el que está en azul

Entonces como conclusión si el reloj vale 0 conserva el valor que no sé quién había guardado en la memoria antes. Mientras el reloj esté en 0 la salida un tiempo después ( $Q_{t+1}$ ) va a mantener el valor anterior.

¿Qué pasa cuando el reloj valga 1? Ahí si van a funcionar los comandos S y R, teniendo entonces cuatro posibilidades para los comandos: 0 0; 0 1; 1 0; 1 1.

En el caso de que S y R tengan 0 0, tendré 0 x 1 quedará 0 pero negado un 1, queda exactamente igual a cuando teníamos al reloj que valía 0 por tanto no hace falta demostrar nada y asumimos que cuando S y R valen 0 0, se mantiene el valor que estaba guardado en el tiempo anterior.



Si S vale 0 y R vale 1 la salida va a ser 0 (pueden intentar hacer la demostración ustedes mismos siguiendo el circuito). Cuando vos aplicas reset la salida se pone en 0 sin tener en cuenta lo que había antes.

Cuando S vale 1 y R vale 0 la salida va a ser 1 (intenten razonarlo y cuando lo hagan usen la compuerta de arriba que tiene el 0 haciendo que lo que venga de abajo no importe).

Si vos pones que S vale 1 y R vale 1 ustedes verán que la salida  $Q_t$  es 1 y la salida de abajo  $Q_t$  negado, quedan ambas 1, en vez de tener valores opuestos puesto que son complementarios, es una incoherencia, por eso esa última condición no se lo toma como válido, porque la memoria dice cosas ilógicas para la memoria 1 y 1, es como pedirle que prenda y apague al mismo tiempo.

### EN RESUMEN HAY 3 COMANDOS:

0 0 RECORDÁ

0 1 RESETEA (PASALO A 0)

1 0 SETEA (PASALO A 1)

### Tabla de Transición

Si la memoria está en 0 y vos quieres que un tiempo después se quede en 0, pones 0 0, pero también podes usar 0 1 por tanto en la tabla de transición pones 0 X porque ambas condiciones pueden ser.

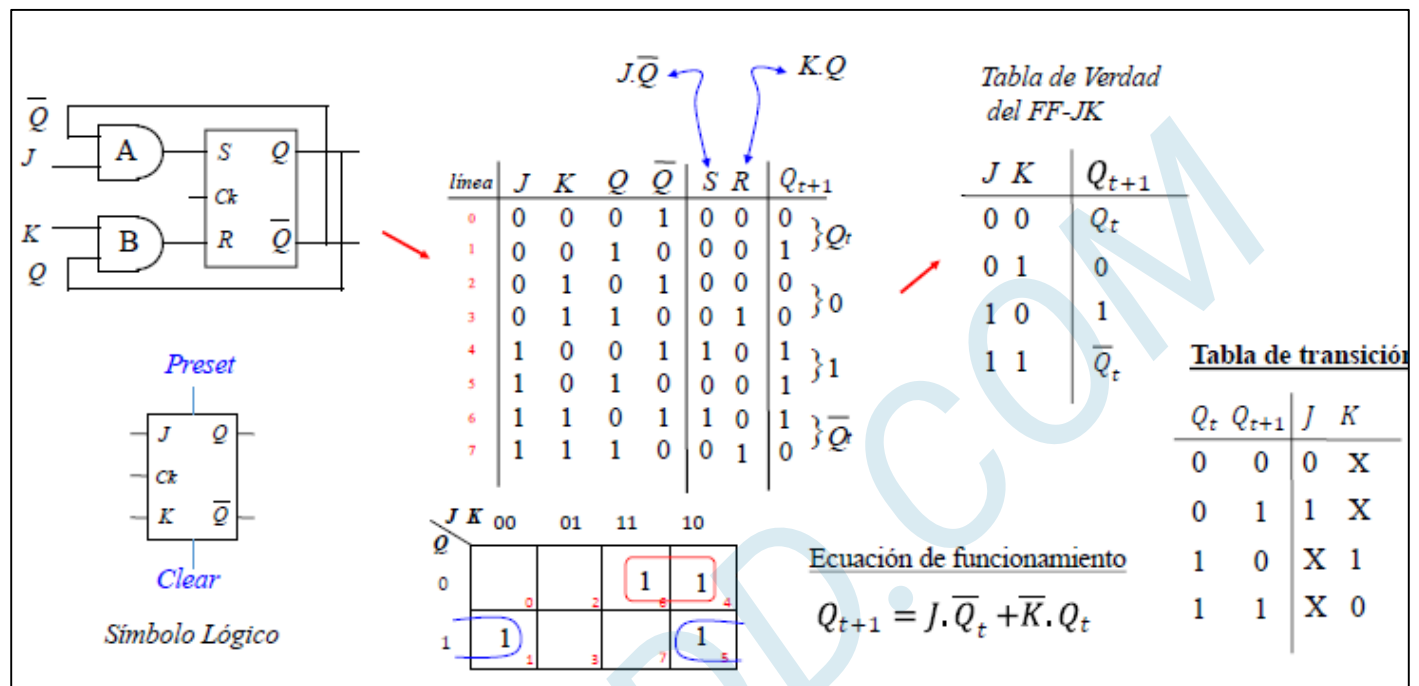
Si quisieras ir de 0 a 1 tenes que accionar pones 1 0

Si quisieras ir de 1 a 0 pones 0 1, no hay otra chance

Si tenes un 1 quisieras mantenerlo podes poner 1 0, y también podes poner 0 0, por tanto usas la X porque da igual que pongas en S y te queda X 0

## Flip Flop JK

Debido a la indeterminación que se produce en el flip flop SR cuando la entrada aparece la combinación 1 1 se ha ideado otro tipo de flip flop que levanta esa indeterminación que se llama JK, debido a su creador Jack Kilby.



Jack propone que agarremos un SR que todos conocemos como anda y le clavamos dos compuertas de entrada AND y las reconectamos de nuevo, estableciendo una retroalimentación (tomar de la salida para reingresar en la entrada).

En el dibujo del circuito la puerta A está multiplicando a J con Q negado que es la información que toma de la salida de la memoria y la puerta B está multiplicando a K por Q que también es información que toma de la salida de la memoria. Entonces con el resultado se lo mete adentro de la memoria, que con 0 0 mantiene con 0 1 se receta con 1 0 se prende (Setea).

Primero hacemos la tabla de verdad grande que tiene las entradas J K Q y Q negado. Luego ponemos una línea intermedia como un cálculo auxiliar intermedio que son las salidas de las compuertas que ingresan al flip flop, por eso pone la columna S y R y finalmente me pregunto a donde va a ir a parar la salida con todas estas combinaciones de entrada.

Esta tabla no tiene 16 valores, a pesar de tener 4 letras, pero no es así, porque tiene a Q y Q negado, por tanto no pueden valer lo mismo al mismo tiempo. Por tanto la primera línea no es 0 0 0 0, sino que es 0 0 para J y K y acá la salida (Q) podría estar en 0, por tanto la otra tiene que estar en 1 ya que es su complemento negado.

La segunda línea de nuevo poner 0 0 en J K y las salidas poner 1 0. O sea que con J y K en 0 0 podes tener 0 1 y 1 0 como valores de las salidas.

La siguiente combinación es 0 1 para J K con el valor de 0 1 o con el valor de 1 0 (se ve en la fila siguiente ya que esos son los dos valores que pueden tener la salida).

El siguiente valor será entonces 1 0 para J K y devuelta repetiremos lo anterior de que puedan haber salidas tanto 0 1 como 1 0. Y así se arma sucesivamente.

Para determinar los valores de S R tenemos que tener en cuenta que S son los valores de la salida al hacer la multiplicación J por Q negado, entonces tengo que buscar a J y Q negado en la tabla y hacer la

multiplicación (multiplicar los extremos) entonces en la primera línea de las entradas S R, en S colocaremos un 0 ya que  $J = 0$  multiplicado por Q negado = 1, dando eso un resultado de 0.

Decimos que para poner lo que va a ir en R en la tabla de verdad tenemos que hacer K multiplicado por Q, entonces como en la primera fila  $K = 0$  y  $Q = 0$  entonces pondremos que R es 0. Y así iremos completando sucesivamente la tabla.

Para la línea de la salida ( $Q_t + 1$ ) yo tengo que seguir mirando el dibujo del circuito, le tengo que preguntar al flip flop S R qué decisión va a tomar, qué valor va a poner en la salida en función de los valores S y R que pongo en la entrada.

En la primera línea S y R vale 0, por tanto yo recuerdo que cuando tenían esos valores la salida no cambia, entonces agarro y me fijo para atrás y veo que tiene Q, que tiene un 0, entonces si no va cambiar en la primera línea de la salida voy a colocar 0.

En la segunda línea S y R tienen 0 quiere decir que no cambia, entonces antes tenía un 1 en Q entonces, ese 1 se va a mantener en la salida.

En la tercera línea (la numero 2) le ponemos un 0 porque 0 0 no cambia, antes tenía un 0 en Q por ende se queda con ese valor que tiene Q.

En la cuarta línea, si S vale 0 y R vale 1 quiere decir que la salida ( $Q_t + 1$ ) va a tener un valor de 0 porque se resetea, no se fija en lo que había antes. Vale 0 porque R lo dijo

En la próxima va un 1, porque se S tiene 1, por tanto setea y se coloca un 1. Y así se arma el resto de esta columna.

Entonces habiendo hecho la tabla grande, ahora la podemos resumir en la tabla pequeña (Tabla De verdad del FF-JK), ponemos las dos entradas J K, la salida  $Q_t + 1$  y las posibles cuatro combinaciones: 0 0; 0 1; 1 0; 1 1.

J K en la tabla grande 0 0 son las dos primeras líneas, nos fijamos en la salidas que hay, que tienen un 0 y un 1, que es lo que tenía antes, por eso se lo llama  $Q_t$ , o sea que mantiene el dato que le habían grabado en el anterior.

Si te fijas en las otras dos líneas que es cuando J K valen 0 1, puedes observar que la salida vale 0 por tanto en la tabla de verdad resumida ponemos que cuando J K valen 0 1 la salida es 0.

Cuando J K valen 1 0 entonces en la salida ponemos 1, porque en las dos líneas donde J K valen 1 0 en la tabla grande tienen ambas líneas en la salida el valor 1.

En el ultimo se coloca Q negado, porque cuando tenemos 1 1 yo acá veo que esta el 1 y el 0, y no puedo poner eso, por tanto me fijo, antes en Q había un 0 y en la salida hay un 1 y en la línea siguientes antes en Q había un 1 y en la salida hay un 0 por tanto podemos ver que son opuestos los valores por tanto decimos que es el complemento de Q el valor de salida cuando J K valen 1 1.

En el símbolo lógico no hay nada extraño, entradas J K y salidas Q y Q negado pero se le agregan dos nuevos elementos que son el Preset y el Clear, que ambas son señales que sirven para prender la salida en el caso que pongas un 1 lógico en preset en cualquier momento y en clear sirve para apagar la salida en cualquier momento, digo en cualquier momento porque no necesitas hacer las combinaciones y esperar al pulso de reloj para que realice el cambio. Lo hace de manera asíncrona, sin depender del pulso del reloj.

Preset setea a Q en 1 y clear lo borra y pone 0.

## Tabla de transición

Si la salida la tenes en 0 y quieres que en el siguiente pulso se mantenga en 0 a J K le tenes que colocar un 0 X, ya que podemos poner 0 0 para mantener y 0 1 porque de esa forma reseteamos y queda 0 lo mismo.

Si de 0 quieres ir a 1 tenes que poner 1 X porque podes poner 1 0 para setear y el 1 1 porque 1 1 te hubiera cambiado de un 0 a un 1 porque 1 1 pone el valor contrario.

El siguiente para pasar de 1 a 0 tenes que poner X 1, porque podes poner 0 1 para resetear, y también 1 1 para que adopte el valor contrario.

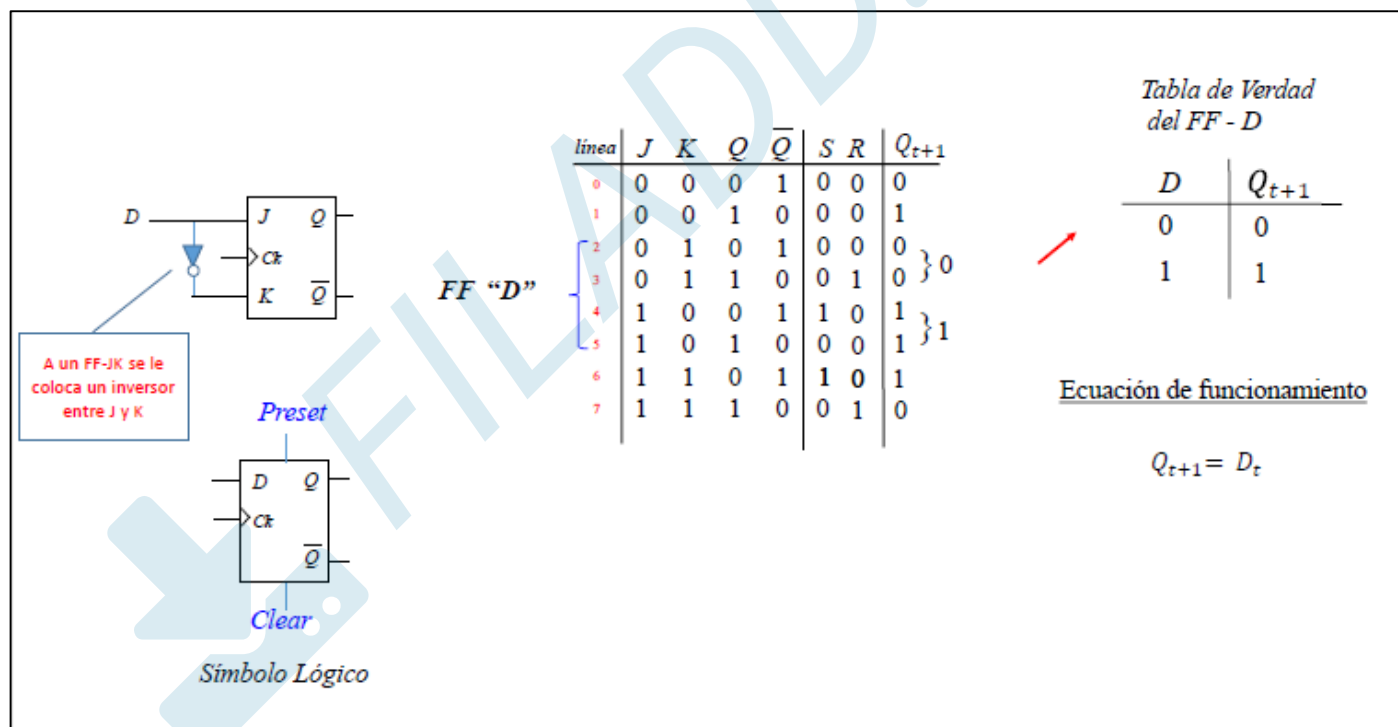
La última opción para mantener el 1 se pone X 0 porque podes usar el 1 0 para setear y el 0 0 para mantener, así que se coloca X 0 porque en la X podría ir tanto 0 como 1.

Una vez que tenes el circuito físico, necesito una ecuación porque me gustaría implementarlo por software.

Para hacer la función tengo que escribir  $Q_{t+1}$  que este en función de las entradas J K y de Q, por tanto hago un Karnaugh, no tomamos en cuenta a Q negado porque depende de Q. Voy a pasar al mapa los 1 que se encuentran en la línea 1, 4, 5, 6, y realizo las agrupaciones y luego escribo la ecuación que representa al circuito físico.

## Flip Flop D (Delay, Data)

Se lo hace tomando un J K, y poniendo la entrada D en J y colocando un inversor entre J y K restringiendo entonces a valer entonces el valor contrario entre J y K.

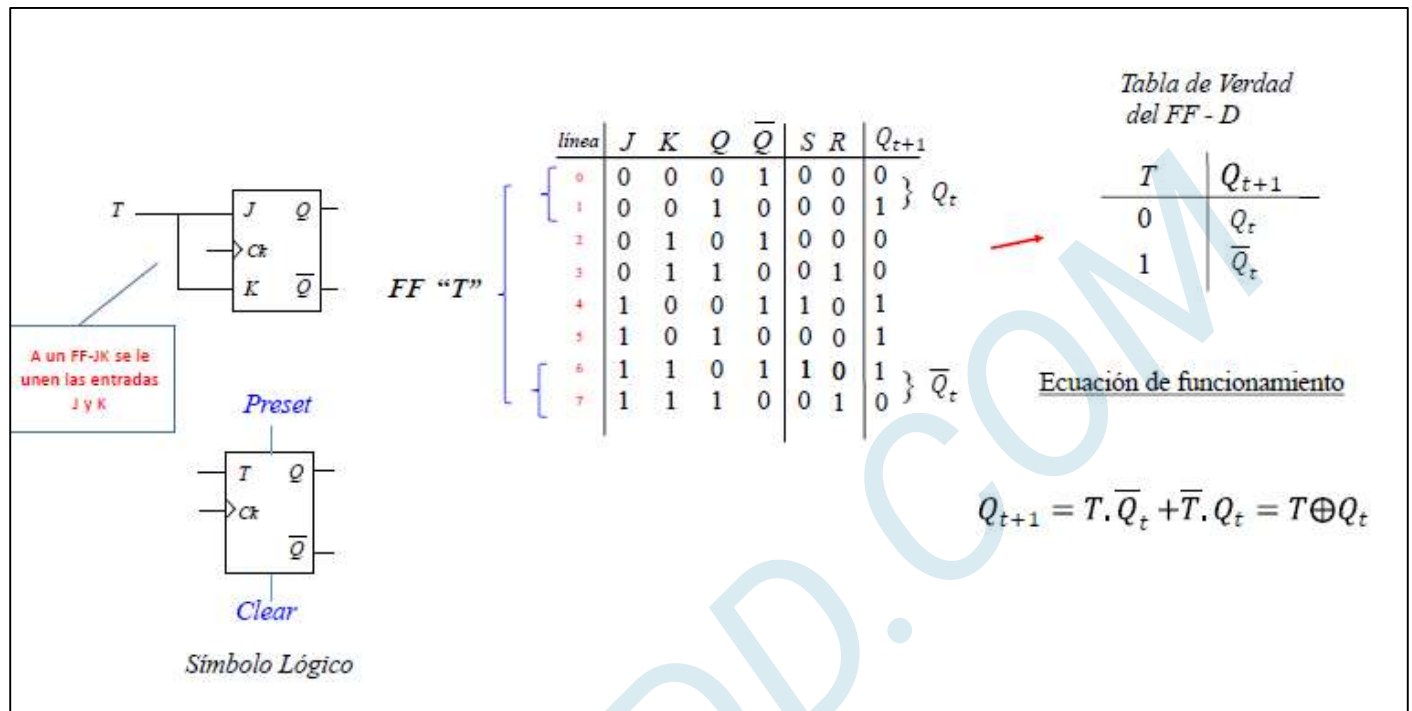


Repetimos la tabla del J K y al poner el inversor solo restrinjo el funcionamiento en las líneas 2 3 4 5, de la tabla grande usamos solo esas que son justamente los lugares donde J K tienen valor contrario.

Nos damos cuenta que la entrada D es J, si J tiene 0 la salida 0, y si J tiene 1 la salida tiene 1, entonces la tabla de verdad es bien sencilla, porque con esta restricción el flip flop es tan fácil como si le pones un 0 a la entrada D le das un pulso de reloj por aquí la salida se te va a grabar en 0 y si le quieres poner un 1 entonces a Q metes en D un 1 le das un pulso y el 1 queda atrapado en la salida y la ecuación de funcionamiento es tan sencilla como que la salida  $Q_{t+1}$  va a adquirir el valor que pongas en D en la entrada.

## Flip Flop T (Toggle) (conmutación)

Se lo forma con un J K que le has unido las entradas J y K con un alambre conductor y a esa nueva entrada le llamas T.



Entonces para el flip flop T usamos de la tabla las líneas donde J y K tienen el mismo valor. Cuando T vale 0, serían las primeras dos líneas, la salida vale  $Q_t$ , si vale cero no cambia. Si T vale 1 toma siempre el valor contrario del estado anterior. Así que la gente utiliza el flip flop t mandándole un 1 a la entrada así cada vez que le tira un pulso prende y apaga o hace siempre lo contrario de lo que tenía antes.

La tabla de verdad es tan sencilla como que a si a T le mandas un 0 pueden venir los relojes que quieran, la salida se queda inmóvil, queda lo que tenía antes, pero cuando a T le pongas un 1 entonces la salida va a empezar a cambiar, de 0 pasa a 1 de 1 a pasa a 0 y así.

## Contadores binarios

Un contador es un circuito digital capaz de contar sucesos electrónicos tales como impulsos, avanzando a través de una secuencia de estados binarios. Un contador entonces cuenta pulsos, o pulsos que provienen de un reloj o de algún contador de eventos

Se dice que el contador binario de "n" bits es un registro de n flip flops y compuertas asociadas que siguen una secuencia de estados determinada en cada transición de los impulsos del reloj. Es un sistema secuencial formado por flip flop memorias y algunas compuertas pueden tener y cuenta pulsos de reloj, se dice tienen "n" bits cuando vos tenes "n" flip flop.

La cuenta binaria o módulo de contador es la cantidad máxima de estados que puede tener la secuencia contada. Generalmente, se determina por medio de  $2^n$  donde n es el número de los flip flops. Supongamos que vos tenes 2 flip flop entonces  $2^2 = 4$  cuentas diferentes, entonces ese contador podría contar por ejemplo 0 1 2 3, pónale que sea cíclico (0 1 2 3 0 1 2 3...) ese sería un contador de módulo 4 porque la cantidad de cuentas máxima son cuatro. Se calcula como dijimos haciendo  $2^n$ , entonces ustedes vieron que la cuenta es 4 pero como parto desde el 0 nomás llegue hasta el 3, entonces se dice que por otra parte, la cantidad máxima contada se determina por  $2^n - 1$ .

La salida del contador es el conjunto de salidas (Q) de los flip flops que forman el contador. Entonces si vos tenes un contador que tiene adentro n flip flops haces una línea y cada una de las salidas del flip flop

constituyen la salida del contador. La entrada del contador es el reloj de todos los flip flops que forma parte de la cuenta de los impulsos.

Pueden ser cíclicos si se repiten periódicamente su cuenta o se detiene solo, cíclico por ejemplo es 0 1 2 3 0 1 2 3.... Y se va repitiendo la cuenta, y si no llega hasta el tope y ahí se queda, es decir: 0 1 2 3 y se terminó. El contador de productos de una maquina por ejemplo le podes pedir 1000 piezas y llegarás hasta ahí. Cuenta ascendente o descendente o según una secuencia determinada.

Pueden ser operaciones síncronas o asíncronas. Un contador síncrono es un tipo de contador en el que todas las etapas o flip flops utilizan el mismo impulso del reloj. El clock de los flip flops están unidos juntos y reciben el impulso al mismo tiempo y se dice entonces que es un contador síncrono. En cambio el contador asíncrono es el cual donde el reloj no afecta a todos los flip flops y unos conmutan tras otros, es decir, el impulso en el asíncrono solo un flip flop va a recibir el impulso del reloj, luego como están enganchados en cascada, entonces uno entrega el pulso del reloj al siguiente y así sucesivamente se lo van pasando.

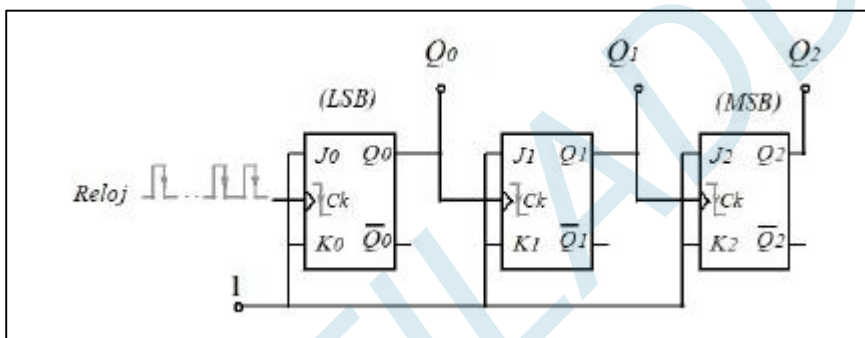
Ejemplo: diseñar un contador ascendente, cíclico, de módulo 8 asíncrono:

Ascendente: cuenta del 0 en adelante

Cíclico es que se repite

Módulo 8 es que son 8 cuentas y según la formula entonces 7 será la cuenta máxima (del 0 al 7)

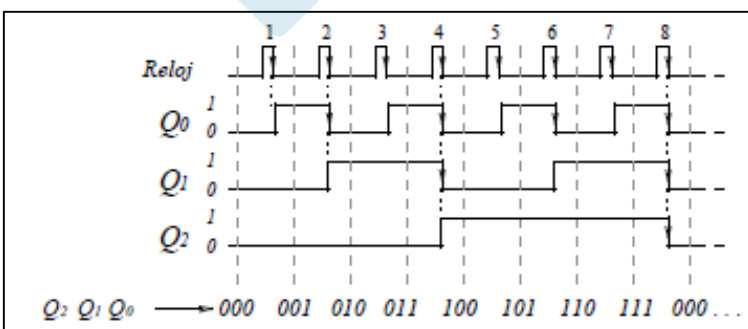
En un contador asincrónico la transición de salida de un flip flop sirve como fuente de activación de otros flip flops.



Acá se ha puesto tres flip flops y se lo han conectado de la siguiente manera: el reloj solamente ataca al primero, fíjense que la salida del primero acciona al segundo, la salida del segundo ingresara al clock del tercero. Las salida del contador son los Q, que son todas y cada una de las salidas de los flip flops. En este caso están ordenados al revés, el 0 va al

último por ser menos significativo, así que recuerden que el q2 iría primero y q0 al último, por ser el menos importante. Se le llama LSB (Less Significant Bit) o séase (Bit menos significativo) al que está almacenado en la primera memoria conectada al reloj en nuestro dibujo y MSB (Most Significant Bit) o séase (Bit más significativo) al que está almacenado en la última memoria en nuestro dibujo. Si vos queres hacer contadores más grandes básicamente tenes que copiar y pegar sucesivamente.

Fíjense que por otro lado todas las entradas están conectadas a un 1 lógico, o sea se puso en 1 haciendo la función Toggle, como el flip flop, que va a ir cambiando cada vez que recibe un pulso la salida si esta en 0 pasa a 1 y si esta en 1 pasa a 0.



Esa es una serie de pulsos de reloj que van a ir llegando al contador (la que dice reloj) tiene 8 pulsos para ver una vuelta completa del contador.

Después tiene dibuja todas las salidas del contador (Q0 Q1 Q2) en diagramas temporales. Voy a ir viendo que mientras el tiempo transcurre puedo ver que la salida va pasando de 1 a 0 de 0 a 1 y así sucesivamente.

En el primer momento todas las salidas Q valen 0, vamos a suponer que todos los flip flops se accionan con el flanco decreciente de la señal de reloj, cuando el flanco cae los flip flops realizan los cambios, no hay mucho que pensar porque los cambios que van a hacer es cambiar las salidas pasar de 0 a 1 y de 1 a 0 porque las entradas están congeladas a 1 todas.

Cuando viene el primer pulso del reloj, el único que cambia el primero porque de 0 va a pasar a 1, mientras los otros dos flip flops se van a quedar en 0 porque es un reloj que no se mueve, al menos no hay caída de los flancos. Cuando el primer pulso del reloj cayo, el Q0 sube de estado, al subir no le provoca ningún cambio al siguiente, porque el siguiente reacciona con el flanco de caída del primero.

Cuando viene el segundo pulso del reloj, cuando cae el flanco del reloj, cae también el flanco de Q0, haciendo que tras esto le haga un disparo al segundo flip flop (Q1), haciendo que el mismo suba su estado lógico, pasando de 0 a 1. Obviamente que levantar la salida del segundo no le provoca cambio a la salida del tercero (Q2) porque solo provocan cambio cuando cae el flanco, por eso el tercero sigue en 0.

Cuando viene el pulso el 3 vuelve a levantarse el primero, pero no le ocasiona ningún cambio al segundo debido a que eso ocurre solo cuando cae el flanco del primero.

En el pulso número 4, cuando cae el mismo hace que cambie (Q0) cayendo, y eso provoca cambios en el segundo (Q1) y el segundo también cae y esto provoca por primera vez que cambie el tercero (Q2).

Si se sigue analizando se vuelve a repetir este patrón en los pulsos subsiguientes.

Finalmente en el curso numero 8 todos caen, haciendo que provoquen cambios y la cuenta se vuelve a cero, y si vuelve a cero se vuelve a repetir todo este proceso nuevamente.

Un contador se lo usa en un procesador para que lleve un conteo de las instrucciones por ejemplo.

La ventaja de este contador que acabamos de ver es que está muy bueno, es sencillo y siempre hace lo mismo. Si quieres hacer un contador más grande copia y pega y tenes un contador gigante.

El problema se da en pulsos como el ultimo, cuando el reloj vino y cayo, se hace una reacción en cadena donde caen todos, evidentemente, esto no se hace en tiempo real por más rápido que sea el circuito, porque entre que cambia el primero y el ultimo pasa un tiempo y el fabricante tiene que considerar que ese tiempo va a ser considerado como el máximo, que es la suma de todos los tiempos, es como un efecto domino, todos van cayendo uno tras otro, mientras más fichas tenes más tarda en caer. Habría que tener un margen de error para no tener problemas en la posibilidad de que la maquina cometa un error en ese lapso de tiempo.

Un contador síncrono tiene como desventaja que cuesta más construirlo pero va a ser más rápido porque el reloj ataca a todos los flip flops al mismo tiempo, el tiempo para cambiar de un estado al otro va a ser el retardo de un solo flip flop.

### Registro de desplazamiento

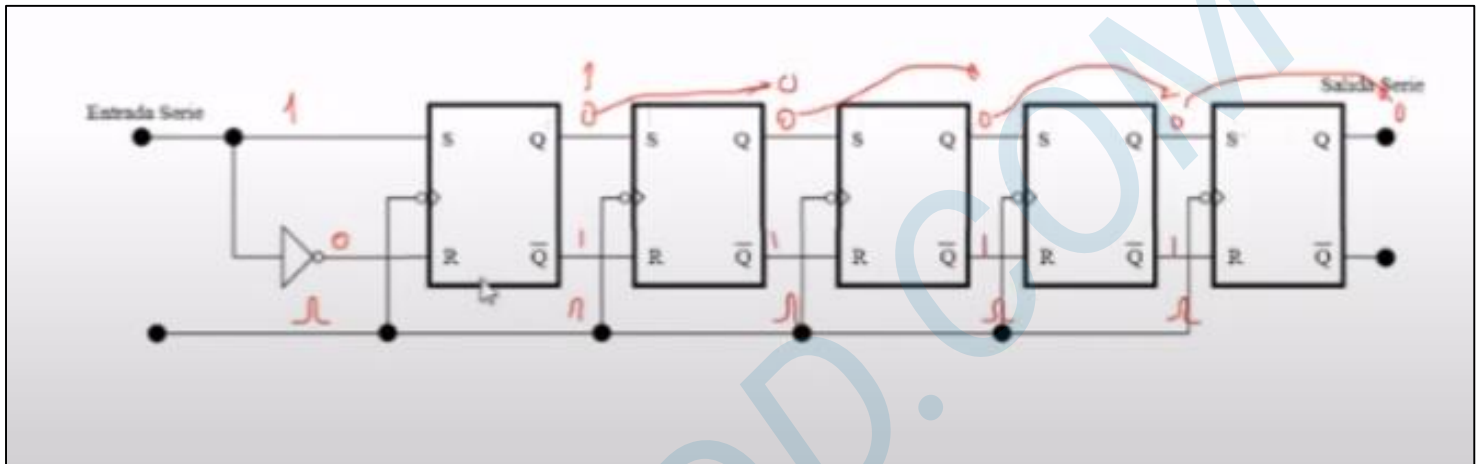
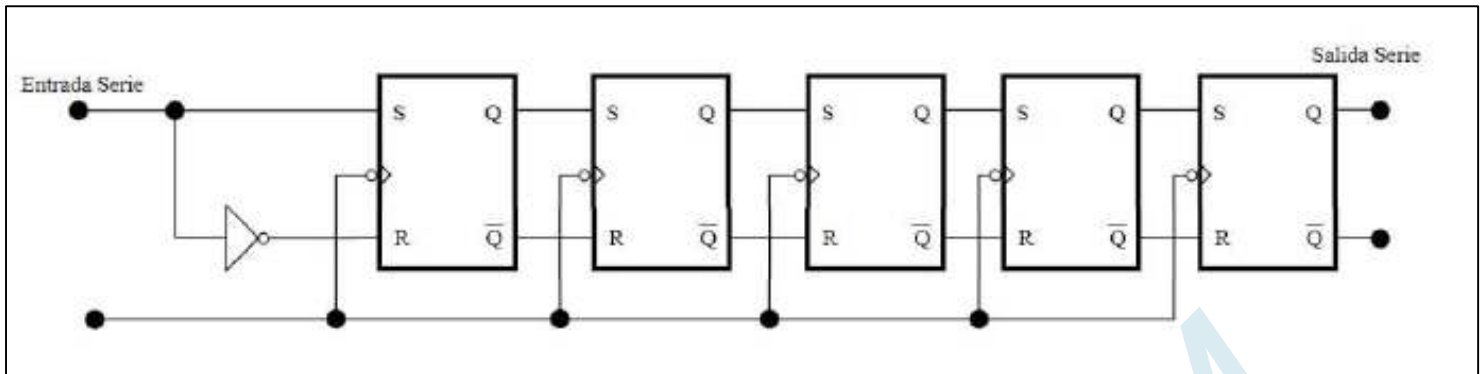
Se usa en las interfaces de comunicación, como en el USB que transporta los datos a una impresora que está allá lejos, a través de un cable, de un alambre, tanto la computadora como la impresora trabajan con datos en paralelo, estos circuito electrónicos de desplazamiento hacen una transformación generalmente de serie a paralelo y de paralelo a serie porque los dos elementos son necesarios en cada uno de las interfaces de comunicación para la trasmisión y para la recepción.

Es como cuando en la calculadora vos pones un número y desplaza los otros, eso sería un registro y está desplazando porque cada número nuevo desplaza hacia a la izquierda a los otros.

Es un circuito digital secuencial, hay 4 tipos

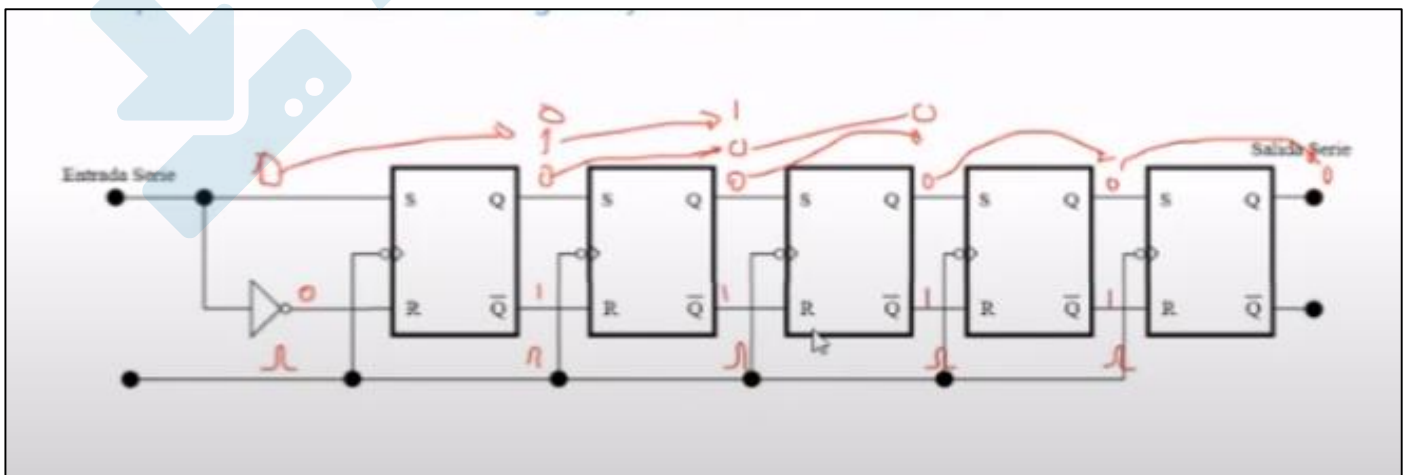
**Serie-Serie (SISO):** tiene una entrada serie y una salida serie (Serial In Serial Out), la importancia que puede tener es que al tomar una señal y trabajar adentro y después puedes liberarla con un delay, retardarla por algún motivo que vos quieras, y entonces ese registro será útil para eso. Solo la entrada del primer flip flop y la salida del ultimo son accesibles externamente, todo lo demás esta internamente. Se emplea como líneas de retardos digitales y en tareas de sincronización.

El último en entrar es el primero en salir.



El reloj (el primer punto negro debajo de entrada) es activado por flanco, le llega a todos los flip flop simultáneamente. Si vos en la entrada le pones un 1 lógico entonces el set va a tener un 1, y el reset va a tener lo contrario por el inversor teniendo un 0, sería como un flip flop de tipo D. Cuando vos mandes el primer pulso de reloj, si vos asumís que todos los flip fops en este comienzo estaban en 0, entonces ahora te das cuenta que cuando venga el primer pulso de reloj, le doy la orden a todos de que tomen lo que tienen en la entrada y lo pasen a la salida. Entonces como dijimos que todos los de arriba tenían 0, los de abajo tendrán 1 debido al inversor. Primer pulso, el de arriba dice que se ponga en 1 porque tiene un 1 set, entonces pasa de 0 a 1 el primero. Como yo tenía 0 1 en ese flip flop le da la orden de que se resetee, por tanto manda a la salida un 0 y hace que se desplace ese cero al siguiente flip flop, y entonces ocurrirá lo mismo en el siguiente y ese 0 se irá desplazando hasta llegar a la salida.

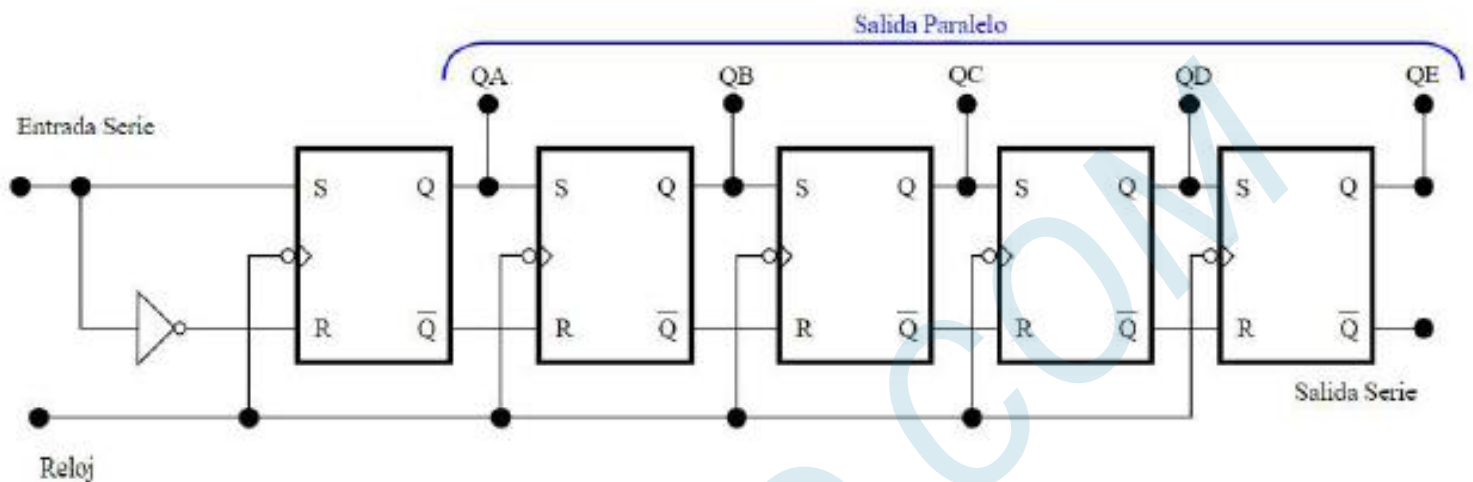
Se produjo un desplazamiento, porque el primer uno se metió adentro y el 0 se trasladó hacia allá y así se fueron todos corriendo como una oruga, se va corriendo toda la información.



Si el próximo paso es colocar un 0 en la entrada, te vas a dar cuenta que el 1 se va a correr y que el 0 nuevo va a tomar su lugar y ves como todos se van corriendo.

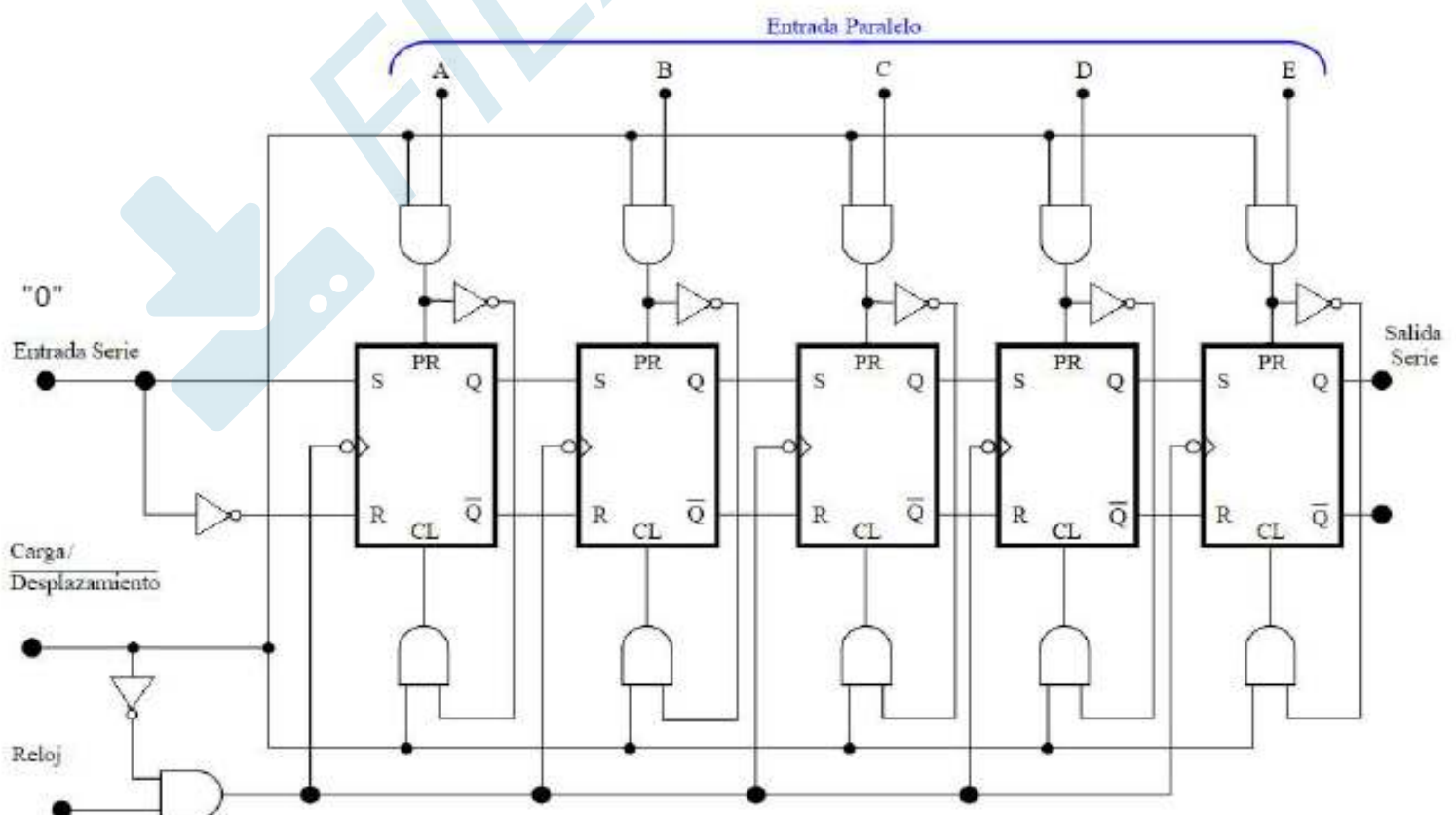
### Registro con entrada serie y salida paralelo (SIPO)

Se le añade una salida a cada una de las Q del biestable, de esta manera se pueden obtener todos los datos a la vez. (Serial In, Parallel Out)

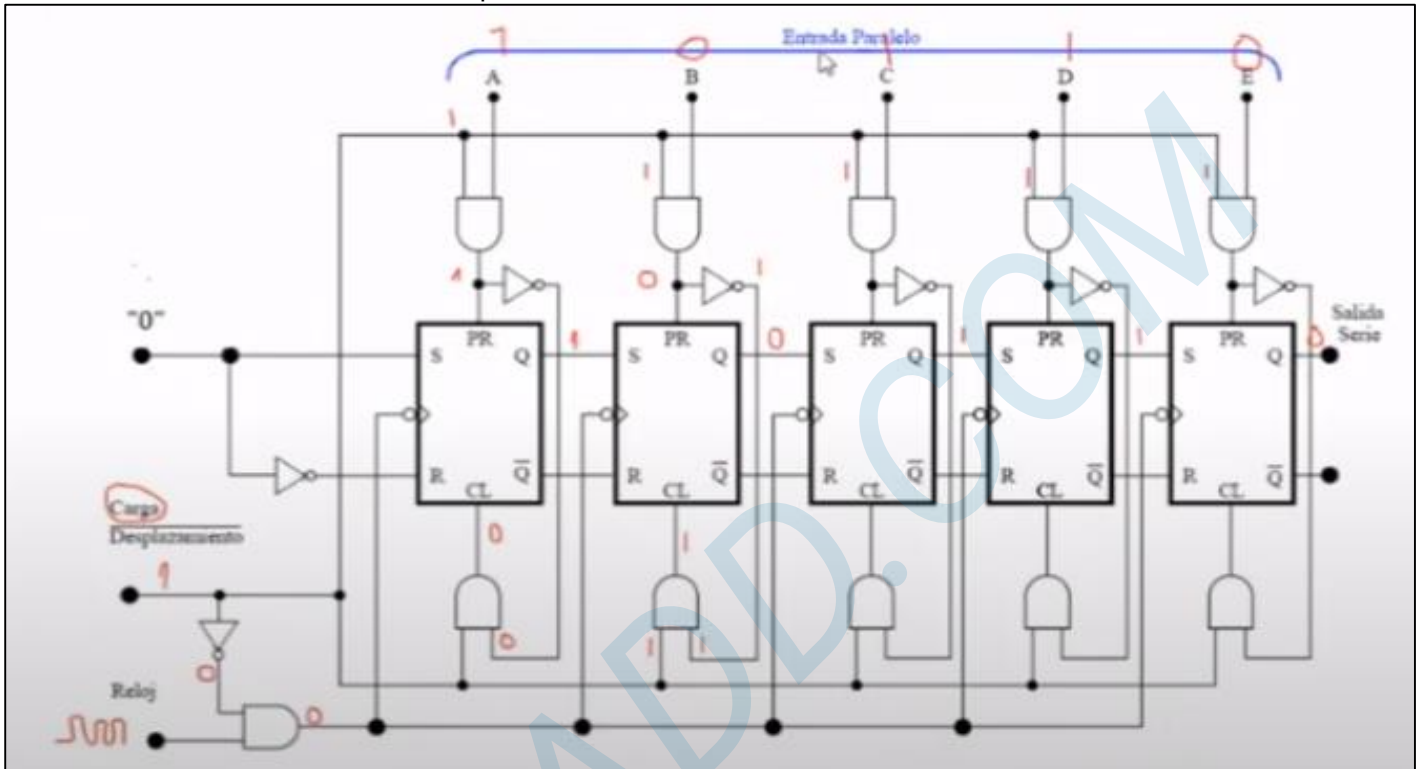


Le voy a mandar 5 pulsos de reloj, y con eso vamos a recibir los 5 ceros y uno que van viniendo del cable o la fibra óptica. En cada pulso del reloj un uno o un cero ira ingresando en la entrada y se va a ir desplazando en el interior hasta que después de los 5 pulsos, la información que has capturado en forma serial se ha quedado en la salida de cada uno de los flip flops acomodadita y esperando que venga el procesador de la impresora y tome el caracter que tiene en la salida QA, QB, etc. Y lo toma de una sola vez porque lo toma en paralelo, todos los 1 y 0 están disponibles en las salidas de los flip flops. Este es un chip que se usa en la interface de la comunicación para convertir la entrada serie a salida en paralelo.

### Registro con entrada paralelo y salida serie (PISO) (Parallel In, Serial Out)



Tiene una entrada de comando que se llama carga y desplazamiento, cuando vos le pones un 1 al punto negro que está debajo de carga desplazamiento, se produce la carga y cuando le pones un 0 se produce un desplazamiento. Carga significa que vos vas a cargar primero el estado lógico adentro de los flip flops que vos queres transportar. La entrada de los flip flops son en paralelo, se hacen por la parte superior A B C D E. Una vez que la información este adentro de la memoria entonces vos la vas a poder hacer salir por donde dice salida serie, a modo de un 1 bit por vez.

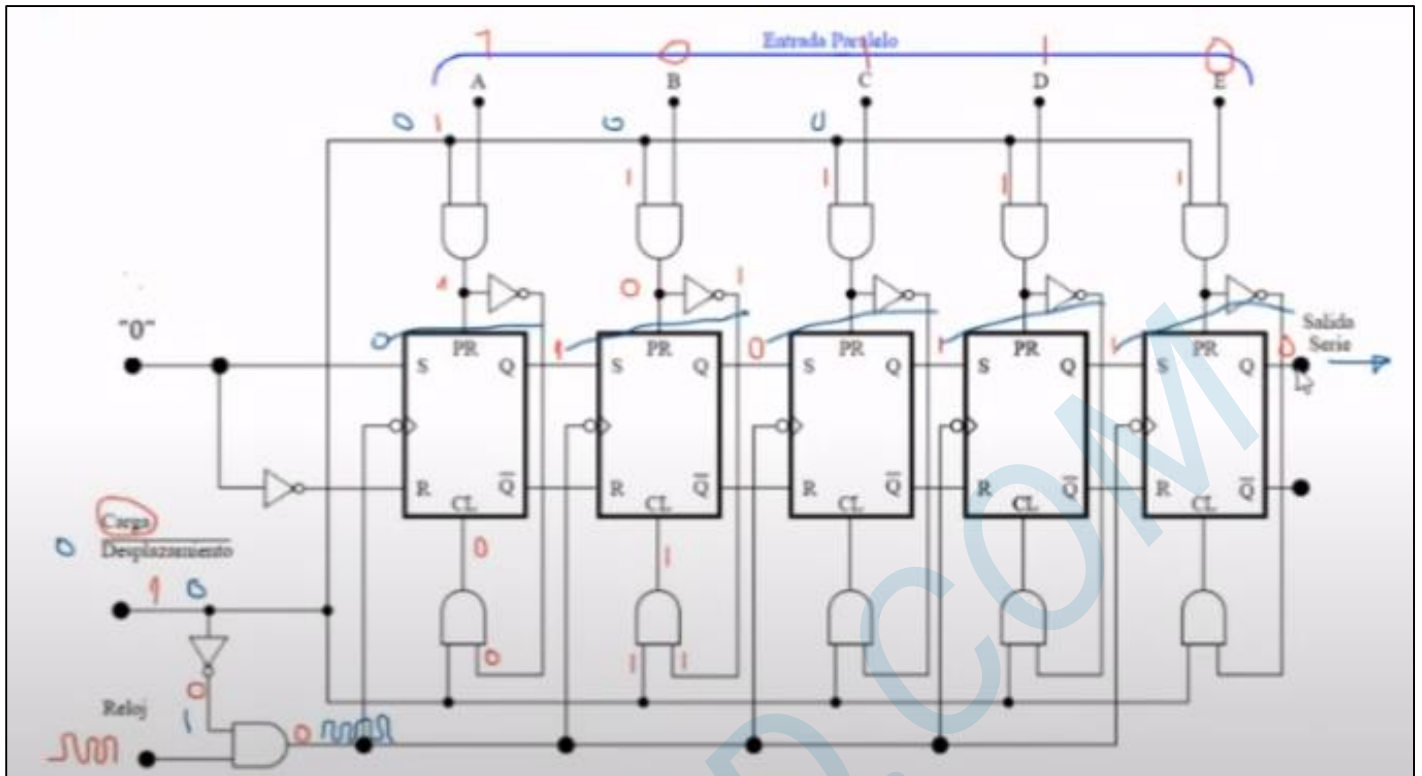


Primero lo pones en carga poniendo el 1 lógico. El 1 le pone un 0 por el inversor, entonces el reloj que se está moviendo no va a poder pasar adentro de la maquina porque se está multiplicando por cero, entonces ninguno de los flip flops se va a mover porque mientras esté en modo carga, el reloj no puede entrar.

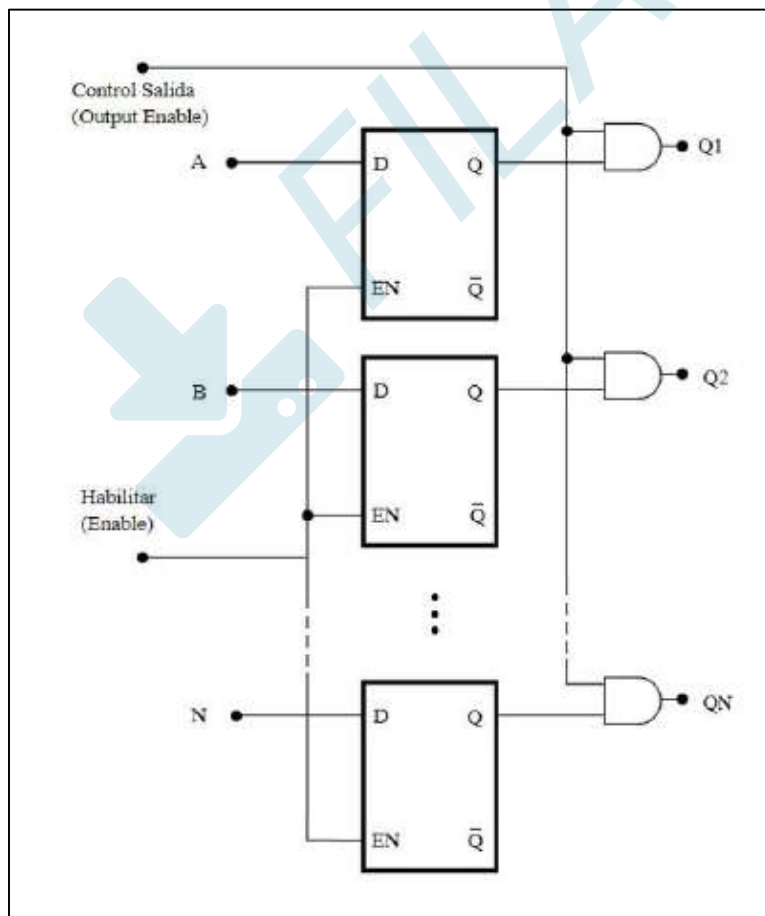
Sigamos el camino del 1, se va a ir para arriba y abajo, permite la entrada o el pasaje de los datos A B C D E de los bits, suponiendo que el número a transmitir sea 1 0 1 1 0 (inventado), el primero hace  $1 \times 1 = 1$  entonces la salida tiene un 1, entonces el preset que está abajo hace fijar la salida en un 1, y ese 1 con el negador le pone un 0 y hace que pase un 0 ya que todo multiplicado por 0 es 0, haciendo que a Clear le ponga un 0 para que no lo borre.

La entrada B vale 0, 0 multiplicado por 1 es cero, entonces no se acciona a preset, pero con el inversor tiene un 1 que viaja hasta abajo, y si se fijan, el primer 1 que pusimos a su vez también está conectado con cada una de las compuertas AND que están debajo de los flip flops entonces ese 1 que se hizo gracias al inversor se multiplica por el 1 lógico del principio haciendo que por la compuerta AND pase un 1, poniéndole un 1 al clear haciendo que recién en ese momento la salida se guarde en cero.

Entonces e esa forma los que son 1 se van a guardar como 1 y los que son 0 se guardan como 0 cuando está en modo carga este registro.

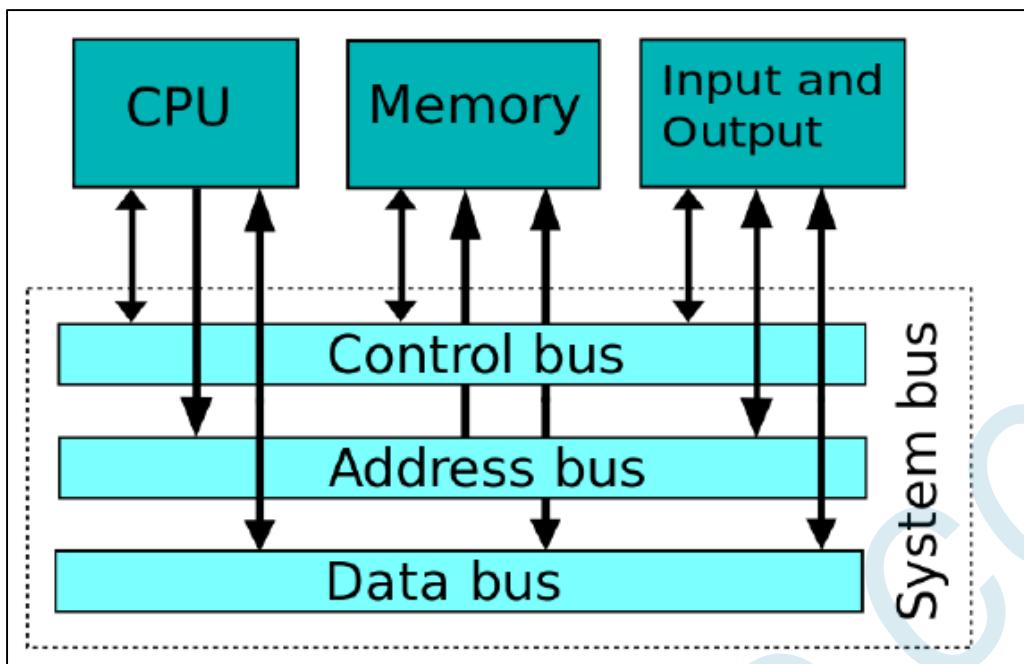


Ahora pasamos al modo desplazamiento con un 0 que estará en otro color. El 0 le pone un 1 en la compuerta AND de abajo debido al inversor, entonces ahora si se va a activar el pulso de reloj, cada pulso de reloj que reciba el flip flop van a impedir que los valores de carga ingresen puesto que son todos 0 multiplicando a las variables de entrada, haciendo que la misma no pasen. En los impulsos que pones del reloj se suponen que van a desplazar, todos los valores que habíamos cargado se van a ir desplazando de la misma forma que habíamos visto antes en el SISO.



### Registros con entrada y salida en paralelo (PIPO) (Parallel In – Parallel Out)

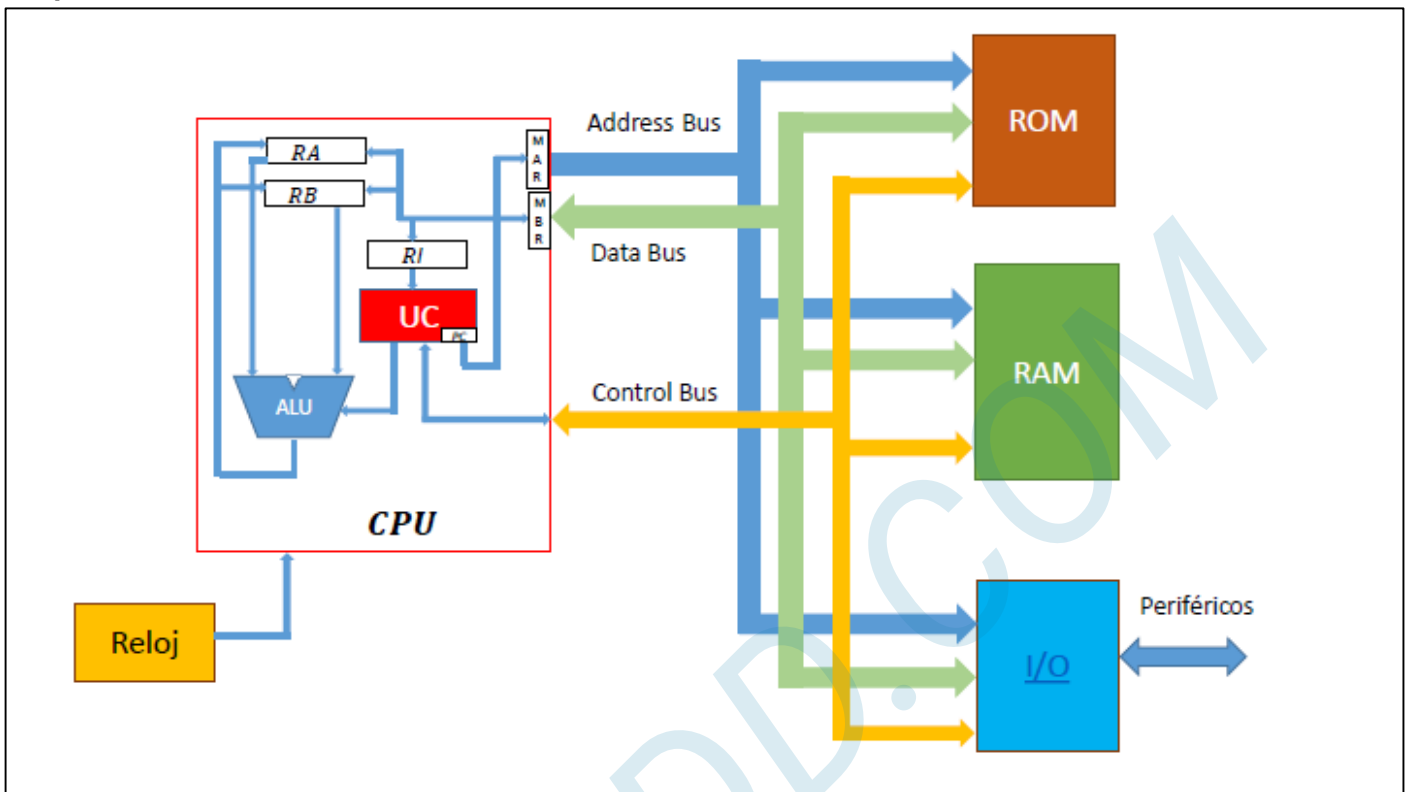
Las entradas y salidas están en paralelo, no hay transformaciones. Se usan en los puestos paralelos de las computadoras de escritorio generalmente. Pones el dato en las entradas y cuando tiras el pulso de reloj que lo recibe a través de enable, el dato se va a quedar atrapado. Cuando decidas que la información sea enviada a la salida, vos tenes un control para la salida que se llama el habilitador de salida (Output Enable), le clavás un 1 y cuando le pones un 1 verás que las barreras de paso se habilitan (Las compuertas AND) haciendo que salga lo que estaba en la salida del flip flop fuera del registro.

Comunicación entre componentes de la computadora

La comunicación entre los componentes de la computadora se hace a través de canales llamados buses, uno para datos, otro para direcciones y el último de control.

Los buses son caminos de comunicación entre dos o más chips o dispositivos y se los consideran como si fueran un medio de transmisión, que es compartido ya que la señal es transmitida a todo el sistema, pero solo dos al mismo tiempo utilizarán estas autopistas de datos. Es la CPU que arbitra a través del bus de control con quien se quieren comunicar todos los dispositivos.

El concepto de bus es como el de una gran autopista por donde pasan los datos. Son líneas de alambre que están uno al lado de otra, la información se manda en paralelo y de acuerdo a la selección solo dos dispositivos están dialogando a un tiempo. Se hace tan rápido que parece que todo pasa simultáneamente, no podríamos tener muchos caminos de conexión, porque entonces no habría como construir la placa madre con todos los alambres.

Sistema de Microcomputador**Arquitectura de Von Neumann**

Un sistema de microcomputador está compuesto por 5 partes: La CPU o microprocesador, memoria ROM, memoria RAM, los dispositivos de entrada y salida (I/O) y el reloj. Siempre toda computadora tiene estos 5 módulos. Se interconectan los módulos con líneas que corren, llamadas buses y tenemos 3 buses: Bus de Dirección (azul), Bus de datos (verde), Bus de control (amarillo). Todos los buses son elementos compartidos que van a todos los dispositivos simultáneamente, pero es 1 y solo 1 de ellos, generalmente el micro, que se va a ir comunicando con cada uno de los módulos mediante los buses para intercambiar información.

**Memoria ROM:** es una memoria que no es volátil, se llama ROM (Read Only Memory, Memoria de Lectura solamente, su acceso es de forma aleatoria), se encuentran los programas de inicio de la placa madre, que son instrucciones de máquinas muy elementales y que cuando la máquina hace un arranque en frío hay un conjunto de programas aquí adentro que le permite chequear que la memoria RAM esté bien, se testean muchos dispositivos para ver si la comunicación anda bien, generalmente en esta parte se ve el logo del fabricante cuando se está encendiendo la máquina.

**Memoria RAM:** se aloja aquí el sistema operativo, es de lectura/escritura y es volátil, es decir que con ausencia de energía eléctrica la información se va a perder. Se le llama RAM por (Random Access Memory que significa Memoria de acceso aleatorio), como es de acceso aleatorio acceder a cualquier byte va a tomar la misma cantidad de tiempo. Es la memoria principal, se alojan según la arquitectura de Von Neumann programas y datos. Está conectada con el disco rígido mediante el módulo de I/O.

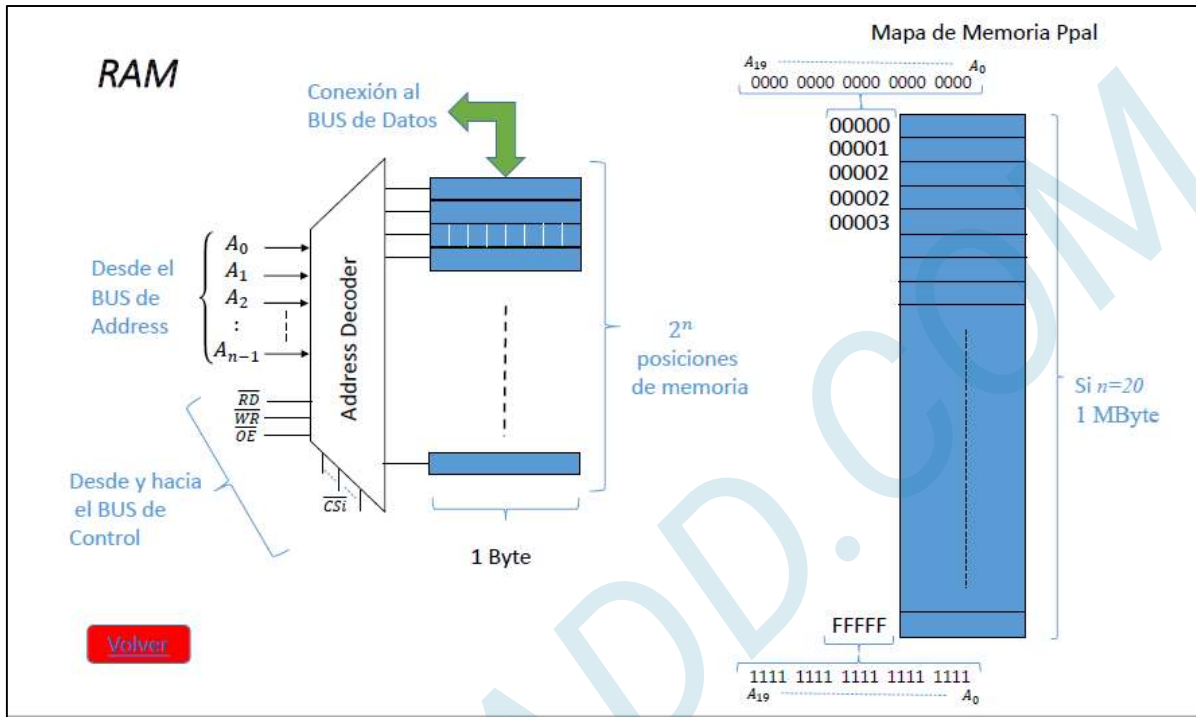
**I/O:** hay una porción de memoria, de puertos, donde nos deja conectarnos con dispositivos externos, como el mouse, la pantalla, el teclado, disco rígido, etc. Y por ahí entran los datos de los periféricos.

**Reloj:** su base de tiempo te va dando la idea temporal de funcionamiento de la máquina

Adentro del procesador existen registros que son arreglos de memorias de un bit todos encadenados y el tamaño de estos registros son múltiplos de 8 (RA y RB). Hay un registro de instrucción (RI) donde se va a alojar la instrucción que se está a punto de ejecutar en este momento. En las puertas de entradas del procesador existen dos registros que no vas a tocar pero van a estar siempre en los procesadores, que son el Memory Address Register y el Memory Bus Register (MAR) y (MBR)

## Distintos tipos de buses (tomando en cuenta el dibujo de la arquitectura de Von Neumann)

**El bus de direccionamiento** (Address Bus) es un bus unidireccional que sale desde el microprocesador y va hacia todos los dispositivos que forman la placa madre. Las líneas de este bus (azul) son 40 actualmente y están en paralelo. Las líneas sirven para elegir un byte adentro de la memoria RAM. Si vos pensás cómo es la memoria RAM, hoy vos tenes 8GB de RAM por tirar un ejemplo, adentro de ella tenés la siguiente estructura:



La RAM es un gran decodificador, donde tiene pocas líneas que le llegan pero que después si son 8GB tiene 8 mil millones de estos bytes almacenados ahí adentro (esas pilas azules), que son lugares donde puedes guardar 8 unos y ceros. Pero no vamos a poder conectar 8 mil millones de alambres, entonces gracias a este decodificador de Address es que podemos hacer las conexiones para buscar el byte que andemos necesitando con pocas líneas que funciona de forma exponencial entre las entradas y salidas, ACORDATE 2 elevado al número de líneas de direccionamiento ( $2^n$ ) te da la cantidad de bytes que va a poder manejar la máquina.

Volvemos luego de haber ahondado un poco en la RAM a la arquitectura de Von Neumann, para seguir explicando entonces como hace el bus de Address para poder entonces elegir ese byte con el que nos queremos comunicar. Las líneas de direccionamiento le llegan a todos los dispositivos en general pero en este momento estoy eligiendo uno y solo uno de los bytes con una combinación binaria en el bus de address, o sea que es unidireccional y te permite elegir una y solo una posición de memoria a un tiempo, eligiéndola o seleccionándola con la combinación binaria que largues en este bus. Si una maquina tiene 32 líneas, la capacidad de direccionamiento es la capacidad máxima o ancho de banda RAM que va a poder manejar es  $2^{32}$  si haces la cuenta corresponde a 4GB de RAM.

Agarrá una calculadora: hace  $2^{32}$  y te va a dar un número gigantesco (porque voy a considerar que el bus de address de esta máquina tiene 32 líneas) una vez que tengas el número, esa es la cantidad de bytes que la CPU va a poder elegir, uno entre todo ese numeron de bytes va a elegir. Esa es la cantidad de bytes que maneja la computadora. Pero vos no vas a ir a comprar una memoria con la calculadora en la mano mostrándole al señor que quieres esa memoria para colocarle a tu máquina. Entonces lo que vas a hacer es dividir a ese número por 1024, cuando haces esto lo pasas a kilobyte, que es una unidad irrisoria de la que nadie habla. Ahora volves a dividir por 1024, entonces lo pasas a megabyte, pero sigue siendo mucho, entonces volver a dividir por 1024, pasándolo a gigabyte, dando exactos 4 gigabytes, de esa forma se hacen las cuentas.

El bus de address es importante porque si sabes la cantidad de líneas que tiene vas a saber cuál es la memoria máxima expandible que vas a poder expandir a tu computadora. Si vos agregas una línea más duplica la cantidad de memoria, o sea que es muy importante tener el concepto de líneas de address, porque solo una línea aumenta al doble la cantidad de memoria, con 33 líneas pasas de 4GB a 8GB.

Las pilas de bytes funcionan como una gran tabla de verdad, vos le ponés una combinación binaria y entonces podés seleccionar el byte que responda a la misma, vas desde el 0 0 0 0 0, hasta el 1 1 1 1 1 y para que sea más fácil de entender, en vez de numerarlas en binario lo haces en hexadecimal, entonces si pones FFFFF estarías seleccionando el último byte (registro) dentro de la RAM.

En la RAM existen líneas que lo tienen todos los chips que vienen desde el bus de control (será explicado en breve), donde dicen que la van a leer (RD) a ella, que la van a escribir (WR) y la memoria se va a elegir con una serie de selectores, cuando yo le ponga 111 entonces el microprocesador esta potable de leer a cualquiera y a través de las líneas amarillas (bus de control) le va a ir a decir, te voy a leer a vos (modulo) o voy a elegir de la RAM.

Una vez que vos elegiste el comando de la RAM, un dato que queres leer o escribir, el bus de datos se va a dedicar a transportar o va a tomar un valor que se encuentre en la RAM y lo va a traer entonces a través del canal verde que es el bus de datos y es bidireccional. Va a tomar el valor que está en la RAM y lo va a traer pasando a través del registro de memoria del bus, que lo va a dejar ahí almacenado y a su vez ese dato podría ir a parar finalmente adentro de un registro del procesador del CPU. Y sino al revés, suponga que una persona tiene un resultado en el registro B y lo quiere guardar en la RAM, entonces lo lleva, lo pone en el registro de memoria del bus y ahí lo va a transferir eligiendo el lugar donde lo quiere guardar con el bus de direccionamiento y una vez que elegido el lugar le va a mandar a través de la línea verde desde el registro hacia uno de los registros de la RAM.

El bus de datos entonces es bidireccional y sirve para mandar información desde la memoria hacia los registros internos del procesador en caso que sea datos y en la arquitectura que sea la de Von Neumann también se mandan las instrucciones por acá.

El tamaño de Bus de Address define la capacidad de direccionamiento, capacidad de memoria que podemos manejar, pero el tamaño de bus de datos coincide con el tamaño del registro. En el pasado los registros de RA y RB eran de 8 bits, al igual que el bus de datos, entonces todos eran felices, los 8 bits iban y venían por el bus de datos de 8 bits, pero después fueron evolucionando, cuando los registros pasaron a 32 bits, el bus de datos fue también de 32.

El bus de control cumple el rol de primero seleccionar con cuál de los dispositivos se quiere comunicar, para lo cual van a haber señales adentro del bus que no tienen la misma regularidad que los otros buses que eran todos iguales e iban desde el menos significativo (0) hasta el más significativo (n-1). En el bus de control hay muchas líneas independientes y cada una se llama distinto. Hay líneas para decirle a la memoria una vez que la seleccionaron tanto el renglón como la información si la vas a leer o escribir. Por ejemplo Intel para leer pone 0 en la línea read y para escribir pone 0 en write (fíjense en el dibujo de la memoria RAM en la parte abajo izquierda). Output enable (OE) quiere decir que una vez que haya seleccionado lo que voy a hacer recién voy a tirar un pulso por la habilitación de la salida para que físicamente la información salga y hace tanto pasos porque quiero asegurarme que la información no vaya a ir corrupta. Entonces cuando yo seleccione la acción, por ejemplo Write, dejo pasar un tiempito y después aplico Output Enable, entonces las cosas van a salir bien. También el Bus de control nos indica qué blíster se seleccionará dentro de la RAM mediante el chip selector) (averiguar que mierda es un blíster).

En el bus de control también puede pasar que cuando vos vas a buscar información del disco duro para guardar en la RAM, en el pasado se tomaba un byte, o sea una instrucción, se seleccionaba esa instrucción a través del bus de address, luego se tomaba el byte que viajaba por el bus de datos y se depositaba temporariamente en un registro del procesador para que luego en otro tiempo salga del registro del procesador y vaya a parar a la RAM, se seleccionaba un lugar dentro de ella y se guardaba la instrucción dentro de ella. Esto es muy retardado, entonces adentro del control bus hay una línea que se llama DMA (Direct Memory Access) que significa Acceso Directo a Memoria, que la maneja la CPU y el dispositivo externo como el disco duro. Cuando el disco duro acciona la DMA, accede, toma la memoria y le pasa

directamente los datos, o sea que suelta las líneas de direccionamiento por única vez, con el acceso de memoria las líneas de direccionamiento las toma el procesador del disco duro y comienza a hacer la transferencia el mismo, transfiere directamente a RAM el programa, ahorrando tiempo. Ese arreglo viene por el bus de control.

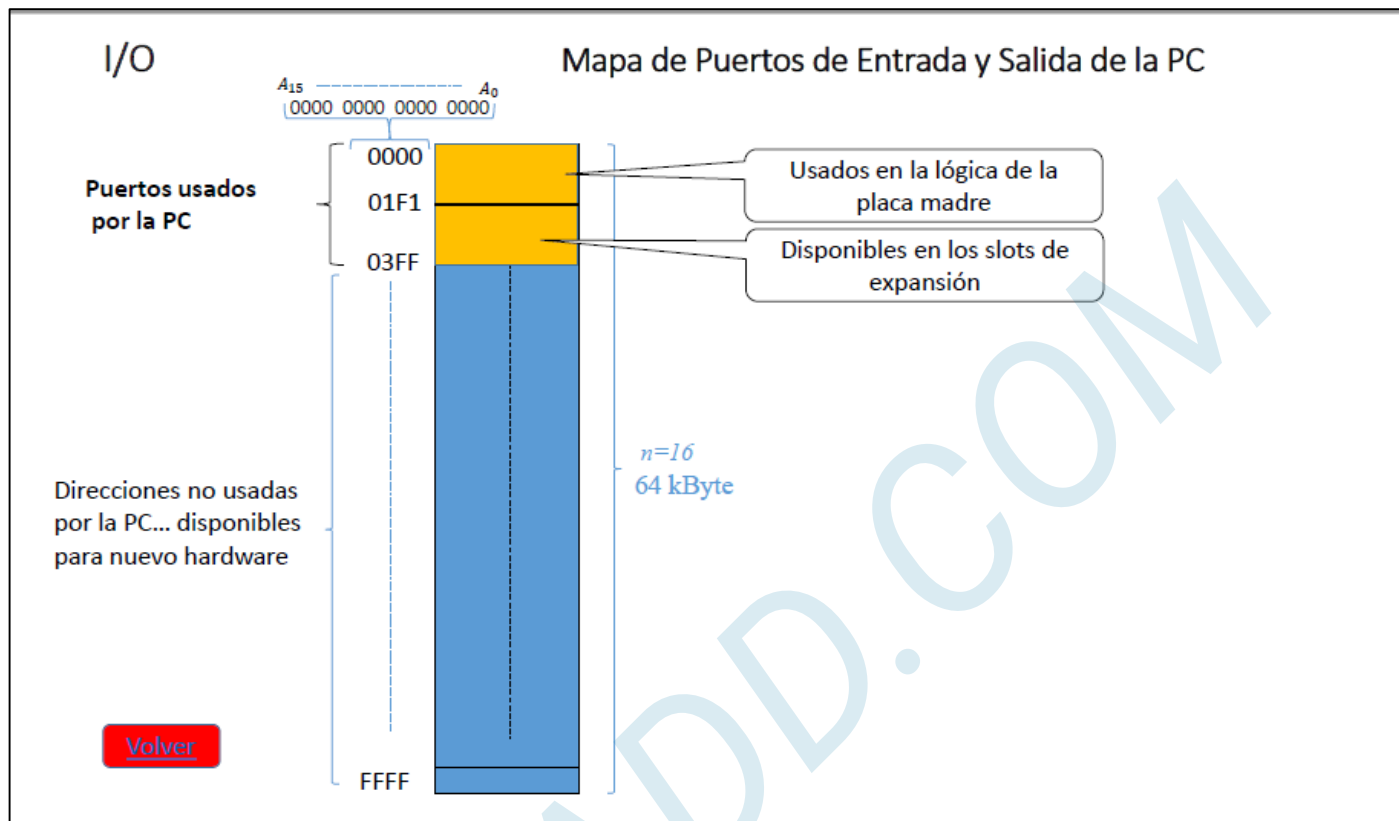
Entonces adentro del bus está Read, Write, Chip Selector, Output Enable, DMA, y luego también hay otro mecanismo. Supongamos que una impresora está imprimiendo y al mismo tiempo estoy escribiendo texto, no es que yo tengo que dejar de hacer algo porque la impresora está trabajando, entonces da una sensación de que todo es simultáneo. Lo primero que hago es mandarle a la impresora un grupo de caracteres para que se ponga a trabajar, ella tiene un buffer, un lugar de memoria RAM a la que le puedo mandar una cierta cantidad de tareas para que haga, pero luego tengo que preguntarle cada tanto si necesita mas datos. Si hago ese mecanismo sería muy retardado porque yo estaría a cada rato si necesita, entonces sería mejor que la impresora indique cuando necesita datos, así no se pierde tiempo en esa operación, por tanto se ideó un mecanismo llamado interrupción, entonces cuando la impresora ve que su buffer se está vaciando le va a solicitar a la CPU que le mande más datos mediante este mecanismo. Hay muchas líneas de interrupción porque puede haber muchos dispositivos periféricos conectados a la computadora, así que a esas líneas se las llama IRQ (Interrupt Request), cada periférico toma una de ellas para que la CPU sepa quién de todos le pide atención a la interrupción. Entonces el mecanismo interrumpe al micro para que pueda atenderlo y eso es de forma asíncrona, aparece en cualquier momento, pero de esa forma la CPU se desentiende de si los periféricos necesitan algo. Estas interrupciones IRQ son enmascarables, o sea, que la CPU cuando las recibe le puede decir al dispositivo: "Bueno ya se que necesitas esto, pero ahora estoy muy ocupado en otra cosa, así que espera". La enmascarable no es tan prioritario, la CPU la recibe, y cuando se haya descongestionado y terminado lo que tenía que hacer, atenderá la interrupción de esta característica enmascarable, siendo todas las IRQ de nuestra computadora enmascarables. Cuando vos enchufas un periférico entonces el sistema operativo lo detecta y le asigna una IRQ.

Hay otra interrupción que se llama NMI (Not Maskable Interrupt) sería la no enmascarable, y hay solo una, esa interrupción cuando entra el micro dice: "Ojo, acá se para todo". Es como el botón de pánico, el microprocesador deja de hacer todo lo que está haciendo y se pone a ejecutar una rutina de atención a esa interrupción, algo que estaba previamente escrito y guardado en la RAM según se cargó en el arranque en frío la rutina de atención de interrupción.

Cuando nos relacionamos con los periféricos el bus de control tiene líneas específicas para ellos), tales como la I/O Read o la I/O Write. Cuando vos quieres leer o escribir los periféricos no usas el mismo Read o Write que usas para la RAM o ROM. Entonces como vos veras, la señal de control pueden ser bidireccionales y cada una tiene un sentido distinto en función de lo que le toca hacer.



Para los periféricos se usan en las computadoras las últimas 16 líneas del bus de direccionamiento para seleccionar el mapa de entrada y salida. Ese mapa que se crea se le llama el Mapa de Puertos de Entrada y Salida de la PC, es el siguiente:



Si vos querés acceder a un puerto usas dicho mapa siguiendo las direcciones de memoria descritas en él. Los primeros 15 bytes se destina al controlador de acceso directo memoria por ejemplo.

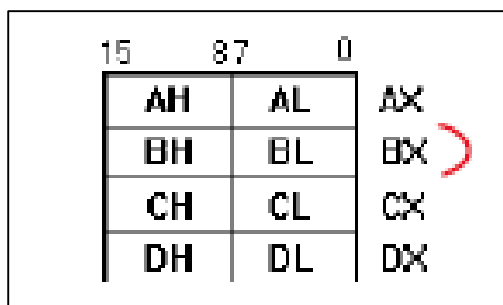
**Puertos usados por la PC**

| Puerto (hex) | Asignación                                              |
|--------------|---------------------------------------------------------|
| 000-00F      | Controlador DMA                                         |
| 010-01F      | Controlador DMA (PS/2)                                  |
| 020-02F      | Controlador maestro programable de interrupciones (PIC) |
| 030-03F      | PIC esclavo                                             |
| 040-05F      | Temporizador programable de intervalos (PIT)            |
| 060-06F      | Controlador del teclado                                 |
| 070-07F      | Reloj en tiempo real                                    |
| 080-08F      | Registro de página DMA                                  |
| 090-09F      | Seleccionador programable de opciones (PS/2)            |
| 0A0-0AF      | PIC #2                                                  |
| 0C0-0CF      | DMAC #2                                                 |
| 0E0-0EF      | reservado                                               |
| 0F0-0FF      | Coprocador matemático, controlador de disco PCjr        |
| 100-10F      | Seleccionador programable de opciones (PS/2)            |
| 110-16F      | LIBRE                                                   |
| 170-17F      | Disco duro 1 (AT)                                       |
| 180-1EF      | LIBRE                                                   |
| 1F0-1FF      | Disco duro 0 (AT)                                       |
| 200-20F      | Adaptador de juegos (Joystick)                          |
| 210-21F      | Puertos de tarjetas de expansión                        |
| 220-26F      | LIBRE                                                   |
| 278-27F      | Puerto paralelo 3                                       |
| 280-2A1      | LIBRE                                                   |
| 2A2-2A3      | Reloj                                                   |
| 0378h        | LPT1 Puerto paralelo1                                   |
| 03F8h        | Com1 Puerto serie                                       |

[Volver](#)

Los últimos lugares están destinados a nuevos componentes de Hardware que el usuario puede llegar a expandir, asignándole una dirección nueva a cada nuevo dispositivo.

Ahora vamos a estudiar los registros internos de la máquina que se emplean para el lenguaje de máquina. Se llaman GPR (General Purpose Register) a los 4 registros que vamos a ver a continuación. Un registro es una sucesión de flip flops pegados uno al lado de otro que tiene un determinado orden que va desde el menos significativo hasta el más significativo.



Intel crea en este momento un procesador de 4 registros de 16 bits. Del 0 al 7 y después del bit 8 al 15, en total 16 bits.

En esa época se utilizaban 8 bits, entonces para que la gente no descartase el producto por ser de 16 bits se dividió al registro en dos partes de 8, teniendo entonces la parte alta H (High) y baja L (Low) en el mismo.

El registro AX se lo llama acumulador, es de 16 bits si lo usas como un todo, si no a la AX lo cortas en dos y lo usas como dos registro,

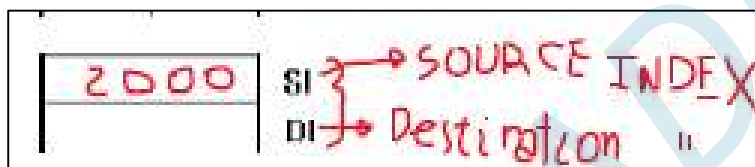
AH por High (la parte alta), y AL por Low( la parte baja). Si vos haces esto último ambas partes se desconocen entre sí.

El registro BX se lo llama base.

El registro CX se lo llama contador o counter.

El registro DX se lo llama data o dato.

Luego vienen dos registros más que los vamos a usar que son punteros y son muy importantes:



El primero se llama Source Index (SI) índice fuente u origen y el otro se llama Destination Index (DI) índice destino.

Si vos agarras a la memoria y te la imaginas, estos dos son punteros en ella. El concepto de puntero viene ligado a estos registros internos del procesador. Si vos tenes un bloque de memoria RAM conformado por el bloque de bytes con sus respectivas posiciones de memoria y lo que está adentro es el dato. A veces yo quiero acceder al valor que está adentro, entonces simplemente voy y le digo al procesador con una instrucción que busque dicho dato y que lo lleve a uno de los registros de propósito general que vimos antes. Pero en otras necesito acceder a un dato detrás de otro como en un texto, una cadena de caracteres (string) por tanto si necesitas datos que son secuenciales no te conviene buscar uno por uno, entonces lo que se hace es usar un puntero en la memoria, que es como indicar con una flechita cual es el primer carácter a trabajar, con un puntero, señalándola, y para ello se escribe dentro del registro (le entran 4 cifras) la posición en memoria donde está el dato. Si quisieras entonces que siga avanzando como una cadena de texto tenes que indicar en el algoritmo (esto ya es más practico) hasta donde queres que vaya, sumándole 1 y deteniendo en la posición de memoria donde queres dejar de extraer datos usando el puntero.

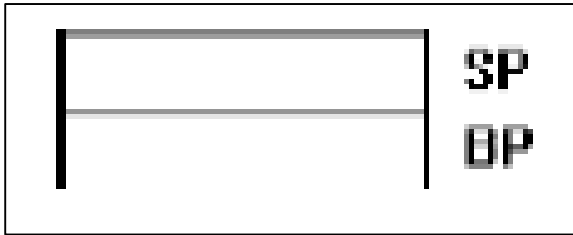
El destination point trabaja igual que el SI, mueve datos a un lugar destino

BX también puede usarse como puntero.



Después tenemos el puntero de instrucción (IP: Instruction Point) lleva el orden de ejecución que es el contador del programa. Va a ir ejecutando la instrucción que tiene en una posición de memoria determinada. Solo tenes que marcar el punto de partida, donde está la primera posición de instrucción y después corre solo.

Estos registros de INTEL usan la arquitectura CISC, donde el largo de las instrucciones es variable, el puntero de instrucción no va saltando de 1 en 1, a veces puede saltar 1 byte, 2, 4 o más byte. El tamaño de los saltos no es un valor predecible, depende del tamaño de las instrucciones, porque el largo de las instrucciones es variable. En cambio en un RISC las instrucciones son una tras otra.



Otros registros importantes son el Stack Pointer y el Base Pointer.

El stack pointer o puntero de pila, es de tipo LIFO (Last In, First Out, ultima en entrar, primera en salir) Todo procesador tiene una pila que almacena datos temporarios. Hablando de forma criolla y a modo metafórico es como si fuese una

especie de historial de la máquina que luego se utiliza para retomar subrutinas o procesos, siguiendo la ruta de acciones llevadas a cabo dentro de esta pila. Se ve mejor en AED. Podes recuperar programas usados antes con sus registros correspondientes.

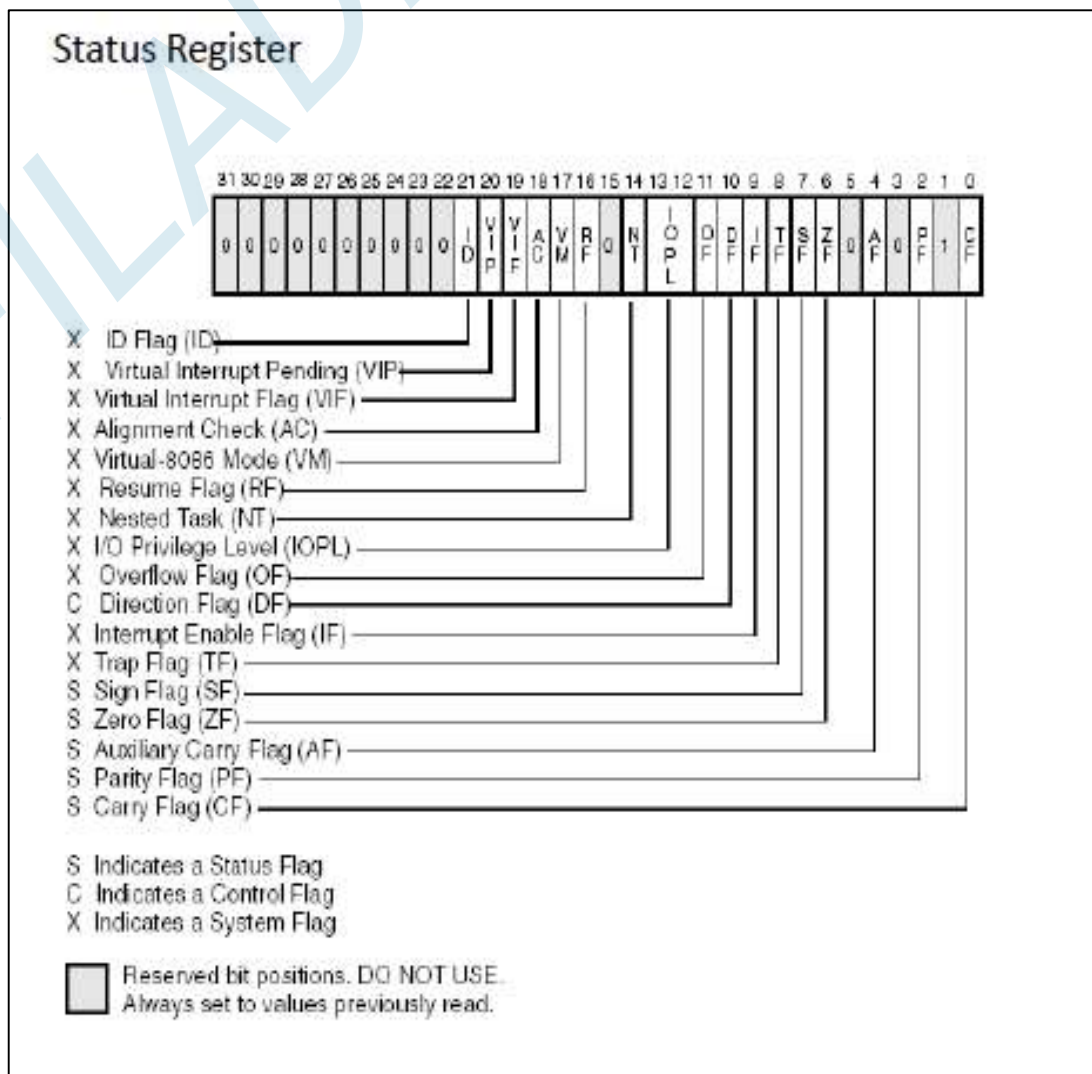
Entonces el Stack Pointer es el puntero de la pila, la flecha que va subiendo cuando apila, y cuando desapila baja y va llevando el lugar de la última posición de esa pila.

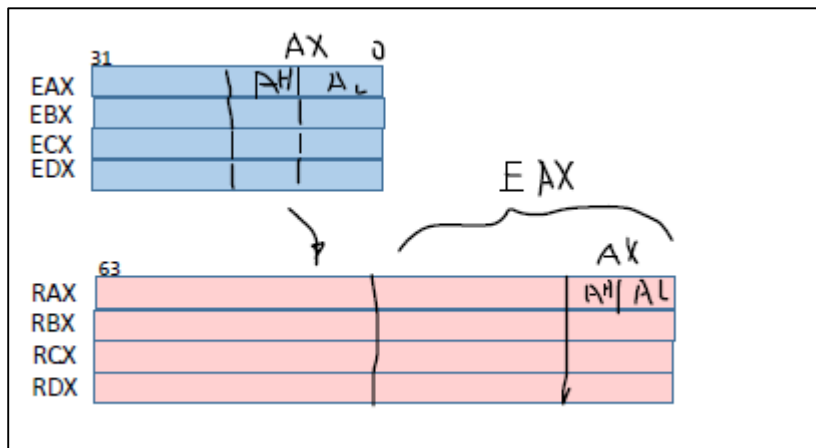
El Base Pointer o puntero base, trabaja también en el segmento de pila, o sea la zona de la pila, podrías poner un puntero para que fuera a explorar los datos y no afectase al normal funcionamiento del puntero de pila que sube y baja. Mueve datos en la zona de la pila.



Tenemos otro registro que se llama el Status Register o de registro de estado. Este registro tiene un concepto distinto, importan cada uno de los bits que forman parte de este registro. Le haremos un zoom al Status

Register. Estos bits son importante cada uno de ellos tienen un significado, se los llama banderas, por eso tienen una letra F en un montón de lugares (flags). Se conoce como registro de estado a los registros de memoria en los que se deja constancia de algunas condiciones que se dieron en la última operación realizada y que podrán ser tenidas en cuenta en operaciones posteriores. Por ejemplo, en el caso de hacer una resta, en el registro de estado queda constancia de si el resultado fue cero, positivo o negativo, o si se sobrepasó la capacidad de representación.





Los registros crecieron así: Intel pasó de 16 bits a 32 bits. En la parte baja del azul están contenido los 4 registros anteriores. Lo expandió hacia la izquierda, duplico el tamaño del registro. La E significa extended, entonces sería ExtendedAX, ExtendedBX, etc.

El amarillo es cuando pasaron de 32 a 64 bits, se replica todo lo azul y se lo mete ahí adentro. Como ven el AL de la máquina de Intel de los 80s puede seguir corriendo tranquilamente. La R es de Register.

## Lenguaje de máquina

### Instrucción:

Una instrucción de maquina es una operación expresada mediante la codificación binaria de cadenas de 1's y 0's. Se le denomina lenguaje de máquina. Es la categoría 4 de código. Pero a su vez es muy difícil acordarse la cadena de unos y ceros en una secuencia muy grande, por tanto se emplea un truco que se llama lenguaje de ensamblador, que es un juego de instrucciones expresados en mnemotecnica que son fáciles de acordarse. El repertorio de instrucciones o juego de instrucciones son el conjunto de órdenes que puede ejecutar un computador.

### Modos de direccionamiento

Los modos de direccionamiento hacen referencia a las distintas formas que los microprocesadores tienen para obtener o acceder a los operando. El operando o dato está en la memoria, entonces las instrucciones tienen distintas formas de ser usadas pero todas tratan de ir a buscar de algún modo u otro los datos que a veces están en la memoria de la máquina, a veces está en un registro interno, a veces ese dato se lo da de una porque te lo da en la misma instrucción. Las instrucciones a veces son las mismas, pero de acuerdo al modo de direccionamiento son las distintas formas de en la que se buscarán los datos.

### **Modo de direccionamiento INHERENTE o IMPLÍCITO**

La instrucción implícitamente contiene la orden a realizar. No hace falta más nada, simplemente con darle el nombre de la instrucción ya sabe lo que hace.

Ejemplos:

CLC (clear carry) borra la bandera de acarreo.

STI (Set Interrupt flag) fija o enciende la bandera de interrupción.

CLI (Clear Interrupt flag) borra la bandera de interrupción.

NOP (No Operation) no hace nada y solo pierde un tiempo de reloj.

### **Modo de direccionamiento REGISTRO**

El dato u operando está en un segundo registro.

Acá debuta la instrucción mas famosa, se llama MOV, que significa mover datos de un lado a otro, si decimos MOV AX,BX tenemos dos registros separados por una coma, el de la izquierda (AX) es el registro destino y el de la derecha (BX) es el registro fuente, se asigna de derecha a izquierda.

|            | Antes                                                                                                                                      | Despues |   |   |   |   |   |   |   |                                                                                                                                            |   |   |   |   |   |   |   |   |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------|---------|---|---|---|---|---|---|---|--------------------------------------------------------------------------------------------------------------------------------------------|---|---|---|---|---|---|---|---|
| MOV AX, BX | AX <table><tr><td>1</td><td>2</td><td>3</td><td>4</td></tr></table><br>BX <table><tr><td>5</td><td>6</td><td>7</td><td>8</td></tr></table> | 1       | 2 | 3 | 4 | 5 | 6 | 7 | 8 | AX <table><tr><td>5</td><td>6</td><td>7</td><td>8</td></tr></table><br>BX <table><tr><td>5</td><td>6</td><td>7</td><td>8</td></tr></table> | 5 | 6 | 7 | 8 | 5 | 6 | 7 | 8 |
| 1          | 2                                                                                                                                          | 3       | 4 |   |   |   |   |   |   |                                                                                                                                            |   |   |   |   |   |   |   |   |
| 5          | 6                                                                                                                                          | 7       | 8 |   |   |   |   |   |   |                                                                                                                                            |   |   |   |   |   |   |   |   |
| 5          | 6                                                                                                                                          | 7       | 8 |   |   |   |   |   |   |                                                                                                                                            |   |   |   |   |   |   |   |   |
| 5          | 6                                                                                                                                          | 7       | 8 |   |   |   |   |   |   |                                                                                                                                            |   |   |   |   |   |   |   |   |

El valor que tiene BX se le pasa al registro AX, eso hace la instrucción MOV. Se copia el valor que tiene BX al valor AX, el 12 34 se pierde para siempre y los dos registros quedan iguales. El hecho de que se mueva un valor del registro B al A, no quiere decir que B se quede en cero, se hace una "copia".

Otra instrucción que tiene el modo de direccionamiento registro es la OR, que hace una OR entre dos registros. Supongamos el siguiente ejemplo:

|           |    |                                                         |   |    |    |    |                                                                                      |   |   |   |   |
|-----------|----|---------------------------------------------------------|---|----|----|----|--------------------------------------------------------------------------------------|---|---|---|---|
|           | AH | AL                                                      |   | AH | AL |    |                                                                                      |   |   |   |   |
| OR AL, AH | AX | <table border="1"><tr><td>A</td><td>C</td></tr></table> | A | C  |    | AX | <table border="1"><tr><td>A</td><td>C</td></tr><tr><td>B</td><td>D</td></tr></table> | A | C | B | D |
| A         | C  |                                                         |   |    |    |    |                                                                                      |   |   |   |   |
| A         | C  |                                                         |   |    |    |    |                                                                                      |   |   |   |   |
| B         | D  |                                                         |   |    |    |    |                                                                                      |   |   |   |   |
|           | AL | → 0011 0101                                             |   |    |    |    |                                                                                      |   |   |   |   |
|           | AH | → <u>1010 1100</u>                                      |   |    |    |    |                                                                                      |   |   |   |   |
|           |    | 1011 1101                                               |   |    |    |    |                                                                                      |   |   |   |   |

El procesador va a tomar los datos y los pasa a binario. Luego hace la OR, haciendo una suma lógica entre los dos números binarios. Recordemos que la suma lógica no es lo mismo que suma binaria, eso explica que hacer 1+1 no da 10 sino 1, porque si ya estás en el estado lógico 1, según el álgebra de Boole, el que le sigas sumando tensiones 1 no va a modificar nada. Entonces al final el resultado de la suma lógica se coloca en AL.

### Modo de direccionamiento INMEDIATO

El dato a operar está contenido en la instrucción inmediatamente después del opcode de la instrucción.

Ejemplo:

Ejemplo:

MOV AX, 1234

Antes

AH

AL

AX

0 0

0 0

Después

AH

AL

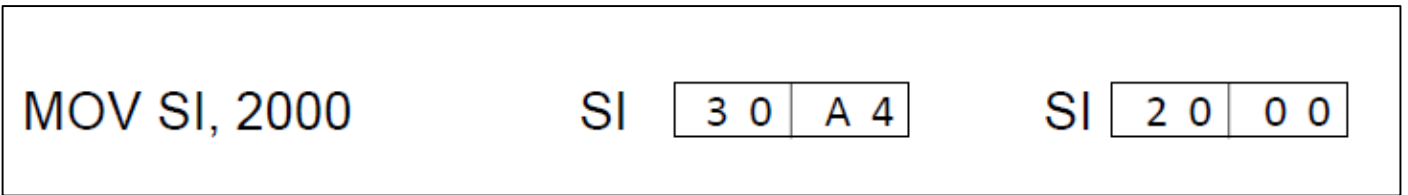
AX

1 2

3 4

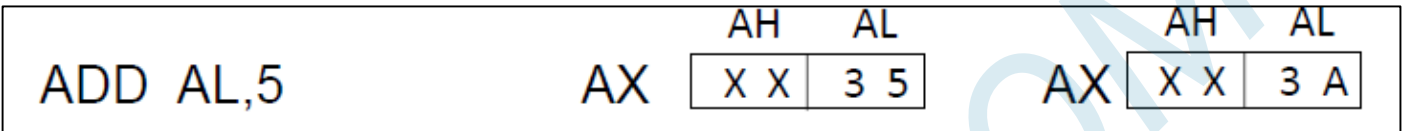
La misma instrucción, que se llama MOV, el destino va a ser en el registro AX, pero fíjate lo que dice ahí, un número le puso el tipo 1234, le pasa el valor que él quiere directamente al registro. Directamente le pones el número que le quieres pasar, te tienes que fijar que la cantidad que le quieras introducir coincida con el tamaño del registro.

Otro ejemplo es el siguiente:



Acá en vez de poner un registro puse un puntero, si yo pongo a la izquierda el puntero y a la derecha 2000 y supongamos que el puntero tenga un numero cualquiera después de ejecutar se va a cargar con el número 2000, o sea que va a ir a la posición 2000 dentro de la memoria RAM.

Otro ejemplo es el siguiente:

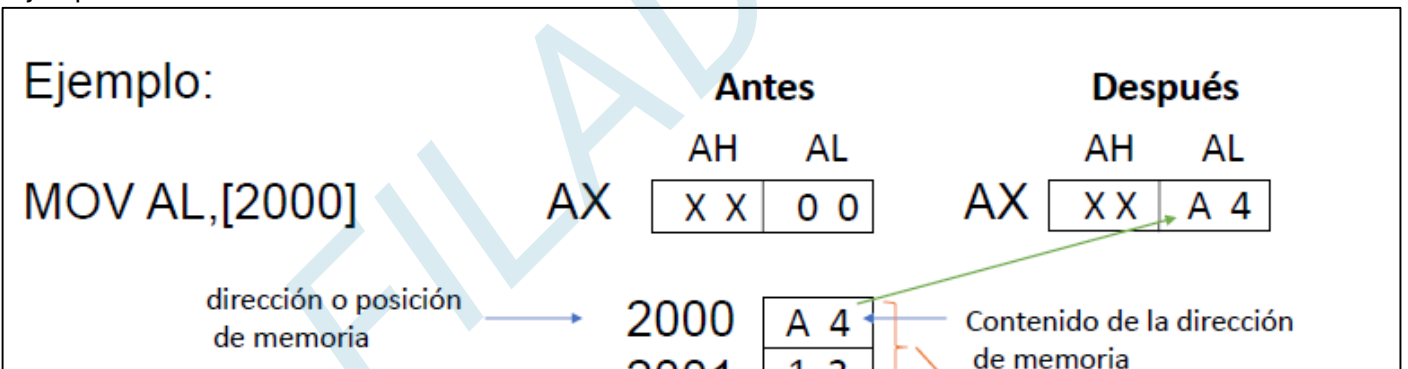


Esta instrucción lo que hace es sumar aritméticamente, dice que súmele 5 al registro AL, es una forma inmediata porque el numero 5 ya estaba definido. Si el AL tiene 35, haces  $35 + 5 = 3A$  porque  $5 + 5 = 10 = A$  en hexadecimal, porque los números en los registros las personas lo usan en hexadecimal porque es una conversión más eficaz para usar ya que todo son unos y ceros.

### Modo de direccionamiento DIRECTO MEMORIA

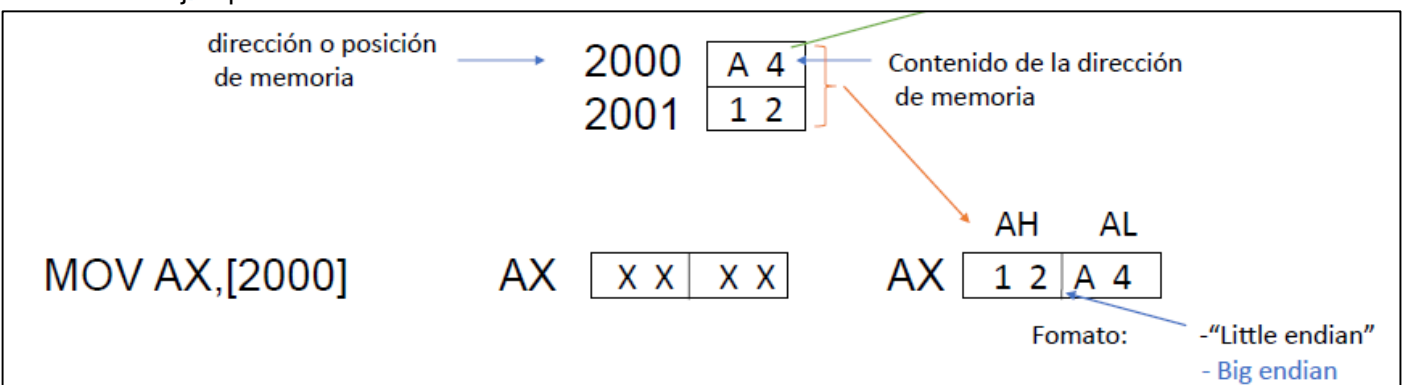
Este es un modo que se usa para que se vaya a la memoria a buscar un dato y que luego se ponga en un registro interno del computador.

Ejemplo:



La instrucción de nuevo es MOV, porque estas queriendo ir a buscar a la memoria un dato y lo quieres poner en el AL. Entonces metes entre corchetes, ojota, siempre entre corchetes siempre indicas un LUGAR de memoria donde está el dato, no es el dato en sí, sino donde se ubica, se le conoce como dirección o posición de memoria. Entonces lo que hace es ir a la posición citada, toma el dato y lo reemplaza en el lugar destino que este caso es en AL. Siempre MOV hace una copia del valor que usaste, pero se pierde el que estaba antes de reemplazar.

Veamos un ejemplo más con una variación:



Ahora la instrucción es la misma pero el registro es más grande, es AX, pero yo hice lo mismo. Te vas a la posición de memoria 2000 y de ahí para abajo, o sea 2001, 2002... agarras tantos bytes como tamaño sea necesario para llenar el registro. En este caso tomas el que indicaste y el siguiente para poder llenar el registro de 16. El orden para acomodar los valores es que como la dirección 2000 es más pequeño que 2001 que es la posición que le sigue va en la parte baja, y como la posición 2001 es más grande va en la parte alta. Este formato en Intel se lo conoce como "Little Endian". Pero también pudo haberse hecho lo opuesto en cuanto al acomodamiento, lo que se conoce como "Big Endian".

### Modo de direccionamiento INDIRECTO MEMORIA

El operando es especificado mediante una combinación de registros índice y base, y puede haber un desplazamiento.

Ej:

|             |    | Antes   |     | Después |     |
|-------------|----|---------|-----|---------|-----|
|             |    | AH      | AL  | AH      | AL  |
| MOV AL,[SI] | AX | X X     | 2 B | XX      | A 4 |
|             | SI | 2 0 0 0 |     |         |     |

|    |   |      |     |
|----|---|------|-----|
| SI | → | 2000 | A 4 |
|    |   | 2001 | 3 6 |

También hay corchetes, en este caso va a ir a buscar a la memoria, la instrucción es MOV, pero la diferencia es que adentro del corchete no tenes un números, sino simplemente el puntero "SI", entonces vos podés mirar en qué valor está posicionado el puntero y extraer los datos que tenga esa posición de memoria. Cuando ejecutes la instrucción, en forma indirecta, porque hay un intermediario que es el registro de por medio, te va a decir dónde está el dato u operando que es el número A4, te va a dar el valor de la dirección de memoria. Entonces el numero 2B se cambia por A4.

Los únicos registros que pueden estar dentro del corchete son el SI, el DI y el BX, no puede ir ningún otro (obvio que si pueden ir números como habíamos visto antes).

Ahora supongamos tenemos lo siguiente:

|                   |    | Antes   |     | Después |     |
|-------------------|----|---------|-----|---------|-----|
|                   |    | AH      | AL  | AH      | AL  |
| MOV AL,[SI+BX +3] | AX | X X     | 3 F | XX      | 2 5 |
|                   | SI | 2 0 0 0 |     |         |     |
|                   | BX | 1 0 0 0 |     |         |     |

|           |   |   |      |     |
|-----------|---|---|------|-----|
| 2 0 0 0   | } | → | 3003 | 2 5 |
| + 1 0 0 0 |   |   |      |     |
| 3         |   |   |      |     |
| 3 0 0 3   |   |   | 3004 | 3 6 |

Esta instrucción dice que vas a ir a buscar una memoria de una forma muy sofisticada porque estas usando 2 punteros. La dirección se obtiene fijándonos en donde están los punteros y después sumas esos valores, cuando sumas todo te das 3003 y buscas dicho lugar en esa memoria. Sacas el 25 de ahí adentro y lo pones en el AL después.

### Modo de direccionamiento RELATIVO

El modo direccionamiento relativo hay varios modos relativos, pero solamente se mencionara uno, que es el relativo al contador del programa. (IP Instruction Pointer)

Cuando vos empezas a ejecutar un programa, tenes una instrucción debajo de la otra, el modo relativo lo usas para saltar, para cuando vos tenes que quebrar o romper el orden normal de ejecución y hacer un lazo por ejemplo, entonces el contador del programa va a empezar a dar vueltas en un bucle cerrado.

Veamos un ejemplo:

|           |          |                                                   |
|-----------|----------|---------------------------------------------------|
| IP → 0100 | CMP AL,3 | Realiza AL-03 pero sólo fija las banderas (Z,N..) |
| 0102      | JZ 0120  | Si da cero, es decir AL=3, el IP salta a 0120     |
| 0106      | —        | } Si no, sigue y ejecuta estas instrucciones      |
| 0108      | —        |                                                   |
| :         | :        |                                                   |
| :         | :        |                                                   |
| 0120      | —        |                                                   |

CMP es la instrucción Compare que ahí vemos lo que hace (hace una resta ficticia solo para comparar valores pero sin modificar el valor que tiene en este caso la AL) y si esa resta da un numero 0, entonces se activa la bandera Z, caso contrario la bandera N.

La instrucción de este modo es la que sigue, la que dice JZ, siendo J una letra que deriva de la palabra Jump (Saltar) con la condición Z que habíamos mencionado anteriormente. Le dice saltá si es 0, y al lado tiene la instrucción a la cual se irá, entonces esto es el modo relativo al valor donde estaba antes ahora se corre a la 0120 y ejecutará la instrucción que allí se encuentre. Si no es cierta la condición sigo normalmente el curso de las instrucciones.

### JNE o JNZ salta si no es igual o no es cero

|     |                               |               |
|-----|-------------------------------|---------------|
| JG  | salto si es mayor             | } (con signo) |
| JL  | salto si es menor             |               |
| JGE | salto si es mayor o igual     |               |
| JLE | salto si es menor o igual     |               |
| JA  | salto si es mayor (sin signo) |               |
| JC  | salto si hay acarreo.... etc  |               |

Quedaron pendientes algunos registros que no fueron explicados antes:

|           |
|-----------|
| <b>CS</b> |
| <b>DS</b> |
| <b>ES</b> |
| <b>SS</b> |

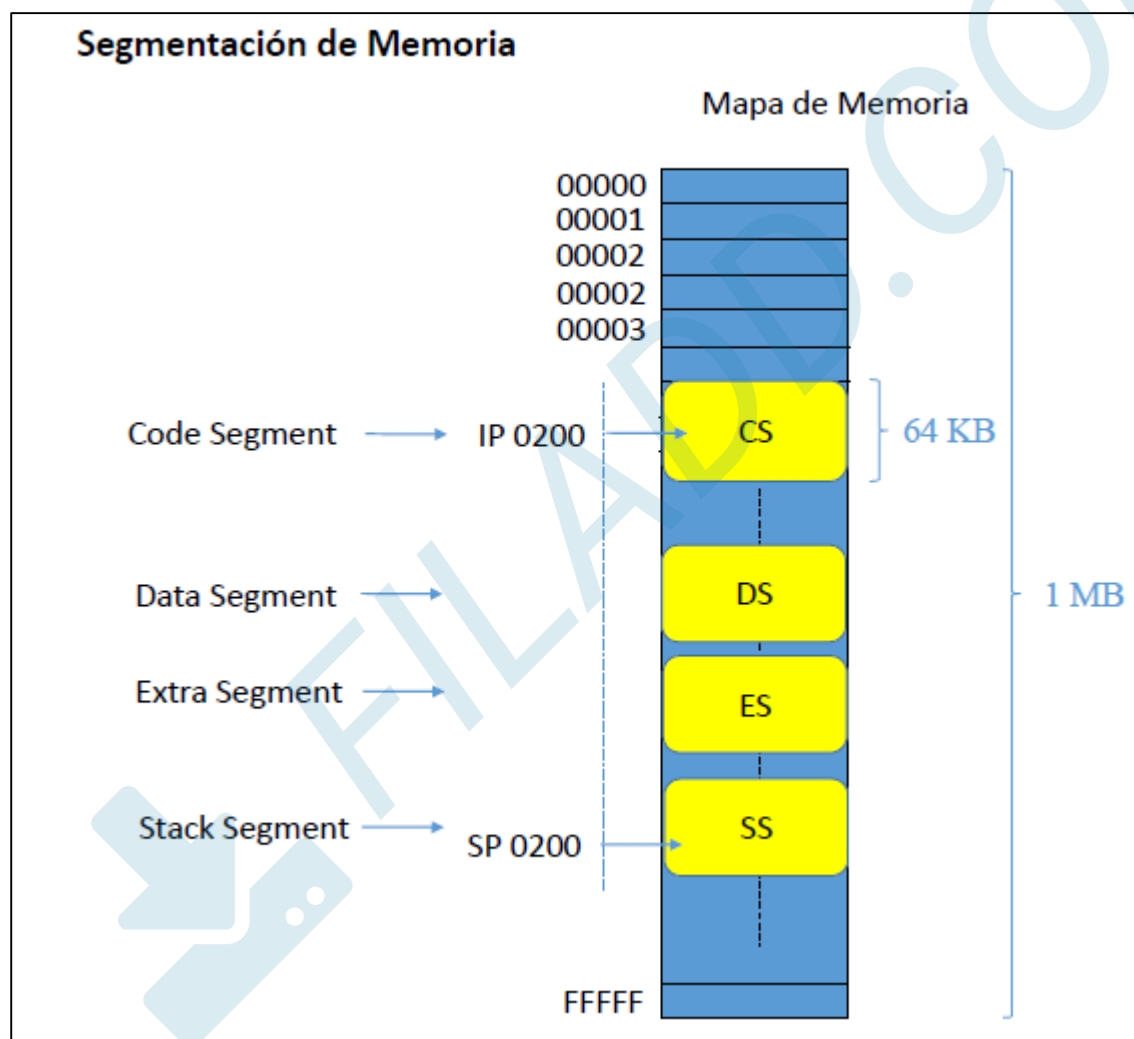
El primero se llama Code Segment (Segmento destinado al código)

El segundo se llama Data Segment (Segmento destinado a los datos)

El tercero se llama Extra Segment (También dedicado a los datos)

Stack Segment (Segmento de la pila)

Del mapa de memoria que tenemos para manejar los distintos bytes y suponiendo que todos los mapas están llenos de memoria, Intel crea 4 zonas en las cuales van a estar contenidos los registros mencionados anteriormente.



La primer zona (Segmento de código) está contenido una porción de programa que se está ejecutando en este momento. O sea, Intel dijo: "Si vas a ejecutar programas, asegúrate de que esa parte del programa que estas ejecutando este dentro de este segmento". Segmento de código, es decir, de instrucciones. Este segmento lo mueve el usuario por Software, vos lo vas posicionando en el lugar donde o vas a colocar una rutina para el código o estás ejecutando un programa y entonces el segmento de código debe estar posicionado ahí. En el caso que trabajes en lenguaje de máquina, sos vos el que posicionas el segmento en el lugar donde esté el trabajo.

Como este programa del que estábamos hablando anteriormente también va a acceder a datos, entonces vos también en la memoria vas a ir a buscar datos para llevar adentro de los registros o de los registros los llevas a la memoria. Por eso Intel creo otra zona que se la llama Segmento de datos y ahí los vas a guardar.

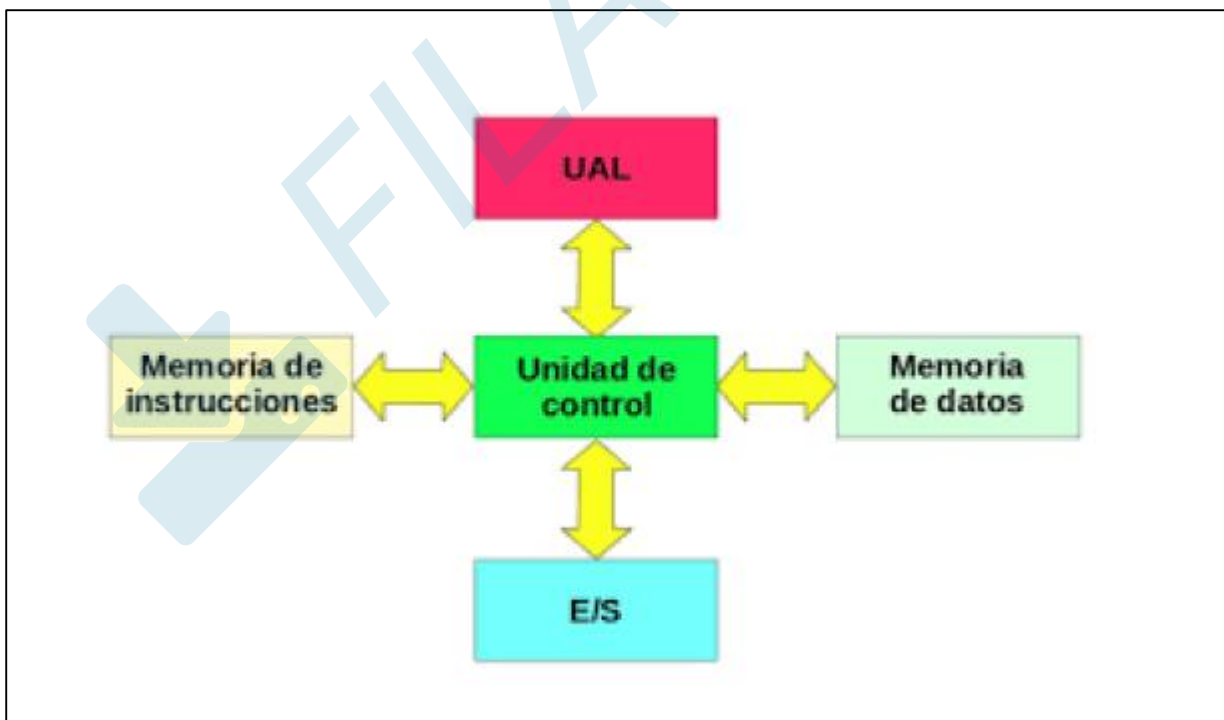
Pero como a veces hay que manipular datos y llevarlos de una zona de memoria hasta otra, sería muy retardado que vos fijes la zona de trabajo para poder buscar un dato y llevarlo a un registro y luego tengo que fijar ese segmento en un lugar alejado de la memoria para poder transportar ese dato al lugar más lejano. Entonces previendo eso Intel colocó otro segmento más llamado segmento extra, que también sirve para el movimiento de datos. Entonces si vos querés mover un bloque o una cadena de caracteres que se encuentra en una zona de memoria a otra zona más alejada fijarás un segmento en la zona de origen y después otro segmento en la zona de destino y así trasladarás bloques de un tamaño máximo de 64KB.

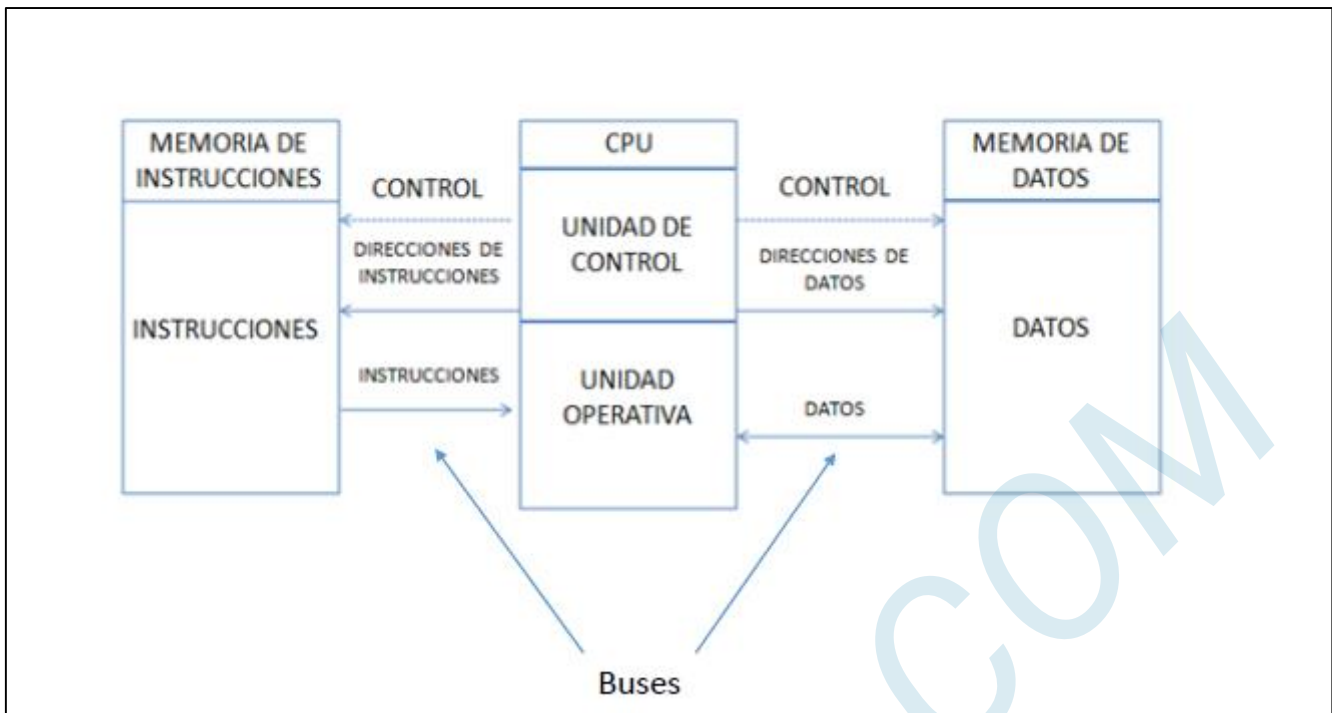
El último lugar corresponde a la pila donde se van almacenando los datos esenciales del funcionamiento del micro cuando se salta a subrutinas o se salta a interrupciones. La pila como habíamos dicho es una memoria tipo LIFO (Última entrar-primero en salir) el puntero que va llevando la cuenta de cuál es la última que entro es el stack pointer y la maneja el microprocesador, el puntero baja o sube según lo que ocurre en el programa y regresará cuando hay instrucciones de retorno. El puntero trabaja en el Stack Segment.

### Arquitectura Harvard

La arquitectura de Von Neumann en la memoria principal colocaba los programas y los datos, provocando un cuello de botella en el bus de datos. En la RAM había programas y datos, cuando había que llevar una nueva instrucción, la misma viajaba por el bus de datos, y se posicionaba en el registro de instrucción, pero cuando ibas a buscar un carácter o un número para procesar (dato) también viajaba por el mismo camino, pero el destino era distinto e iba a parar dentro de uno de los registros de propósito general del procesador. Entonces se forma un cuello de botella porque inmediatamente después de hacer esa operación necesitas una instrucción nueva, devuelta a venir por el mismo camino. Por tanto surgió la arquitectura Harvard que viene de la universidad de EEUU.

Propone que se separe a la memoria en dos, y separar a los buses en dos, uno para memoria y otro para instrucciones. Entonces se pueden estar trayendo nuevas instrucciones a la unidad de control para que las ejecute, al mismo tiempo que datos que están en la memoria podrían estar siendo almacenado en los registros para que después la ALU (Unidad Aritmética Lógica) haga algunas cuentas.





Así entonces la arquitectura HARVARD posee un bus de control, de instrucciones y bus de datos para las instrucciones y bus de control, bus de datos, bus dirección para la memoria de datos.

### Clasificación de instrucciones en lenguaje de máquina

#### **Instrucciones de transferencia**

Copian en el destino que es un registro o posición de memoria la información del operando fuente sin modificar éste.

Cuando se hace un movimiento, se usa una instrucción de esa, no se modifica el estado de la bandera. Solo se modifica cuando sea una operación sobre la ALU, una suma o resta, si moves datos no alteras banderas.

Generalmente transfieren palabras pero pueden mover fracciones de ellas o bloques enteros.

Ejemplos:

Mov: es por excelencia la instrucción que transfiere el operando fuente al destino.

Push: transfiere el operando fuente a la pila.

Pop: transfiere el último dato de la pila al operando destino.

#### **Instrucciones aritméticas**

Dentro de las instrucciones de este tipo tenemos:

ADD: suma sin acarreo.

IDIV: división con signo.

ADC: suma con acarreo.

INC: incrementar.

SUB: resta sin acarreo.

Dec: decrementar.

SBB: resta con acarreo.

NEG: cambia de signo dejando el operando en C2

MUL: multiplicación sin signo.

IMUL: multiplicación con signo.

DIV: división sin signo.

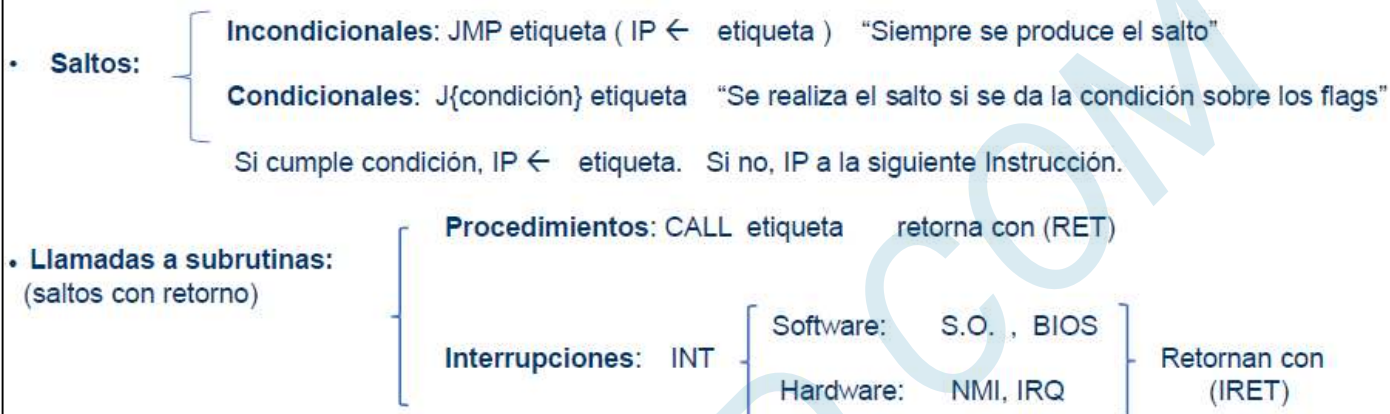
**Instrucciones lógicas:**

AND, NOT, OR, XOR.

**Instrucciones de bifurcación**

Modifican la secuencia normal de ejecución de un programa.

Actúan sobre el contador de programa (PC o IP), controlan la secuencia de ejecución de un programa.

**Clasificación:****Instrucciones de comparación:**

No almacenan el resultado, solo modifican los flags. Fueron creadas para crear modificaciones en las banderas y permitir el salto a las instrucciones de salto.

La más famosa es CMP (Compare). Compara números (resta pero no guarda el resultado, solamente modifica los flags de estado) no guarda el resultado. Tanto SUB como CMP modifican el estado de la bandera.

**Instrucciones de bit**

Te permiten prender o apagar una de las banderas por ejemplo simplemente. Tienen la función de modificar un solo bit o leer su estado. Son típicas en la configuración de los flags. Trabajan sobre un punto, sobre un bit, por ejemplo en un registro de estado.

Ejemplo:

TEST: comparación lógica a nivel de bits

CLI: clear flag de interrupción

STI: set flag de interrupción

CLC: clear flag de acarreo

STC: set flag de acarreo.

## Instrucciones de desplazamiento

Todas las instrucciones de desplazamiento y rotaciones tienen en común que el último valor desplazado se copia en el flag de acarreo.

SAR: desplazamiento aritmético a la derecha.

SAL: desplazamiento aritmético a la izquierda.

SHL: desplazamiento lógico a la derecha.

SHL: desplazamiento lógico a la izquierda.

ROR: rotación a la derecha.

ROL: rotación a la izquierda.

RCR: rotación a la derecha a través del acarreo.

RCL: rotación de la izquierda a través del acarreo.

## Instrucciones de entrada / salida (E/S)

IN: transfiere información desde un puerto de entrada a un registro.

OUT: escribe información en un puerto de salida desde un registro.

## Instrucciones de control

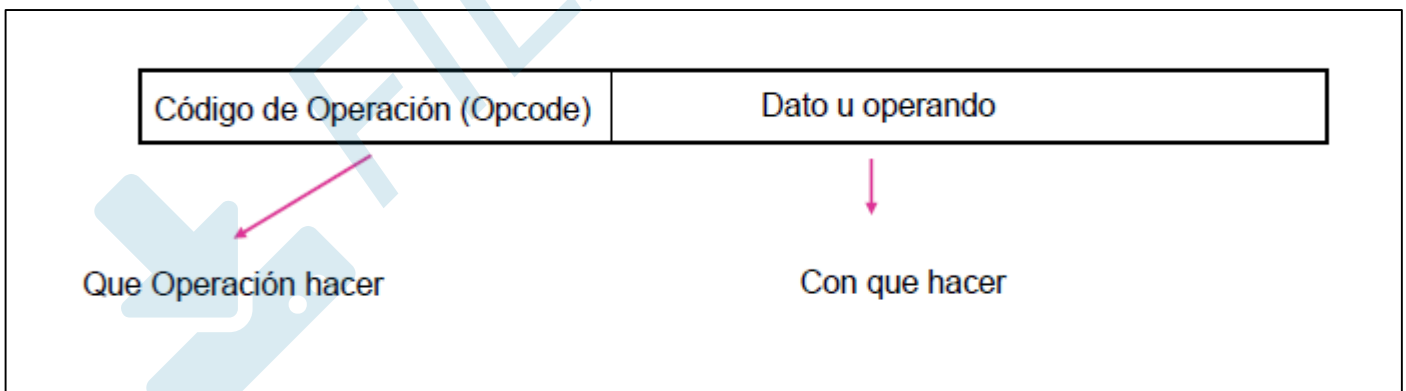
WAIT: hace esperar al procesador

HTL: detiene el procesador

NOP: no operation

## Formato de Instrucción INTEL

Una instrucción es una orden que se le da a un procesador, la misma tiene dos partes:

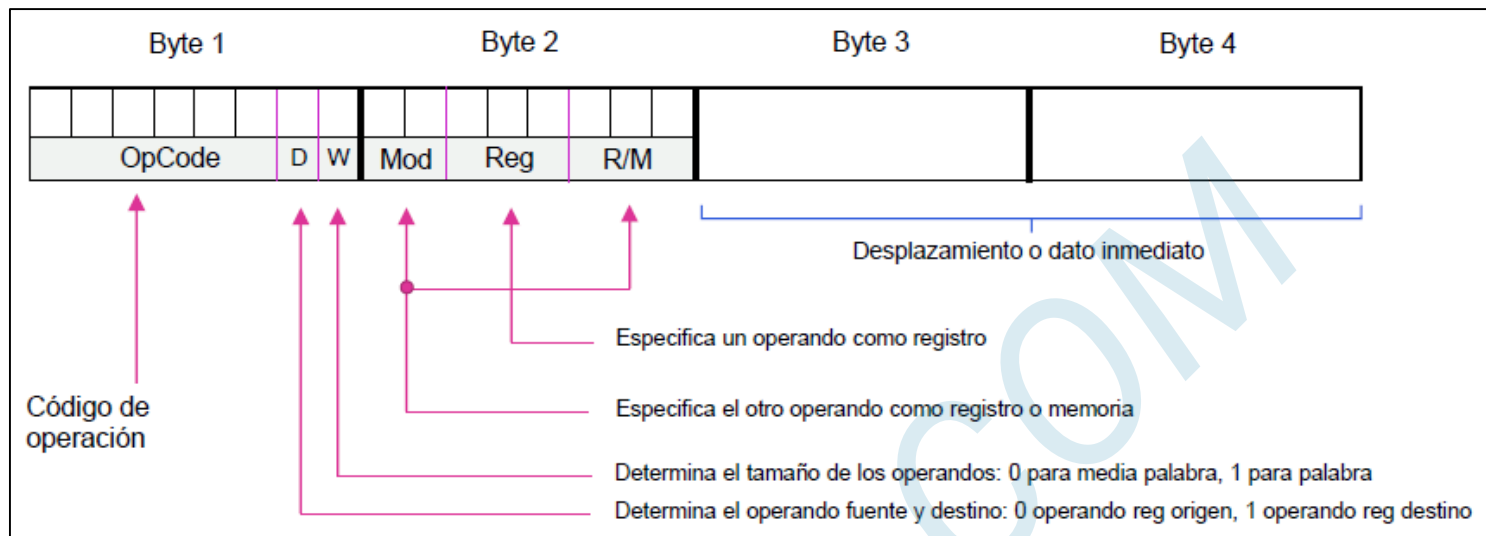


Este procesador es del tipo registro memoria, por lo que no puede trabajar, con dos operandos en memoria

Esto hace que la mayoría de las instrucciones tengan un bit que indique quién es el operando fuente: el registro o el operando en memoria.

Formato general de las instrucciones de Intel x86

Hay múltiples formatos cuyo código va desde un byte hasta seis bytes.

**Formato de las instrucciones registro-registro y registro- memoria**

En el primer byte tenemos el código de operación, Intel dirá cuáles son los 1's y 0's para que sea la instrucción MOV por ejemplo. Después hay un bit (D) que determina quién es el operando fuente y destino, si le pones un 0 el operación está en el registro origen, y si le pones un 1 el operando está en el registro destino. Supongamos que elegís un registro, con D le decís si quieres que ese registro sea fuente o destino, que este a la izquierda o derecha de la coma.

El otro bit (W) determina el tamaño de los operandos, si le pones 0 es media palabra o es un byte, y si le pones 1 es para la palabra o sea 2 bytes. El 1 es palabra o Word y si es 0 es un byte.

Después viene el modo (MOD) que especifica otro operando como registro o memoria. De acuerdo a la combinación de bits que son cuatro te va a determinar si el otro operando se encuentra en un registro o en la memoria. En definitiva define el modo de direccionamiento.

Luego REG es el registro fundamental que siempre tiene que existir y de acuerdo al código binario especifica un operando como registro. Tiene una combinación binaria y dice a qué registro pertenece.

Una vez que definiste el primer registro, el otro puede ser otro registro, entonces pondrás a MOD en forma de registro y en R/M escribirás el nombre del otro registro.

En los otros dos bytes pertenecen al desplazamiento en el caso que en vez de elegir registro elijas memoria, entonces vas a tener que escribir los números en uno o en los dos bytes para poder especificar cuál es la dirección de la memoria donde querés ir a buscar el dato.

Ahora veremos en detalle (y volviendo a repasar lo que contiene cada byte):

## El primer byte contiene:

- Código de operación MOV Destino, Fuente
- El bit de dirección de registro (D):
  - Si D = 1 tengo que REG = operando destino
  - Si D = 0 tengo que REG = operando fuente
- El bit de tamaño del dato (W): especifica si la operación será realizada sobre datos de media palabra o de una palabra:
  - Si W = 0 los datos son de 8 bits
  - Si W = 1 los datos son de 16 bits

## El segundo byte contiene los operandos (uno de ellos es un registro):

- **REG** se usa para identificar un registro
- **MOD** indica el modo de direccionamiento

| REG | W=0 | W=1 |
|-----|-----|-----|
| 000 | AL  | AX  |
| 001 | CL  | CX  |
| 010 | DL  | DX  |
| 011 | BL  | BX  |
| 100 | AH  | SP  |
| 101 | CH  | BP  |
| 110 | DH  | SI  |
| 111 | BH  | DI  |

### MOD Función:

|    |                                                  |
|----|--------------------------------------------------|
| 00 | Modo memoria sin desplazamiento                  |
| 01 | Modo memoria con desplazamiento de media palabra |
| 10 | Modo memoria con desplazamiento de una palabra   |
| 11 | Modo registro                                    |

## El segundo byte contiene los operandos (continuación):

- R/M** se usa para identificar un registro o una posición de memoria y depende del valor del campo MOD

| MOD = 11 |     |     | CALCULO DE LA DIRECCION EFECTIVA |                   |              |               |
|----------|-----|-----|----------------------------------|-------------------|--------------|---------------|
| R/M      | W=0 | W=1 | R/M                              | MOD = 00          | MOD = 01     | MOD = 10      |
| 000      | AL  | AX  | 000                              | [BX]+[SI]         | [BX]+[SI]+D8 | [BX]+[SI]+D16 |
| 001      | CL  | CX  | 001                              | [BX]+[DI]         | [BX]+[DI]+D8 | [BX]+[DI]+D16 |
| 010      | DL  | DX  | 010                              | [BP]+[SI]         | [BP]+[SI]+D8 | [BP]+[SI]+D16 |
| 011      | BL  | BX  | 011                              | [BP]+[DI]         | [BP]+[DI]+D8 | [BP]+[DI]+D16 |
| 100      | AH  | SP  | 100                              | [SI]              | [SI]+D8      | [SI]+D16      |
| 101      | CH  | BP  | 101                              | [DI]              | [DI]+D8      | [DI]+D16      |
| 110      | DH  | SI  | 110                              | direccion directa | [BP]+D8      | [BP]+D16      |
| 111      | BH  | DI  | 111                              | [BX]              | [BX]+D8      | [BX]+D16      |

- D8 es un desplazamiento de tamaño media palabra (8 bits) y D16 es un desplazamiento de tamaño palabra (16 bits)

Ejemplo:

La instrucción MOV BL, AL "mueve el byte contenido en el registro fuente AL al registro destino BL".

¿Cuál será el código de máquina de esta instrucción?

Primero hay que preguntarle a Intel cual es el código de operación de la instrucción MOV, que son los primeros bits, el código de operación es: 100010.

Rellenamos:



El bit W debe indicar una operación de tamaño byte. Por esta razón su valor será 0. BL es un registro de 8 bits.

El bit D indica si el registro que señala el campo REG del segundo byte es el operando fuente o el destino. En este caso se codificará el registro BL en el campo REG del segundo byte, por tanto, D será igual a 1. Tengo que poner a BL (011 su número en Intel) como destino.

Rellenamos lo que dijimos:



Como el segundo operando también es un registro en Mod se pone 11 entonces.

En el campo R/M debe especificar que el registro es AL por tanto R/M = 000.

Queda todo así:

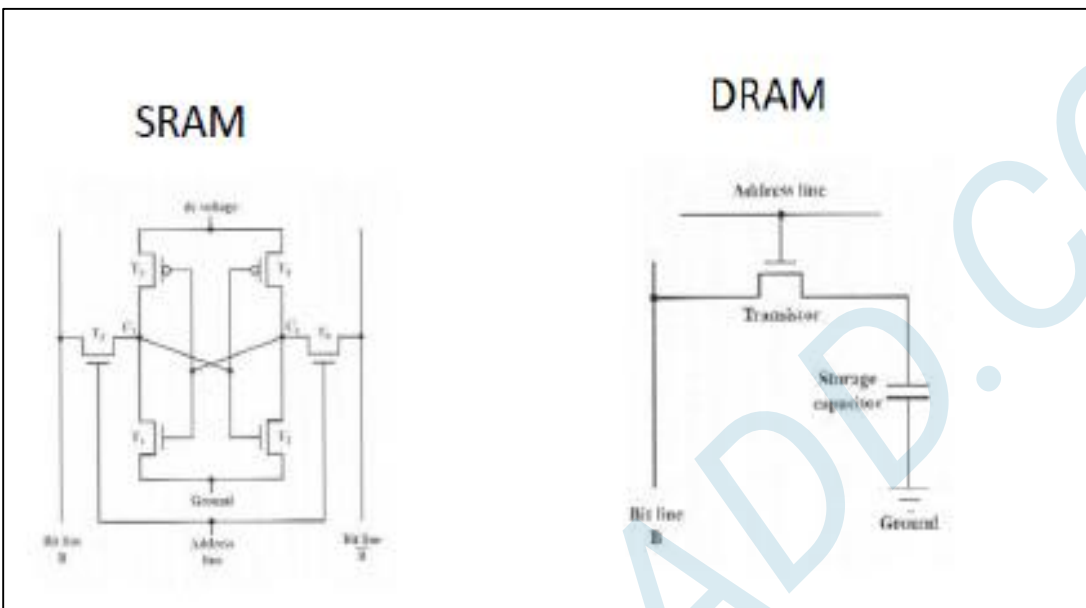
| Byte 1 |   |   |   |   |   | Byte 2 |   |     |   |     |   |
|--------|---|---|---|---|---|--------|---|-----|---|-----|---|
| 1      | 0 | 0 | 0 | 1 | 0 | 1      | 1 | 0   | 1 | 1   | 0 |
| OpCode |   |   |   |   |   | D/W    |   | Mod |   | Reg |   |

## Memorias

### Memorias RAM

Tenemos dos formas básicas de memorias semiconductoras que son de acceso aleatorio, son las RAM dinámicas (DRAM) y las RAM estáticas (SRAM). La SRAM son más rápidas, más caras y de menor capacidad que las DRAM, y son usadas para la memoria caché. Las DRAM son usadas para la memoria principal.

Las RAM son volátiles porque en ausencia de energía eléctrica se pierde su contenido, son de acceso aleatorio porque el tiempo que tarda en acceder desde el primer byte al último o uno del medio es siempre el mismo.



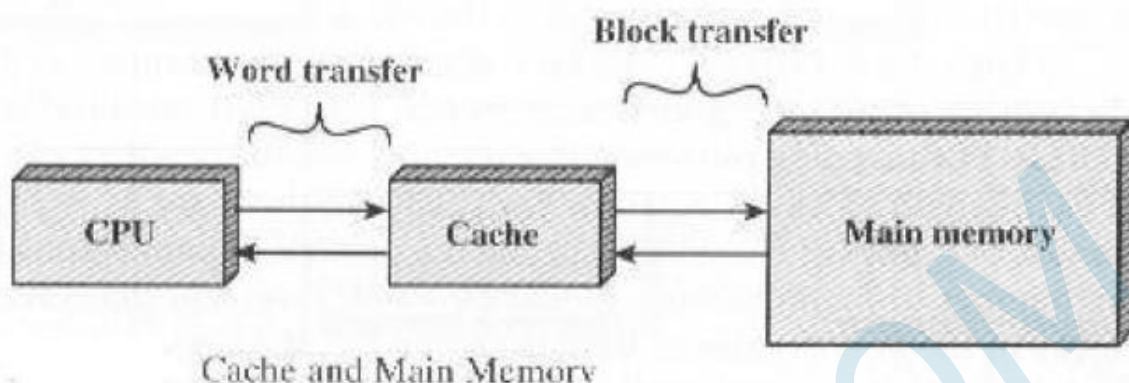
Tenemos una clasificación de dos categorías, las estáticas y dinámicas. Las estáticas son el punto de memoria, se construye con 6 transistores, esto hace que este punto de memoria sea costoso en términos de área de silicio, porque cuando pongamos muchos bytes, o sea, muchos de estos puntos de memoria uno al lado del otro van a ocupar mucho espacio. En cambio la RAM dinámica se la forma con un solo transistor y un capacitor.

La SRAM se coloca en la memoria cache, que está adentro del procesador, es costosa, ocupa mucho espacio por tanto vamos a hacer chips menos densos, con menor capacidad de memoria pero es la más rápida de todas y es ahí donde se van a ejecutar las instrucciones y los datos más utilizados en el momento por el procesador.

El espacio que ocupa el punto de memoria, o sea el lugar donde se va a guardar un uno o un cero en la DRAM es extremadamente pequeño al lado de la estática, pero su desventaja es que en este capacitor el 1 y el 0 se almacenan en función de la carga que está almacenada adentro del capacitor. Si el capacitor tiene cargas, diremos que este punto tiene un 1 lógico y si el capacitor lo han dejado descargado se dice que hay un 0 lógico. El inconveniente es que el capacitor va a perder su carga, hay pérdidas de carga, el nivel va bajando y entonces corremos riesgo. La carga se va perdiendo aun con fuente de energía, lo que tienen que hacer es aplicar un pulso de refresco para recuperar cada tanto capacitores que están cargados y volverlos a recargar para evitar que esto pierda. El capacitor es como un vaso de agua que está pinchado, vos lo llenaste y decís "ahí hay uno", pero pasa el tiempo se te empieza a vaciar. Entonces tiene que venir el pulso de refresco para reponer las cargas que están faltando para que no se pierda ese uno, pero esa acción hace que se pierda tiempo, lo que la hace más lenta frente a la estática que no tiene dicho problema, pero es más barata. Esta es pequeña, por tanto podemos hacer chips con cantidad masiva de memoria. Se usa en la memoria principal.

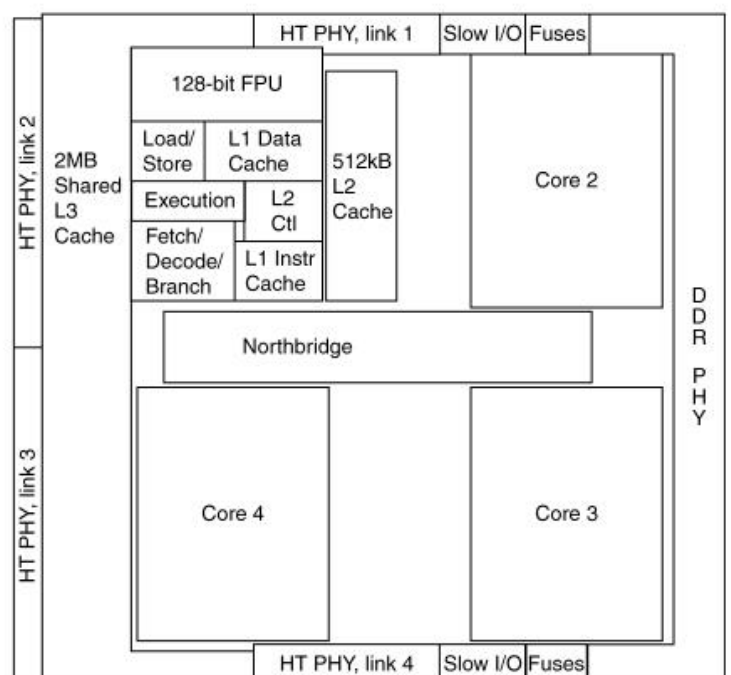
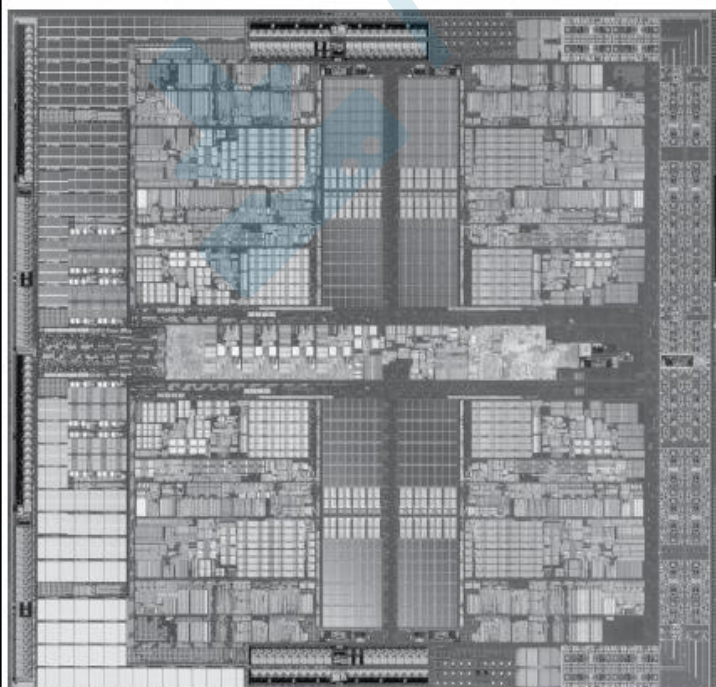
### Memoria Caché

La memoria cache es una solución de compromiso entre la memoria lenta que esta allá en la memoria principal de la maquina hecha de RAM dinámica y una memoria rápida como lo es la RAM estática, esta puesta en el medio del camino. Está destinada a proporcionar una velocidad de memoria que se acerque a la de las memorias más rápidas disponibles y al mismo tiempo proporcionar un tamaño de memoria grande al precio de memorias menos costosas.



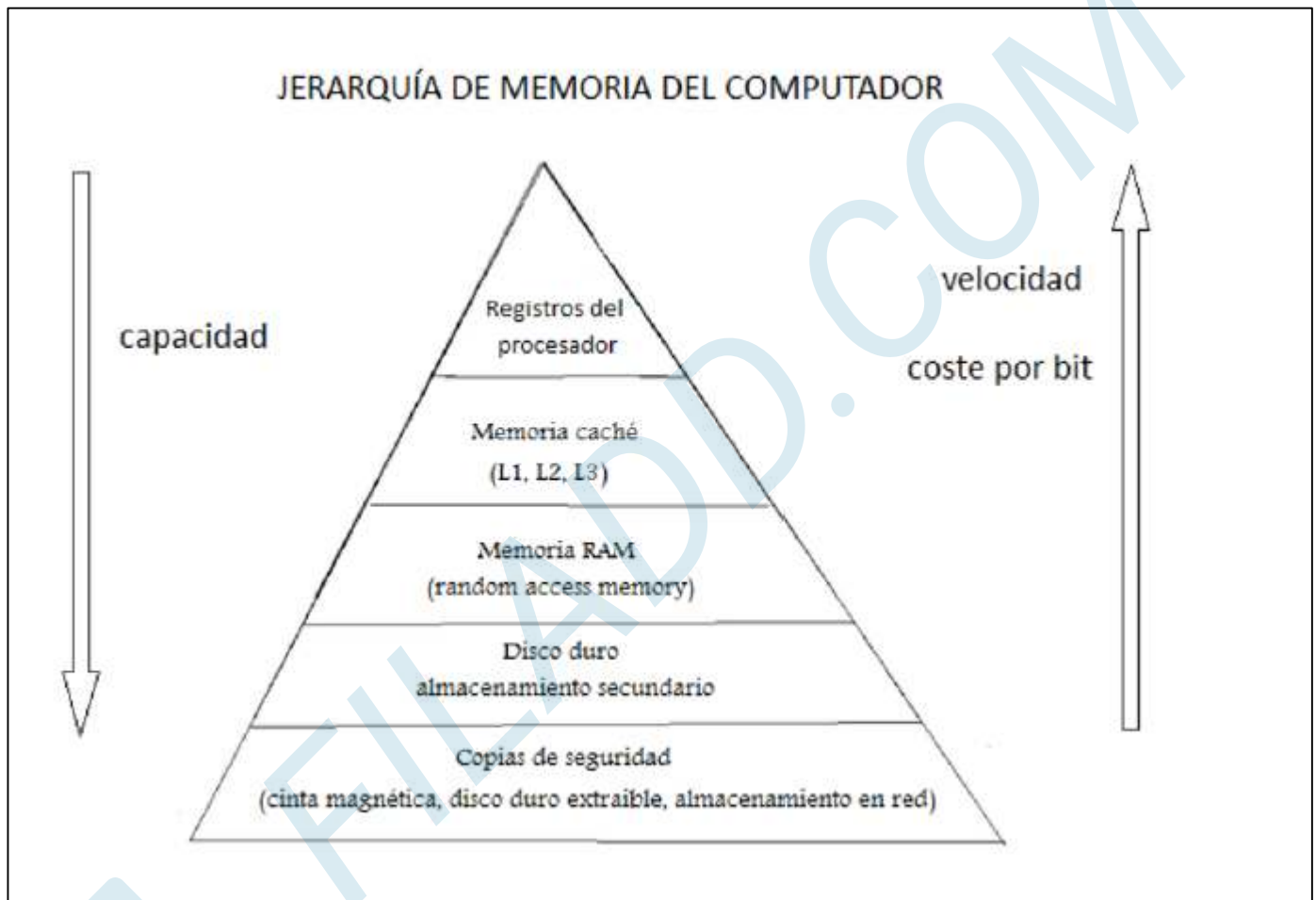
La memoria cache le transfiere a la CPU palabras, por ejemplo, hay dos niveles de caché, una de instrucciones y otra de datos, van las instrucciones que se están usando una a una, van pasando a la CPU y también van pasando de la caché de datos, los datos que son más usados por la ALU en este momento. Son palabras las que van desde la caché a la CPU. Sin embargo, de la memoria principal se transfiere de a bloques, es decir, si vos estabas ejecutando un programa y dentro de poco vas a seguir con ese programa porque ya ejecutaste todas estas instrucciones, vas a la memoria principal y traes otro bloque y lo almacenás adentro de la caché.

## Niveles de la Cache



Tienen cache de instrucciones y de datos de nivel 1, es la parte más rápido del microprocesador y después tenemos un cache de nivel 2, es un poquito más lenta a pesar de ser más grande. Si la unidad de ejecución está buscando una instrucción en la cache L1, le pasa a la unidad de ejecución y ejecuta. Si no la encuentra busca en el nivel 2 de cache el bloque y trae un bloquecito con las próximas instrucciones de ese programa. Si no llega a estar en el nivel 2, se buscará en el nivel 3 que es compartido por todos los núcleo y si no está acá recurre a la memoria principal, sale afuera del procesador y va hasta a la memoria principal de RAM dinámica, se busca desde ahí el bloque del programa que se está ejecutando y se lo trae de nuevo adentro del procesador y esos bloques se copian en la L2, y finalmente en la L1 y ahí continua la ejecución.

### Jerarquía de memorias de un computador



En el tope de la pirámide están las memorias más rápidas que puede haber adentro de un procesador y la caras que puede tener y después bajando más a la base vamos a encontrar las de mayor capacidad, de almacenamiento masivo y de costo más barato por bit.

Capacidad aumenta hacia abajo y para arriba vamos a tener más velocidad y costo.

En el primer lugar, en el top de la pirámide se encuentran los registros del procesador, es realmente la memoria más rápida, que tiene más al interior, al alcance de la ALU y de la unidad de control, entonces esos registros del procesador serian la sucesión de bit uno al lado de otro y que actualmente esos registros tienen un tamaño de 64 bits, esos son rápidos. Los procesadores de los teléfonos tienen otra tecnología distinta, son procesadores del tipo risc y los diseñadores se basaron en poner cantidad terrible de registros ahí adentro del procesador, eso los vuelve muy rápido en el manejo de datos internos.

En el segundo lugar viene la memoria cache, con sus niveles L1, L2, L3. Le siguen en velocidad y encosto, son un poco más lenta que los registro, pero son rápidas de todas formas.

En el tercer lugar viene la memoria RAM dinámica, son las memorias de la memoria principal del procesador. Es la más grande dentro de la computadora.

Después viene el disco duro donde ahí tenemos un almacenamiento más masivo, el costo por bit es más barato, se guardan gran cantidad.

En la base tenemos algunos sistemas de almacenamiento de copias de seguridad, como pueden haber sido y todavía se usan las cintas magnéticas, algún disco duro extraíble, almacenamientos en una red o en la nube.

### Memoria principal

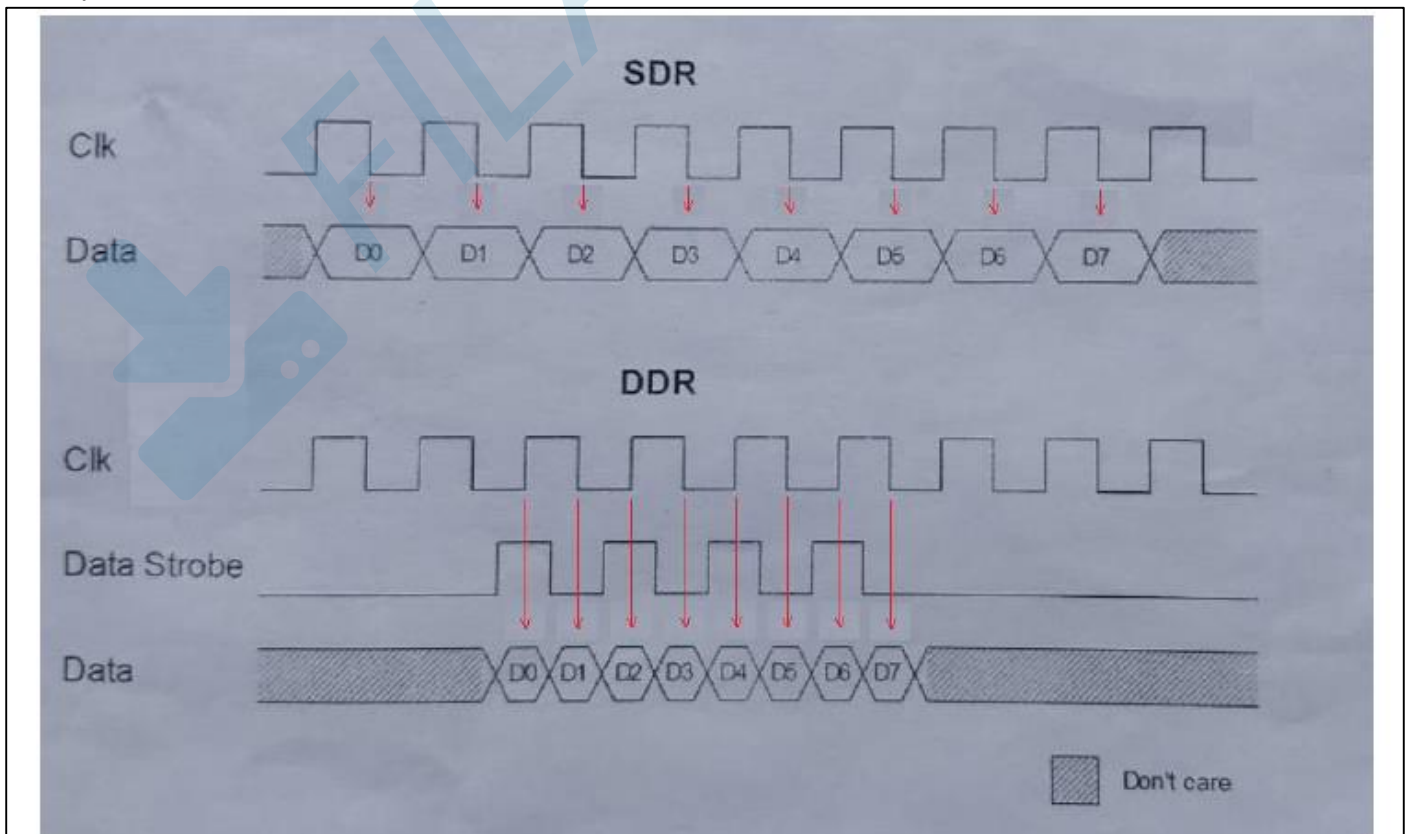


SDRAM (Synchronous Dynamic random Access memory)

También pueden estar especificadas como DIMM dual in-line memory module y eso significa que es un módulo de memoria con contactos duales, al contrario de los SIMM en que cada contacto está unido a su opuesto.

Todas esas chapas doradas establecen una conexión con alguna señal que se quiere llevar desde la memoria hacia la placa madre o procesador. En la foto se ven la línea de chapas y por debajo si la agarramos y damos vuelta a la memoria hay otra línea de chapas. En las primeras memorias cada chapita superior e inferior que estaban alineadas eran la misma cosa. Cada señal que aparecían arriba y abajo se trataban de la misma señal

porque estaban unidas. En cambio cuando apareció la DIMM pusieron en las chapas superiores un tipo de señal y en la de abajo otras, por eso Dual in line memory, porque eran doble en línea memoria, y este concepto están en todas en la actualidad.



La memoria que estudiamos recién eran del tipo SDR y después aparecieron las DDR. LA diferencia entre S y D es que una quiere decir Simple y la otra Doble. En la simple cada vez que el pulso de reloj cae un dato se transfiere desde la memoria hasta el registro del procesador o viceversa, del procesador a la memoria. Cada vez que otro pulso de memoria cae, acá el dato 1 se transfiere, avanza un tercer pulso y otro dato 2 se transfiere.

En cambio las DDR cuando el pulso de reloj se levanta el dato 0 es transferido, cuando el pulso cae el dato 1 se transfiere. Hace un doble acceso a datos en el tiempo que la otra lo hacía una sola vez, obviamente eso duplica la velocidad. Es de Doble Data Rate. Hay transferencia al doble de velocidad que las anteriores.

**DDR**

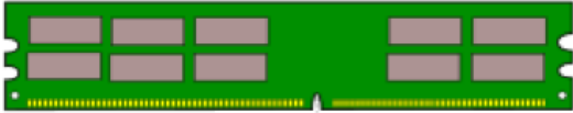


Diagrama de un módulo de memoria DDR, mostrando su forma rectangular y los contactos en la parte inferior.

**DDR :**

**DDR-SDRAM (Double Data Rate Synchronous Dynamic random access memory)** son del mismo tamaño que los DIMM de SDRAM, pero el conector con mas contactos: 184 pines  
Tensión de operación: 2.5V

**DDR 2**



Diagrama de un módulo de memoria DDR2, mostrando su forma rectangular y los contactos en la parte inferior.

**SDRAM DDR2** es la segunda generación de SDRAM DDR.  
Frecuencia 400-1200 MHz

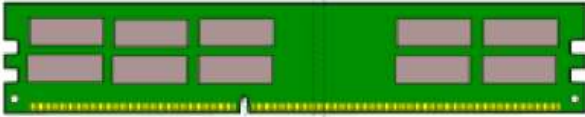
Pueden transferir un volumen de información de 8 bytes (Bus = 64 bits) en cada ciclo de reloj en Dual Chanel (4+4) Bytes.

Menor tensión de operación que DDR-SDRAM se redujo de 2.5V a 1.8V. Esto disminuye el consumo de energía y la generación de calor, así como permitir una mayor densidad de memoria para configuraciones de mayor capacidad.

El número de contactos se elevó de 184 a 240.

Bandwidth: **3,2 - 9,6 GB/s** (en Dual Channel)

Con el tiempo surgen nuevas generaciones de DDR, como la DDR2, la frecuencia es la velocidad con la que se hacen las transferencias, multiplicas la velocidad de la memoria, por la cantidad de bits que vas a transferir y si luego lo dividís por 8 que son los bytes, te podes dar una idea de a qué velocidad se transferirían los datos entre la memoria y el procesador.

**DDR 3****Mejoras****\*Mayor Velocidad**

La reducción de consumo de energía debido a la tecnología de fabricación de 90nm

El voltaje de DDR3 SDRAM DIMM's se redujo de 1.8V a 1.5V.

**DDR3:**

DDR3 SDRAM (Double Data Rate type 3 Synchronous Dynamic Random-Access Memory ) es la tercera generación de SDRAM-DDR

Frecuencia: 1600-2133 MHz

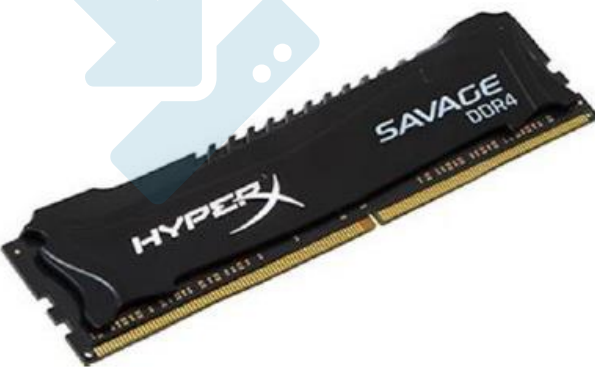
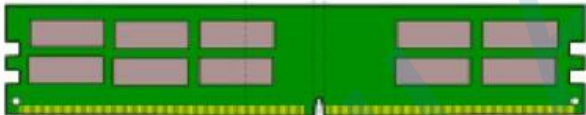
Bus de 64 bits

Las DDR3 tienen 240 pines

Máx. capacidad de transferencia:

Bw = **12.8 - 17GB/s**

Son más rápido, la mejora es que tiene mayor velocidad, siempre con respecto a la anterior, reduce el consumo de energía que es porque al empezar a trabajar más rápido los chips toman mucha temperatura, por eso era indispensable ponerle disipador de calor. Si vos a la transferencia de datos lo multiplicas por 64 bits y lo dividís por 8 te da una máxima capacidad de transferencia de 8,5 a 18 GB/s.

**DDR 4**

**DDR4** : Double Data Rate type 4 Synchronous Dynamic Random-Access Memory)

Bus = 8 Bytes o 64 bits con dual channel se tiene (8+8)=128bits

Transferencias de datos ( 1600 a 3200 MHz

Bw= **25.6 - 51.2 GB/s**

Aunque Corsair presentó una que llega a los 5000 MHz → **80GB/s**

Tensión de alimentación de 1,2 a 1,05 V

Los DDR4 tienen 288 pines

## GDDR5

Son usadas para las GPU y consolas de videojuegos, tienen tamaños de buses de 192-256-385, la capacidad máxima de transferencia es de 224 GB/s. Esta ya es obsoleta.

## GDDR6

Es de Octo Data Rate, es decir, trabaja a 8x el clock, tiene un bus de 352 bits.

Si una placa de video con un clock de 1750 MHz pone a la memoria GDDR6 a trabajar a 1750 multiplicado por 8 que te da un equivalente a 14000 MHz. Está entregando un bandwidth de 616GB/s. Este dato (o sea la capacidad máxima de transferencia) se obtenía multiplicando la frecuencia, por el ancho del bus dividido por 8.

$$14000 \text{ MHz} \times 352 / 8 = 616 \text{ GB/s}$$

GDDR6 trabaja a un voltaje de (1.35V).

## Disco duro

Es la unidad de almacenamiento masivo para nosotros donde la información no se pierde con ausencia de la energía eléctrica. Se encuentra el sistema operativo y todos los programas que vamos a utilizar. Utiliza un sistema de grabación magnética para guardar datos digitales.



De la parte superior vas a notar que hay un plato, que es como un espejo, si vos lo abrís y ves te podés reflejar, y es en esta superficie donde se guarda la información.

También se puede observar un brazo que está estacionado en este dispositivo porque está apagado, está en modo parking para que el disco se pueda trasladar de un lugar a otro sin que se dañen los cabezas lectoras que se encuentran en ese lugar,

Tenemos dos ejes (uno en el plato y otro en el brazo) ambos giran a velocidades elevadas pero estable, una vez que logra la velocidad escucharemos como algún soplo que está representando que el plato está girando a una velocidad muy estable, controlada por un motor que está del lado de abajo. El brazo se mueve por el actuador (la cosa gris) y cuando comienza a funcionar sale del parking y las cabezas se ponen arriba del disco duro. No hay un solo plato, pueden haber varios (ya voy a adjuntar una foto con las partes para que se entiendan) el brazo se mueve por el actuador y hace que las cabezas se paseen por arriba del disco, hacen un movimiento de pivot sobre el eje.

Estructura Interna

# Estructura Interna

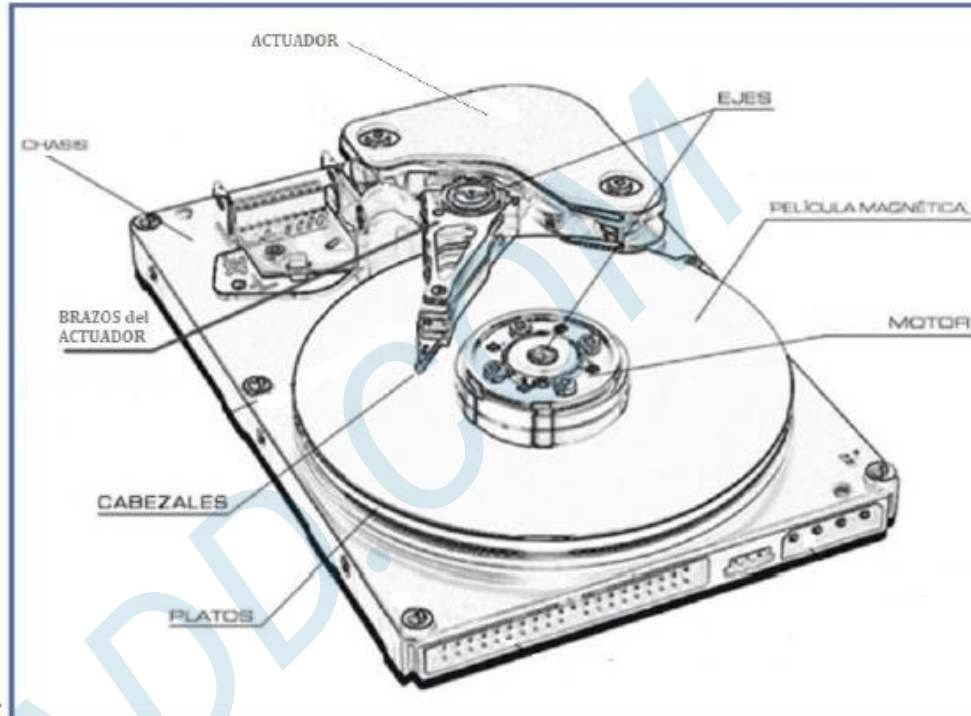
Consta de una parte mecánica:

**Carcasa:** recubrimiento metálico que protege los componentes internos de las partículas de polvo

**Platos:** pila de discos compuesto de un sustrato de cristal, aluminio o una aleación de aluminio recubiertos de una película magnética.

La información está almacenada en ambos lados de cada plato.

**Ejes:** Hay dos, uno actúa como soporte sobre el cual están montados y giran a velocidad constante los platos. El otro permite el movimiento lateral del brazo actuador sobre el cual están montadas las cabezas de Lectura/Escritura.



Hay varios platos, también varios cabezales (uno por cada plato) un brazo por un cabezal por la cara superior y después hay otro brazo por la cara de abajo, accediendo al mismo punto. Para el segundo plato habrá otro par más de cabezales de la misma forma, los cabezales se mueven controlados por el actuador, en la puntita están los cabezales que son chips electrónicos que leen o escriben datos en la superficie del disco. Hay un motor que hace girar a los platos. Hay un chasis que contiene todo el sistema, y una tapa de aluminio que tapa todo porque una partícula de polvo que caiga puede ser catastrófico porque como el disco gira a altas velocidades y trabarse esa partícula contra un cabezal puede rayar la superficie del disco y perder la información.

**Cabezales de Lectura/ Escritura:** Los datos se graban y leen en el disco a través de unas bobinas llamadas cabezas. Hay dos cabezales por cada plato.

**Actuador y brazos del actuador:** los cabezales están en el extremo de unos brazos mecánicos ubicados en el mismo eje y deslizados por el actuador (o impulsor de cabezales).

**Motor:** La velocidad de rotación varía entre los 5400 , 7200 y 10000 r.p.m.



Funcionamiento de la cabeza: van a grabar di polos magnéticos sobre las superficies, las van a orientar de modo tal que con una polaridad de esos dipolos (norte, sur) represente un 1 o un 0 de acuerdo a lo que la bobina haya grabado sobre la superficie. Todo eso es por las leyes físicas del magnetismo, se orientan los dipolos con el campo magnético. Cuando ese campo magnético pasa por debajo de la cabeza a gran velocidad crea una tensión proporcional a los 1 y 0 entonces las cabezas van a poder leer y recuperar la información que está sobre la superficie.

Mientras los discos más rápidos giran más información manejan pero elevan mucho más temperatura y hacen más ruido.

### Consta también de una parte Electrónica:

**Placa controladora:** actúa como interfaz con el sistema. Tiene posee un microprocesador dedicado que se encarga de recibir órdenes de la CPU y calcular lo necesario para leer o escribir en el disco.



Si das vuelta el disco duro vas a encontrar la controladora electrónica, con conectores para que se conecten hacia la placa madre por donde van a ir los datos, también maneja el motor y controla su velocidad exacta. Tiene un microprocesador que se ponía de acuerdo para hacer acceso directo memoria el disco duro con la memoria RAM, también tiene una memoria cache, un buffer para poner en esa memoria los datos más utilizados en disco, de forma que de golpe el usuario le vuelve a pedir algo y en vez de ir a buscar al plato se lo entrega en un pelín, ahí nomás porque ya tenía esta memoria llena de datos de los más usados haciendo astutos algoritmos. Lo importante de un disco duro es la velocidad sostenida, no la de la caché, porque ponele quieres buscar un dato que la caché no tiene y ahí ya no te sirve para nada.

### Funcionamiento de un disco duro

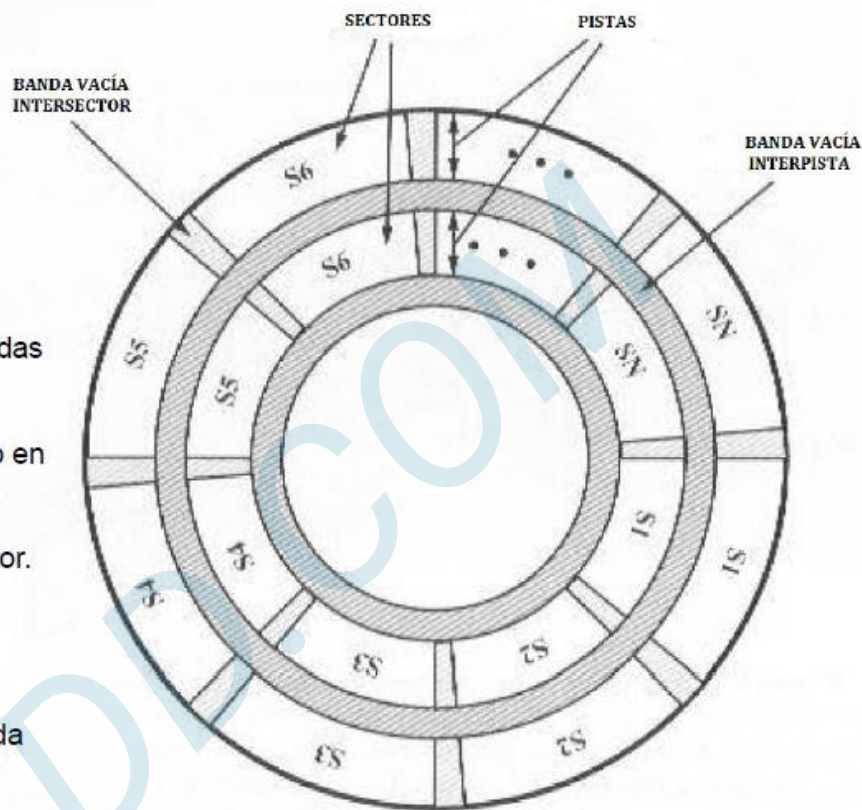
El actuador de cabezales desplaza los cabezales de Lectura/ escritura hasta el lugar donde empiezan los datos. Del modo parking se va hasta algunas de las posiciones del plato que quieres acceder. Luego se espera a que el primer dato que gira con los platos llegue al lugar donde están los cabezales, los que se mueven son los datos que están parados arriba del disco y los cabezales están quietos, solo se mueven de un lado al otro eligiendo el lugar del disco donde están los datos. Una vez que se posiciono el cabezal con el brazo actuador en un lugar fijo y se queda quieto, tiene que esperar que el dato pase por debajo del cabezal. Por último se lee o escribe el dato con el cabezal correspondiente.

Dentro de un disco duro hay varios platos, que giran todos al mismo tiempo. Los cabezales nunca tocan el disco, flotan en una capa de aire que se forman entre los platos y los cabezales, se conoce como el efecto Bernoulli, de esa forma no se desgasta ni el cabezal ni los platos. Si la cabeza toca los platos eso, o sea que "aterrizza" sobre el plato, eso se raya y entonces una línea de información se pierde con lo cual vos podes perder gran volúmenes de datos, porque si es un archivo de un programa, con lo que le dañe un poco de

bytes, ya dañaste todo un archivo. Un aterrizaje puede ocurrir en un golpe que sea sometido el disco mientras trabaja, es más frecuente que pase en una Notebook.

## Organización de los datos y formato:

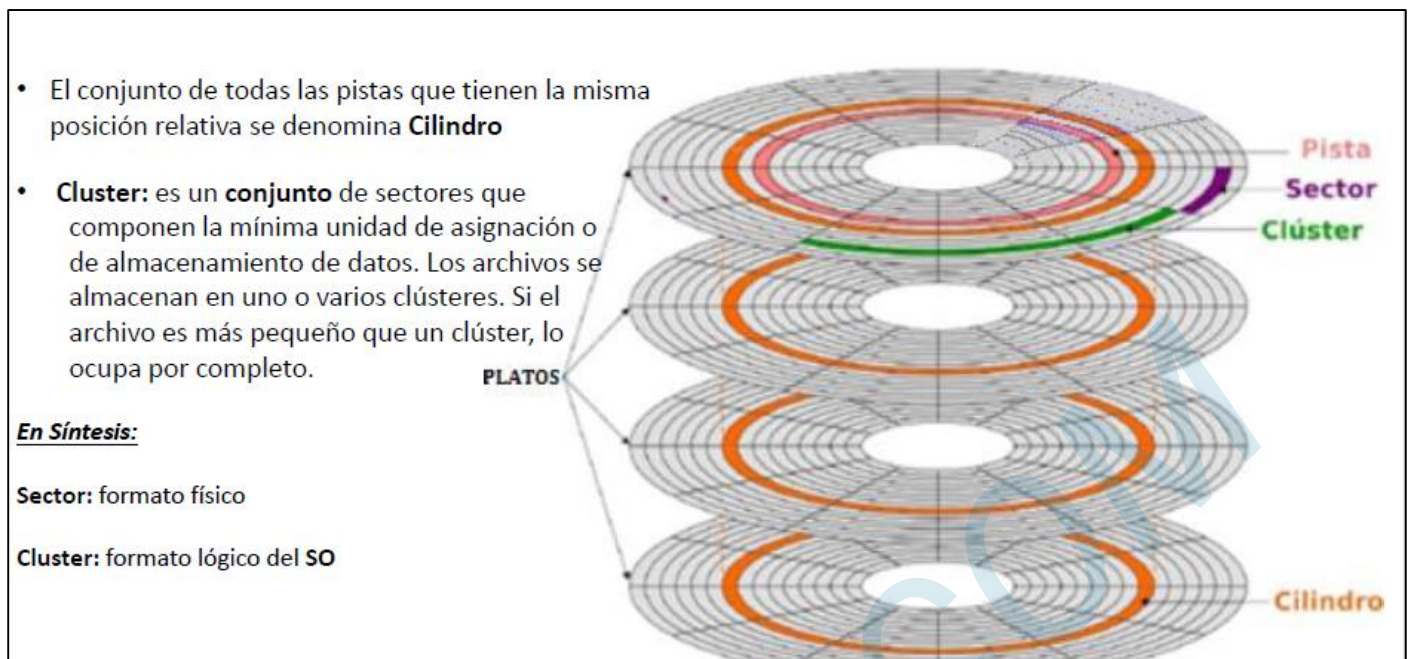
- Los **platos** tienen un conjunto de anillos concéntricos llamadas **pistas**. Cada pista es del mismo ancho que la cabeza.
- Puede haber miles de **pistas** por cara.
- Las **Pistas** adyacentes están separadas por bandas vacías.
- Los datos son transferidos hacia y desde el disco en **sectores**, hay cientos de sectores por pistas.
- 512 Bytes es el tamaño casi universal de un sector.  
 En realidad son 600 Bytes  
 - 88 Formateo  
 512 Bytes
- Los sectores se separan entre ellos por una banda intersector



Los platos o sus caras se dividen en anillos concéntricos donde se guarda la información, se deja un espacio sin usar que se llama “banda vacía interpista”, a los anillos se los conoce como pistas o track, luego se deja la banda vacía y se vuelve a generar otra pista. El tamaño de la pista es del mismo tamaño del cabezal, entonces el cabezal se sitúa justo en el medio de uno de estos anillos y va a esperar que los datos que se encuentran en los sectores pasen por debajo de la cabeza para ser leídos.

Dicho devuelta: los platos tienen un conjunto de anillos concéntricos llamadas pistas, cada pista es del mismo ancho que la cabeza, pueden haber miles de pistas por cada cara del platos, las pistas adyacentes están separadas por bandas vacías, los datos son trasferidos hacia y desde el disco en sectores. La unidad mínima que vos podes transferir es un sector y hay cientos de sectores por cada una de las pistas, cada uno de esos sectores tienen una capacidad útil de 512 bytes, este es un valor casi universal, con excepciones a la regla por supuesto. Esos 512 son útiles pero por cada segmento en realidad gasta 600 bytes ya que 88 bytes se dedican al formateo. Cuando el fabricante le da un formato se van marcando los lugares donde empieza el sector, para lo cual se gastan esos 88 bytes, entonces 512 es la cantidad limpia que te queda después de ese descuento. Si ahí no tuvieras información valiosa no sabrías donde buscar el archivo que queres buscar, por eso se gastan los 88 necesariamente, que están en el inicio de cada sector.

Los sectores se separan entre ellos por otra banda vacía llamada intersector para que la cabeza puede distinguir que termina un sector y comienzan los 88 bytes de formateo que le está indicando que viene el siguiente y así sucesivamente.



Esto es un disco de 4 platos, y 8 caras.

Pista: anillo que contiene sectores

Sectores: mínima capacidad donde vos puedas almacenar información en un disco duro

Cluster: es un grupo de dos o más sectores que forman el formateo lógico del sistema operativo, en este caso hay 3, es la división mínima que el sistema operativo puede hacer, puede considerar que un sector es muy poca información para manejar, entonces el sistema operativo trabaja con 3 sectores a la vez por ejemplo. Si tenés que guardar un archivo chiquito, el sistema va a usar un clúster, va a gastar los 3 sectores.

Todas las pistas de la cara superior del disco, de todos los discos, que tienen la misma distancia al centro, de la cara superior, misma posición relativa, más todas las naranjas que están en las caras inferiores, todas alineadas forman un cilindro. Entonces un cilindro son todas esas cantidades de pistas alineadas en su lugar relativo.

#### Parámetros para medir las prestaciones de un disco

**Tiempo de búsqueda:** es el tiempo que tarda la cabeza en posicionarse sobre una pista. Está en el orden de los milisegundos. Cuando tiene que ir de una pista a otra.

**Retardo o latencia rotacional:** es el tiempo que tarda el sector en alcanzar a la cabeza. Depende de la velocidad de rotación. Se toma como promedio la mitad del tiempo que el disco emplea en dar una revolución. Si una vez que vos te paraste arriba de la pista pueden pasar dos cosas: que tengas suerte de que el dato justo estaba viniendo y lo agarraste justo, o que el dato ya haya pasado y tengas que esperar que el disco de otra vuelta para que el cabezal lo pueda agarrar.

El tiempo que tarda en dar una vuelta es 4 milisegundos, pero por las dudas teniendo en cuenta los casos anteriores haremos un promedio de 2 milisegundos. Es esperar que el dato pase por debajo del cabezal

**Tiempo de acceso:** desde que le das la orden de buscar un dato hasta que lo tiene, es un parámetro que sirve para comparar un disco contra el otro. El tiempo de acceso es la suma del tiempo de búsqueda más el retardo rotacional.

**Tiempo de transferencia:** es el tiempo que tarda desde que levanta el archivo hasta que lo pone en la memoria principal, en la RAM dinámica. Depende del largo del archivo. Se calcula como que ese tiempo de transferencia viene dado por la cantidad de bytes a transferir, que es el tamaño del archivo, obviamente está en el numerador, en el denominador está la cantidad de byte que trasladas por unidad de tiempo, que se mide en unidad de rotación y n es la cantidad de bytes de una pista.

$$T = \frac{b}{rN}$$

$b$  = Nº de Bytes a transferir.

$N$  = Nº de Bytes de una pista

$r$  = velocidad de rotación en vueltas por segundo.

Por lo tanto el tiempo de acceso medio total se expresa como:

$$T_a = T_{búsqueda} + \frac{1}{2r} + \frac{b}{rN}$$

Será más rápido si hay más velocidad de rotación o es más chico el archivo.

Esa podría ser la velocidad que te importa, no la del caché. El tiempo de acceso total  $T_a$  se expresa como el tiempo de búsqueda + el retardo rotacional + tiempo de transferencia.

## Interfaces y velocidades

- IDE Integrated Device Electronics 16 a 166 MB/seg
- SATA Serial Advanced Technology Attachment max 600MB/seg
- SCSI Small Computer System Interface
- SAS Serial Attached SCSI
- USB Universal serial Bus typ 25-40MB/seg

## IDE: Integrated Device Electronics

Conector de 40 pines a la placa madre (bus de datos) + conexión de 4 pines para conexión a la fuente de alimentación.  
Velocidad entre los 16MB/seg y los 166 MB/seg.



El ide lo que tenía de bueno era que se podía poner dos unidades en cada bahía, en total cuatro dispositivos podían ser manejados, aunque ya no se usa.

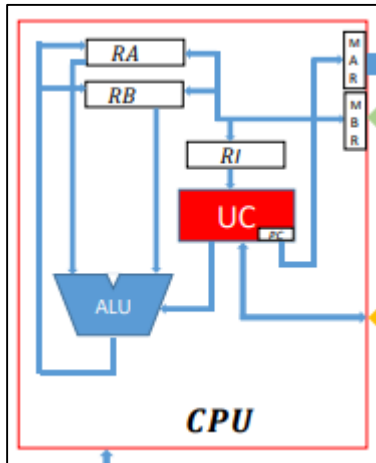
## SATA: Serial Advanced Technology Attachment

Conector de datos de 7 pines + conexión a la fuente de alimentación. Velocidad entre los 150MB/seg y los 600 MB/seg



La información viaja a través de los pines, donde en cada uno viaja un chorro de bits en forma serial, seguramente en forma de SISO que van impulsando los datos uno a continuación de otro, ingresando el conector en la placa madre, mueven datos entre 150 a 600 Mb /Seg.

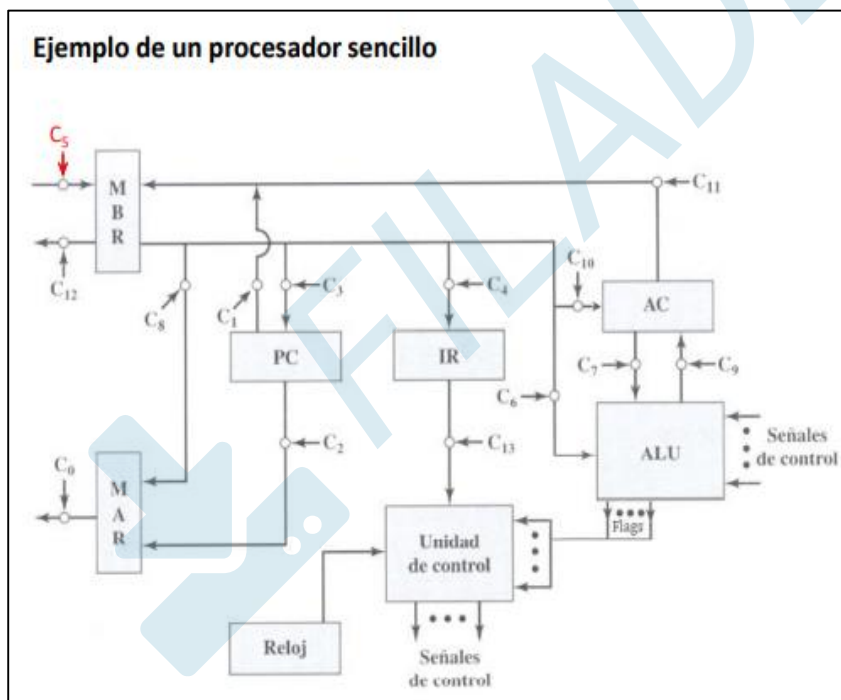
## Implementación de la unidad de control



En la arquitectura de Von Neumann se hablaba de la UC y también de la unidad de ejecución, esta última se encargaba de tomar la instrucción que se alojaba en el registro de instrucciones y mover los hilos internos del procesador, o sea en la CPU para manejar la ALU o para hacer las operaciones que el registro le dice que haga.

Ahora nos toca hablar de la Unidad de Control, que tiene dos formas de implementación. La más antigua es la forma de implementación cableada y después la otra es la unidad de control microprogramada. La cableada son usadas por los teléfonos celular y la microprogramada la usan los procesadores de INTEL.

La implementación cableada es una implementación que es esencialmente un circuito combinacional, las señales de entrada a ese circuito combinacional son el **Registro de instrucción (IR)**, **CLK** (clock o reloj) lo usa como una señal que le dice: "ahora ese este cálculo, ahora este otro" y así da referencias de las etapas de ejecución. También vamos a tener en el circuito combinacional en la unidad de control que le entran los **indicadores** que son las banderas (flags) que son los registros de estado que utiliza para tomar decisiones, y las **salidas** son señales de control que son 1 o 0.

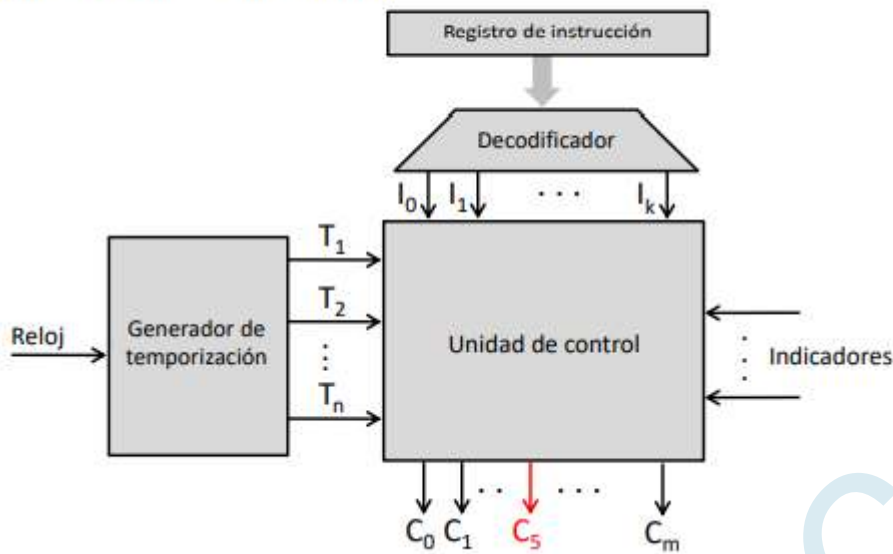


Este procesador tiene una unidad de control, como entrada tiene el registro de instrucción. Desde la ALU salen las banderas para ingresarlas a la unidad de control, y por ultimo este circuito recibe lo que le manda el reloj, dando como salida las señales de control, que manejan montones de llaves (círculos blancos que se ven en la imagen), siendo las líneas negras los buses donde viajan datos o las instrucciones. Esto sería como una canilla, si le pones 1 se cierra, permitiendo pasar información, caso contrario se abre y no permite el paso. La unidad de control de forma muy astuta va a ir creando las señales de control ordenadamente para hacer que la información se mueva ágilmente adentro de todo el procesador sin producir colisiones en los buses.

Las señales de control son aquellas que la unidad de control maneja tanto adentro del procesador como en los buses externos para lograr el movimiento de datos e instrucciones adentro del interior y también manejar a la ALU y los registros internos, conectándolos o desconectándolos con las llaves que se habían mencionado anteriormente.

Eso fue a groso modo, ahora veremos esto más a profundidad para entender mejor el funcionamiento de una unidad de control cableada centrándonos en la llave C5 (está en rojo) en el dibujo anterior).

### Al detalle: Lógica de la unidad de control



En el registro de instrucción se alojará la instrucción. Entonces como vos tenés un set de instrucciones vas a poner un decodificador, porque para una entrada dada prende una y solo una de las salidas, entonces la unidad de control se dará cuenta fácil cual es la instrucción que se quiere resolver. Supongamos que es la instrucción numero 1, el reloj tiene un generador de temporización que cada vez que viene un pulso va prendiendo uno a uno cada una de las Ts que tiene como salida, cuando el reloj haya

dado n pulsos, habrá terminado de completar la instrucción, entonces una instrucción cualquiera no se hace en un solo pulso, sino en varios porque hay que abrir y cerrar muchas llaves para lograr lo que es la instrucción. Si es una suma por ejemplo se va a ver alterada por los indicadores, por eso también son entradas a este circuito combinacional.

La salida (C5) se implementa primero definiendo dos señales de control nuevas llamadas P y Q que tendrán la siguiente interpretación.

PQ = 00 Ciclo de captación  
PQ = 01 Ciclo indirecto  
PQ = 10 Ciclo de ejecución  
PQ = 11 Ciclo de interrupción

$$C_5 = \bar{P} \cdot \bar{Q} \cdot T_2 + \bar{P} \cdot Q \cdot T_2 + P \cdot \bar{Q} \cdot (LDA + ADD + AND) \cdot T_2$$

Esto resume en que tiempo del reloj se encuentra, porque toda instrucción consta de 4 etapas.

Primero está el ciclo de captación, donde tengo que ir y buscar la instrucción para ponerla adentro del registro.

Luego sigue el ciclo indirecto que es ir a la memoria (no siempre se hace, por ejemplo, en las instrucciones con modo de direccionamiento implícito).

Después viene el ciclo de ejecución donde la instrucción hace lo que se supone debe hacer

Por último, se puede prestar el ciclo de interrupción, en caso de aparecer alguna de las interrupciones enmascarables o no ya vistas anteriormente.

La salida C5 dice que si estas en el ciclo de captación (P y Q negados) y cuando vaya a ser el tiempo 2 (ya que el tiempo 1 es cargar el programa para seleccionar donde está la instrucción) hay que cerrar la llave 5 porque seguro el tipo va a traer la instrucción. Para el resto de los términos se aplica lo mismo, se tiene que pensar en qué distintos momentos se abre y se cierra la llave.

Esto es muy retardado, pero cuando está hecha esto vuela porque no hay nada más rápido que un circuito combinacional tomando valores y dando la respuesta a la salida, por eso fue elegido, no por su grado de dificultad, ya que hay que hacer tantas situaciones como señales de control existan.

Los microprocesadores que usan este tipo de aplicación de la unidad de control se los conoce como RISC (Reduced Instruction Set Computer) son computadoras de un set de instrucciones reducidas. Usan pocas y simples instrucciones.

Los micros de INTEL son todo lo contrario, ponen un set de instrucciones gigantesco y esperan que los compiladores sean inteligentes y después usan adentro las mejores de las instrucciones. Por eso los micros RISC toman la delantera porque agarran un set de instrucciones sencillo ya definido y llenan con muchos registros internos, y como ya está todo definido las instrucciones que entran se ejecutan rapidísimo y los compiladores no tienen que pensar mucho.

### Implementación microprogramada

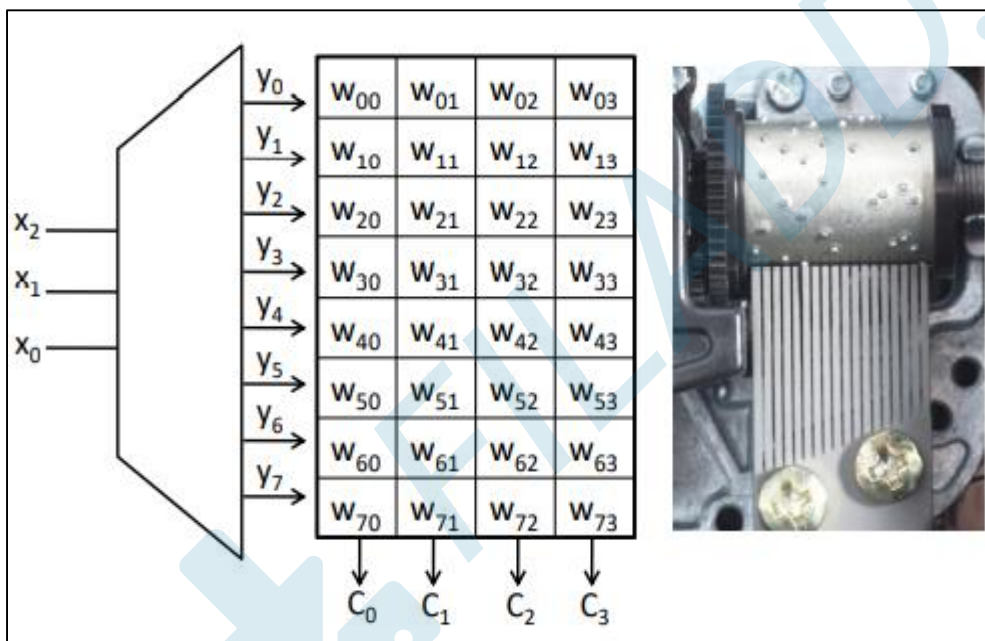
Es una alternativa a la unidad de control cableada.

Una microinstrucción es una instrucción muy elemental que especifica microoperaciones.

Un conjunto de microinstrucciones que permiten la ejecución de una sola instrucción de máquina se llama microcódigo, vos agarras una instrucción que habías estudiado y que vos pensaste que era lo más indivisible que había, estabas equivocado, ya que esta a su vez se divide en muchas microoperaciones, y entonces un conjunto de microinstrucciones hacen un programado llamado microcódigo y este microcódigo es lo que conocemos como la instrucción de máquina, por ejemplo, MOV. Una microinstrucción por ejemplo es cerrar una llave. De muchas de estas microinstrucciones se hace una instrucción de máquina.

Como en la unidad de control cableada las señales de control generadas por una microinstrucción se usan para producir transferencias entre registro y operaciones en la ALU.

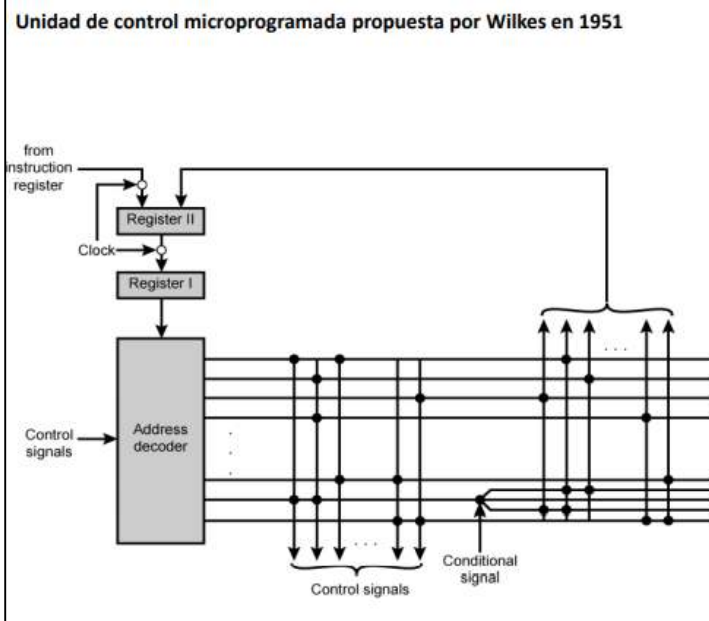
### Memoria ROM



La ROM tiene un decodificador, con unas líneas de direccionamiento para poder seleccionar cuál es la palabra que se quiere leer. En la foto se ve una palabra de 4bits.

Utiliza una memoria llamada memoria de control, donde por ejemplo se pone 0 0 0 en las líneas de entrada y se selecciona la primera palabra y si en ese momento para esa microinstrucción tenía que prender el C3 se clava un 1 en la memoria y le va a poner un 0 a todos los otros, entonces no se quiebra la cabeza como al armar un circuito combinacional. De esa forma irá recorriendo una a una las líneas.

Esto fue propuesto por Wilkes en el año 1951, alegando que si pudiéramos tener una memoria de control donde un montón de bits pudieran ser manejados a un tiempo y estos fueran manejados por unos registros que le van haciendo secuencialmente barrera una a una las líneas tendríamos otra forma de implementar una unidad de control distinta a las cableadas.

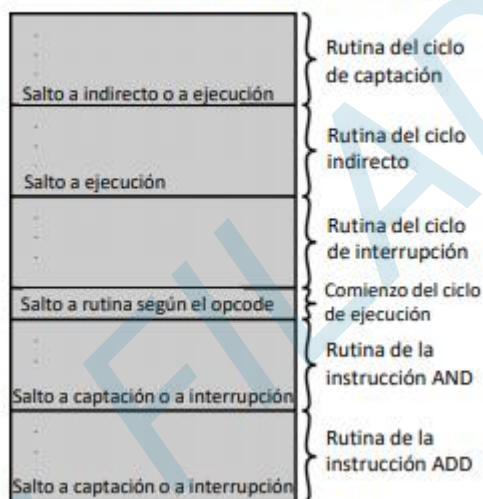


Ahora veamos cómo serían los ciclos de instrucción con esta forma de implementación.

### Ciclo de instrucción

Captación – Indirecto – Ejecución - Interrupción

#### Organización de la memoria de control

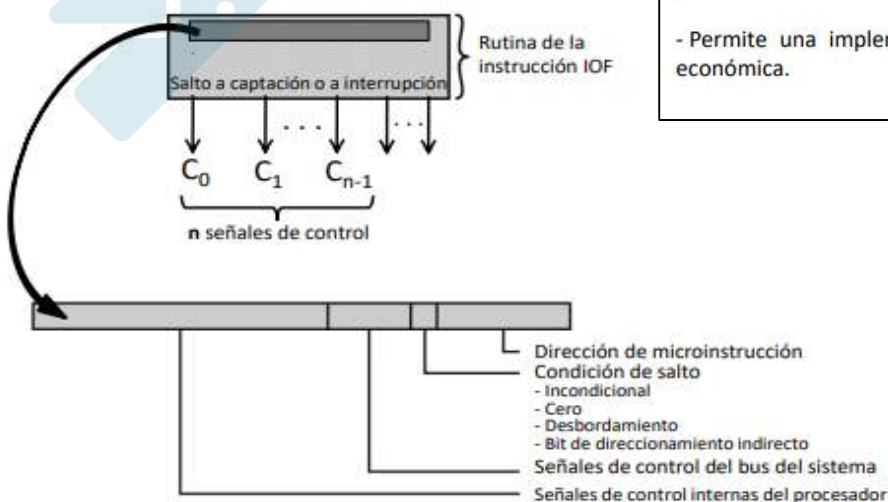


Con este sistema le podés poner un set de instrucciones gigantescos, se los conoce como arquitectura CISC (Complex Instruction Set Computer) Computadoras de set de instrucciones complejos. Esta es una máquina secuencial.

### Resumen

#### Microprogramación

- Es una alternativa a la unidad de control cableada. Provee mayor flexibilidad en el diseño de la unidad de control.
- Permite corregir errores en el microcódigo mediante parches (como en el parche crítico para Intel C2D en 2007).
- No es tan veloz como la implementación cableada. Típicamente, los procesadores RISC utilizan implementación de unidad de control cableada.
- Permite brindar un mejor time-to-market en la producción de nuevos procesadores.
- Permite una implementación más sencilla y por consiguiente más económica.



Formato de microinstrucción típico

## Periféricos

### Monitores

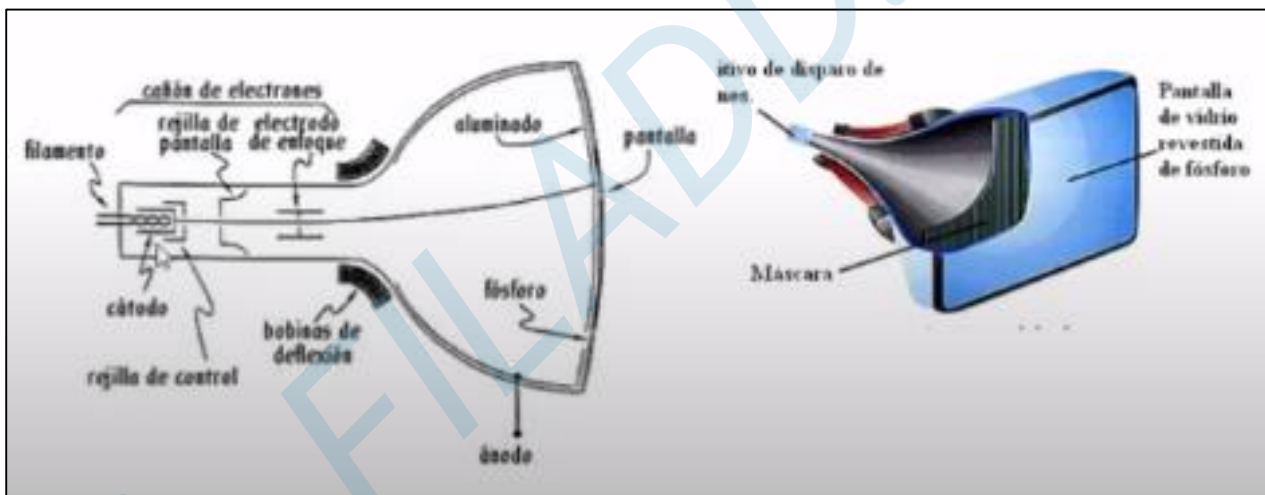
El monitor es la pantalla donde se ve la información suministrada por la computadora mediante un interfaz, muestra los resultados de un proceso, una aplicación que estemos ejecutando, pero antes los monitores que existían eran de tubos de rayos catódicos (CRT) y anteriormente eran del tipo monocromático. Hoy en día la tecnología empleada es la de monitores por diodos de emisiones de luz (LED).

La información se representa mediante pixeles, un pixel es la unidad mínima de información para la luz, representada en un monitor. Cada pixel se enciende con un determinado color para formar la imagen, cuando mayor cantidad de pixeles puedan ser representados en una pantalla mayor resolución se tendrá.

#### Monitor CRT

Están basados en un tubo de rayos catódicos, funciona por medio de un cañón que dispara constantemente hacia la pantalla rayos de electrones. La pantalla está cubierta de fósforo, que se ilumina al entrar en contacto con los electrones. Para poder prender estos puntos de fósforos hace un barrido constante de electrones, entonces por eso cuando estábamos al frente de estaba pantalla éramos expuestos a la radiación de estos electrones.

No veíamos imágenes, sino solo texto en las primeras versiones de estos tipos de monitores. De esta manera iluminando cualquier punto con diferentes intensidades de puede obtener un color (modelos posteriores de este monitor).



#### Monitor LCD

Este tipo de tecnología se la conoce como cristal líquido, el funcionamiento es una pantalla de cristal líquido la cual tiene una estructuración delgada y plana ya que no necesitamos electrones, tampoco hay radiación. Cuenta con un soporte de color como gama de colores y con un retardo de tiempo en mostrar una imagen en lo que dura un pixel para cambiar de color.

#### Monitor LED

Es el mismo monitor que el LCD, funciona con cristal líquido, cuando recibe electricidad se ilumina un pixel. Un led por sí solo no ilumina demasiado, pero muchos de ellos pueden dar una luz pura y eficaz. Están formados por un conjunto de LED que al recibir un impulso eléctrico se iluminan. Tienen muchas ventajas ya que no se funden como ocurren con los pixeles, permiten fabricar pantallas extremadamente planas y de mucha calidad.

## Impresoras

Es un dispositivo periférico de la computadora que permite producir textos o gráficos de documentos almacenados en un formato electrónico imprimiéndolos en medios físicos, generalmente papel, utilizando cartuchos de tinta o tecnología láser.

Hay que tener en cuenta la **velocidad de impresión** que se mide en **CSP (Caracteres por segundo)** cuando es una matriz de punto y a chorro de tintas lentas y cuando es una impresora láser y a chorro de tintas rápidas se tiene en cuenta la **PPM (Paginas por minutos)**.

**La resolución de una impresora es la cantidad de puntos que puede imprimir en una pulgada PPP o DPI**

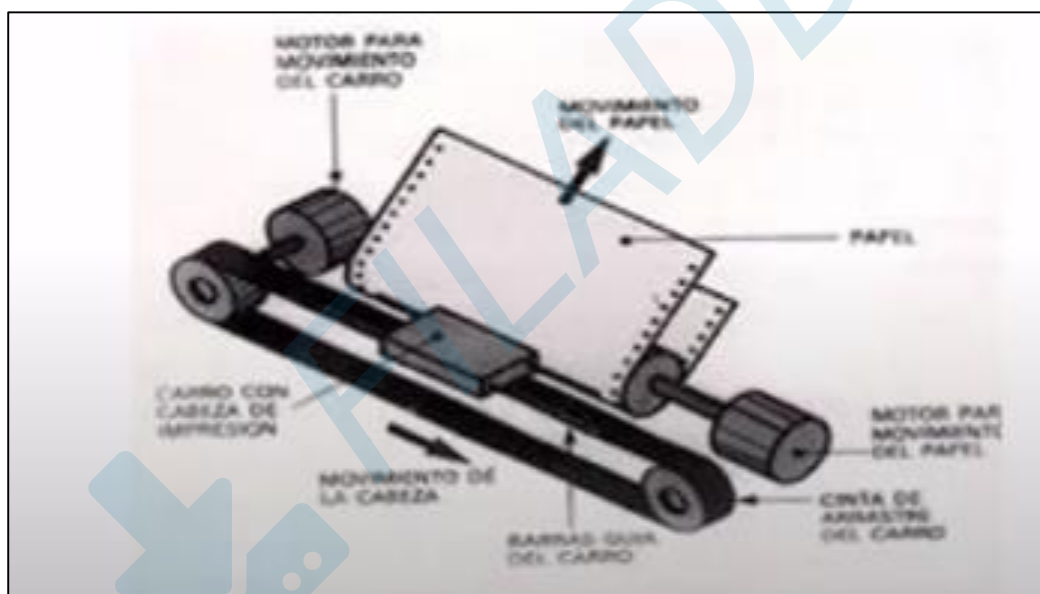
### Tipos de impresoras

#### Impresora de matriz de puntos (o matricial)

Tiene un cabezal, que tiene agujas incorporadas y son impulsadas hacia una cinta, empujando la cinta que impacta contra el papel dejando un punto impreso retornando a su posición de reposo.

Tiene dos motores, uno conectado a la correa que se encarga de desplazar el cabezal en forma horizontal (mover el papel en forma horizontal para ver si está imprimiendo) y otro que hace un movimiento vertical.

La precisión de este mecanismo y el tamaño del punto generado por el impacto de la cinta impulsada por la aguja definen la resolución horizontal máxima de la impresora.

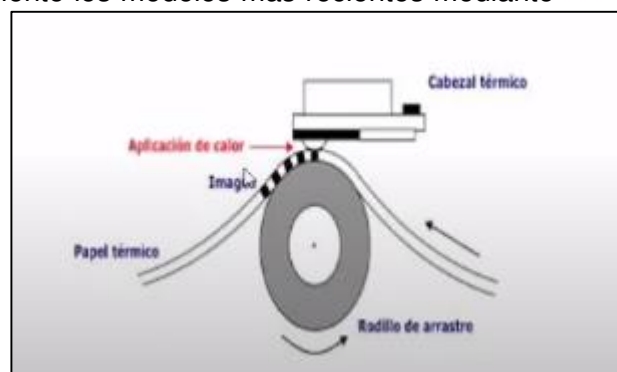


Bueno si le ponen onda se entiende lo que dice la foto.

Estas impresoras son lentas y con una resolución muy baja.

#### Impresora térmica

Solo posibilita la impresión en monocromo color negro, y únicamente los modelos más recientes mediante un papel especial adicionan el rojo y el azul. Los costos son muy bajos ya que no consume más que el propio papel, no hay tinta. La velocidad de impresión puede tener en este caso en mm/s refiriéndose a los milímetros de rollo de papel que salen de la impresora. El cabezal es térmico, la aplicación es por calor, es la que suelen estar en los colectivos y los cajeros automáticos. La durabilidad de estas impresoras es relativamente baja puesto que el desgaste que tiene el papel, en particular en temperaturas altas, hace que se pierda el texto o imagen escrito en el mismo.

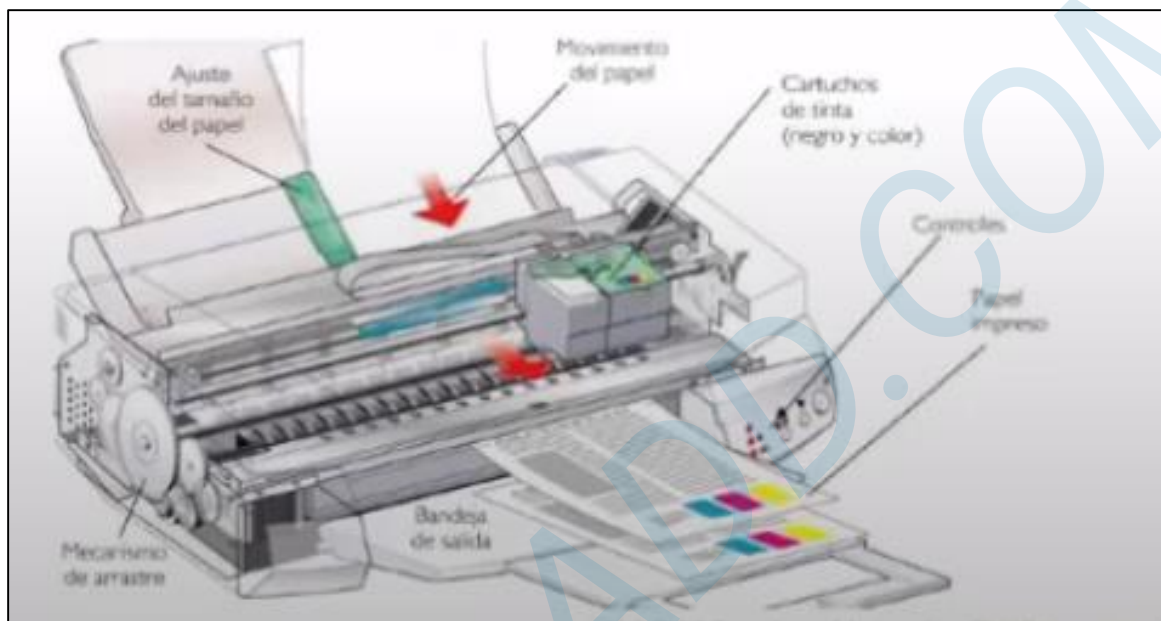


## Impresora a chorro de tinta

Las impresoras a chorro de tinta utilizan la técnica de pulverizar pequeñísimos chorros de tinta directamente sobre el papel y forman los puntos de la impresión. El cabezal de estas impresoras produce un movimiento similar al de las impresoras matriciales, es decir, un motor se encarga de transmitirle el movimiento mediante una correa en forma horizontal, y otro mecanismo se encarga de desplazar el papel en forma vertical.

El cabezal está compuesto por varios inyectores, que son tubitos por donde pasa la tinta, apuntan al papel que dejan pasar pequeñísimas gotitas para que se estampe contra el papel.

La resolución de esta impresora varía entre 300 y 1440 dpi, pero no se llega a cumplir con esta resolución en papel común porque cuando se estampa contra el papel la gotita se agranda, perdiendo resolución.



## Impresora láser

Hace una impresión fotomecánica similar a la fotocopidora, la resolución es mucho más alta que las anteriores. El método de impresión que utilizan las impresoras láser es el fotomecánico y es similar al que se utiliza en las fotocopadoras, brinda una excelente resolución, nitidez y altas velocidades de impresión.

La impresora recibe la información a imprimir y la guarda en su propia memoria. A continuación, un cilindro de cristal (cilindro de impresión) con un recubrimiento especial, generalmente silicio, se carga con electricidad mediante un anillo de alambre del grosor de un cabello (anillo coronario o corona).

La luz de un rayo láser se desvía mediante un sistema de espejos de altísima precisión para que el rayo se proyecte en los puntos del cilindro de impresión que se desean descargar, estos puntos del cilindro descargados serán los que formarán la imagen en el papel.

Finalmente, el tóner (un polvo extra fino de óxido de metal y pequeñísimas partículas plásticas) se carga eléctricamente de manera que solamente se adhiera a la superficie del cilindro que recibió la proyección del láser. El cilindro va girando y así se va formando en este la imagen que debe trasladarse al papel.

El papel se carga con electricidad con signo opuesto a la usada para cargar el tóner. Mientras el cilindro gira, las partículas del tóner que se habían posado en este, son atraídas por las cargas opuestas al papel y se depositan en este abandonando el cilindro. Para fijar el tóner al papel, se hace pasar al papel por un rodillo caliente que funde el tóner contra el papel permanentemente.

El fusor de una impresora láser se encarga de fijar el polvo de tóner en el papel mediante la aplicación de calor a través de dos rodillos. Compuestos por rodillo cerámico de calor y rodillo de presión.